FINAL-YEAR ENGINEERING MAJOR PROJECT · COMPLETE TECHNICAL GUIDE

SmartAttend AI

The complete A to Z guide — every layer explained from first principles, for a reader who has never built a web application before.

SmartAttend AI is a web application that records class attendance using facial recognition, with manual alternatives, human review for every uncertain case, and a privacy model that is honest about what it does. This document explains *what* the system is, *why* every technology was chosen, *how* the code is organised, and *exactly how it works* from the moment a student sits down to the moment a report is exported — in plain language, with every technical term defined where it first appears.



Project
SmartAttend AI

Author
Abhishek Mishra

Stack
React · TypeScript · FastAPI · PostgreSQL

Licence
MIT

Repository
github.com/Abhishek-stacked/smartattend-ai

CONTENTS

What is in this guide

Nineteen chapters, written to be read in order. Every chapter builds on the one before it, and every new idea is defined before it is used.

FOUNDATIONS — READ THESE FIRST

1 How to read this guide
2 The words you will meet (glossary)
3 What a web application actually is
4 What SmartAttend AI does

THE SHAPE OF THE SYSTEM

5 Architecture: the three layers
6 The repository, folder by folder

THE LAYERS IN DEPTH

7 The front end — what it is and why
8 The front end, in detail
9 The API — what it is and why
10 The back end, in detail
11 The database
12 The AI service — face recognition and liveness

PUTTING IT TOGETHER

13 Security and privacy
14 End-to-end journeys, step by step
15 Testing
16 Running it: development and deployment

CLOSING MATERIAL

17 How this project was built
18 Limitations, honest answers and next steps
19 Appendices

A note on the code in this guide

Snippets are real excerpts from the project, trimmed to the lines that matter. Where a file is long, the path is given so you can open it. Everything shown here exists in the repository at `github.com/Abhishek-stacked/smartattend-ai`.

CHAPTER 1

How to read this guide

Before a single line of code, a short note on how this document is organised and how to get the most out of it.

Who this is for

This guide assumes you can use a computer and you have maybe seen a website before, but that you have **never written a web application**. It does not assume you know what an *API* is, what a *database* is, what *React* is, or why a *container* exists. Every one of those is explained in the place where it first matters.

If you already know some of this, you can skip ahead — each chapter is largely self-contained, but they are written so that reading them in order costs you very little.

Three kinds of text

Three visual styles appear throughout. They are always used for the same thing, so you can read the page without reading every word:

DEFINITION

A concept that is being introduced for the first time, or a term whose precise meaning matters. Blue-violet boxes like this one always mean: "*this is what the word actually means*".

SIDE NOTE

Extra context, a trade-off, or something that surprised us while building. Cyan boxes are commentary — useful, but you can skim them on a first read.

CAREFUL

Something that is easy to get wrong, or a limitation you must know about before you trust a result. Amber boxes are the ones worth reading twice.

WHAT WORKS

Something that was actually built, measured or verified. Green boxes tell you what is real as opposed to what is planned.

Status labels

Throughout the project's own documentation, features carry a status label so that nothing is over-claimed. This guide uses the same labels, and you will see them in the code comments too:

Label	Meaning	What you can rely on
Implemented	Written, wired up and tested.	The code exists and the test suite covers it. It is not a mock-up or a stub.
Experimental	Written and running, but not yet validated for accuracy.	It works mechanically. No accuracy claim is attached to it. In this project the liveness detector is the experimental part.
Planned	Designed but deliberately not built yet.	Nothing. It is on the roadmap, not in the product.

> **Why this fussiness?**
>
> Because a system that records who is in a classroom carries consequences. If the system says "94% accurate" and nobody measured it, that is a lie told by software. SmartAttend AI would rather report nothing than report something untrue, and this guide follows that rule: where a number has not been measured, the guide says so.

A suggested route

1. **Nothing technical yet.** Read chapters 1 to 4. You will end up knowing what the system does and how the web works, in plain language.
2. **Meet the shape.** Read chapters 5 and 6. You will learn how the code is laid out and why it is laid out that way.
3. **Go deep, layer by layer.** Chapters 7 to 12 walk through the front end, the API, the back end, the database and the AI code. Read these in order — each one assumes the previous.
4. **See it working.** Chapters 13 to 16 cover security, the real user journeys, testing and running the thing.
5. **Understand the decisions.** Chapters 17 to 19 explain why the project looks the way it does, what it cannot do, and where the reference material lives.

Throughout, file paths are shown relative to the repository root, for example `backend/app/core/security.py`. Lines are occasionally given as `file.ts:42` so you can jump straight there in an editor.

CHAPTER 2

The words you will meet

Every technical term this guide uses, defined in plain language before it is used anywhere else in the document.

If you read nothing else in this chapter, read the two sections on **how the web works** and **how a database works** — those are the two ideas everything else rests on.

2.1 The web, from the bottom up

Client

The program on *your* machine that talks to a server. In this project the client is the web browser (Chrome, Edge, Firefox) running the React application.

Server

A program that listens on a network port and answers requests. In this project there are three servers: the web server (nginx/Caddy) that serves the interface, the API server (FastAPI) that does the work, and the database server (PostgreSQL) that stores the data.

HTTP

The protocol (set of rules) a browser uses to ask a server for things and for a server to answer. It is a request/response protocol: the browser sends a request, the server sends back a response. Every page load, every button press and every API call is one of these.

Request

One message from client to server. It carries a **method** (what you want to do), a **path** (which resource), **headers** (metadata such as cookies and content type) and sometimes a **body** (the data itself).

Response

One message back from server to client. It carries a **status code** (a number such as 200 or 403) and usually a body.

HTTP methods / status codes

`GET` reads, `POST` creates or triggers, `PATCH` updates, `DELETE` removes. Status codes: `200` OK, `201` created, `204` no content, `400` rule violated, `401` not signed in, `403` signed in but not allowed, `404` not found, `409` conflict, `413` upload too large, `422` input failed validation, `429` too many requests, `503` a dependency (such as a model) is unavailable.

HTTPS

HTTP wrapped in encryption (TLS). It stops anyone between your browser and the server from reading or altering the traffic. The little padlock in the address bar means HTTPS is in use.

URL

A web address, such as `https://attendance.example.edu/attendance`. The first part is the **origin** (scheme + host + port); the rest is a **path**.

REST API

A convention for designing HTTP interfaces so that URLs name resources and methods say what to do with them. `POST /api/sessions/12/start` means "start session 12". REST is a style guide, not a technology.

JSON

(JavaScript Object Notation) The text format the two halves of this project use to talk to each other. It looks like nested dictionaries: `{"id": 12, "name": "Asha Rao"}`. Any language can read and write it, which is why it is used for web APIs.

Multipart form data

The format used when uploading a file (a camera frame) together with text fields. The browser assembles it; the server splits it apart again.

Cookie

A small piece of text the server asks the browser to store and then send back on every later request to that site. This is how the user stays signed in without the server keeping any sensitive information in the page itself.

Cookie flags: HttpOnly, SameSite, Secure

`HttpOnly` means JavaScript in the page cannot read the cookie, so a cross-site scripting bug cannot steal it. `SameSite=Lax` stops the cookie being attached to requests triggered by other sites. `Secure` means "only ever send this over HTTPS".

Session

A logged-in state that lasts across requests. The server issues an opaque random token, puts it in a cookie, and remembers the token's hash. Every later request carrying that token is treated as the same user.

CSRF (Cross-Site Request Forgery)
An attack where another website makes your browser send a state-changing request to a site you are signed in to. The defence used here is a *double-submit token*: a value stored in a second cookie that must be echoed back in an `X-CSRF-Token` header. Another site can make the browser send the cookie, but it cannot read it to build the matching header.

CORS (Cross-Origin Resource Sharing)
The browser rule that decides whether page A may call page B's API. When the front end and the API are served from the same origin, no CORS permission is needed at all.

Origin
Scheme + host + port together. `http://localhost:8080` and `http://localhost:5173` are different origins even though the host name is the same, because the ports differ.

2.2 Programming languages and their tools

TypeScript (TS)
JavaScript with types added. Types are checked *before* the program runs, which catches a whole class of mistakes (passing a string where a number is expected) at build time instead of surprising you at runtime. The browser still runs plain JavaScript; TypeScript is compiled away.

JavaScript
The language browsers run. Every website you have ever used is ultimately running it.

Python
A readable, general-purpose language. This project's back end, AI service, migrations and test suite are all written in it.

Node.js
Node.js is a JavaScript runtime for servers and tooling. It lets JavaScript be used outside the browser — here it powers the build tool, the test runner and the package manager.

npm
The package manager that ships with Node.js. `npm install` downloads the libraries a project declared in `package.json`.

pip
Python's package manager. This project uses `pip install -r requirements.txt` to install FastAPI, SQLAlchemy, OpenCV and the rest.

Virtual environment (venv)
An isolated copy of Python plus the packages you install for one project. It stops project A's library from breaking project B. The path used throughout this guide is `backend/.venv/Scripts/python.exe`.

Library / package / dependency
Code written by someone else that you import and call. "Dependency" is the same thing viewed from the project that needs it.

Compiler / transpiler
Turns source code into something runnable. TypeScript is transpiled to JavaScript; Vite bundles many JavaScript files into a few.

2.3 Front-end words

Front end
Everything the user sees and touches: layout, buttons, forms, charts, the camera preview. In this project it is a React application that runs entirely in the browser.

Back end
Everything behind the scenes: the API, the business rules, the database, the AI models. The user never sees it directly.

SPA (Single-Page Application)
A website that loads one HTML page and then swaps parts of it in JavaScript as the user navigates, without reloading the whole page. It feels like a desktop app rather than a stack of linked documents.

React
A JavaScript library for building interfaces out of reusable pieces. A **component** is a function that returns markup; React keeps track of which components need to re-render when data changes.

Component
One reusable piece of interface. This project has 12 reusable UI components (`frontend/src/components/ui/`) and 19 pages built from them.

Props
The values a parent component passes down to a child, like function arguments.

State
Data a component owns and can change. When state changes, React re-renders the parts of the screen that depend on it.

Hook
A built-in function that lets a component use React features such as state or timers. This project defines its own hooks, for example `useCamera`, which wraps the browser camera API.

Context
A way to share a value (such as the signed-in user) across the whole component tree without passing it through every level. This project uses it for authentication state.

JSX
JavaScript that looks like HTML. It is a syntax extension that TypeScript compiles into plain JavaScript calls.

Routing
Mapping a URL to a screen. In this project React Router maps `/attendance` to the attendance page, and route guards decide who may see it.

Route guard
A wrapper that checks the user's role before rendering a page, and redirects to the login screen if they are not allowed. Implemented in `frontend/src/components/RequireAuth.tsx`.

CSS
(Cascading Style Sheets) The language that controls how things look: colour, spacing, size, shadow, animation.

Tailwind CSS
A utility-first CSS framework. Instead of writing named classes like `.card`, you put small utility classes directly in the markup: `class="rounded-xl p-4 shadow-lg"`. Version 4 defines the design tokens in CSS using an `@theme` block, which is what produces the `bg-brand-500` and `shadow-lift` classes used across the app.

Design token
A named value in the design system — a colour, a shadow, a font — defined once and reused everywhere so the interface stays consistent. SmartAttend AI defines an indigo brand ramp, an aqua accent ramp, and five depth shadows.

Vite
The development and build tool for the front end. In development it starts a server and refreshes the browser as you edit; for production it bundles and minifies the app into static files.

Recharts
A charting library for React. It draws the dashboard's line, bar and pie charts from plain data arrays.

Build
The step that turns source files into something shippable: `npm run build` runs the TypeScript checker and then Vite's bundler.

Bundle
The single (or few) JavaScript files Vite produces from many source files, so the browser downloads less and parses faster.

Dev server / proxy
During development the front end runs on port 5173 and forwards `/api` calls to the back end on port 8000. The "proxy" is just a relay so the browser sees one origin and no CORS complications arise.

2.4 Back-end and API words

API
(Application Programming Interface) A documented contract for how one program may ask another program to do things. Here it is a set of HTTP endpoints under `/api`.

FastAPI
A Python web framework for building HTTP APIs. It validates input, converts Python objects to JSON, and generates interactive documentation at `/api/docs` automatically.

ASGI / Uvicorn
FastAPI speaks an async protocol called ASGI; Uvicorn is the server program that runs it. Async means one process can interleave many requests instead of blocking on each one.

Pydantic
A Python library for describing data shapes and validating them. Each request body, query parameter and response is declared as a Pydantic model, so bad input is rejected at the edge of the system with a clear error.

Validation
Checking that input matches the declared shape before the rest of the code sees it — required fields present, types correct, lengths and ranges respected.

Dependency injection (FastAPI `Depends`)
A way to say "before this endpoint runs, do these things and give me their results". Used here for authentication, role

checks, database sessions, pagination parameters and rate limiting.

ORM (Object-Relational Mapper)
A layer that lets you work with database rows as Python objects instead of writing SQL by hand. SQLAlchemy is the ORM used here.

Session (database session)
Not to be confused with a login session. A database session is a short-lived "unit of work" that holds a transaction and a connection to PostgreSQL.

Transaction
A group of database operations that either all succeed or all roll back. Writing an attendance record and its recognition event together is one transaction: the interface never says "recorded" unless both rows really landed.

Commit / rollback
Making a transaction permanent, or undoing it entirely.

Migration
A versioned script that changes the database schema — adding a table, a column or an index — in a way that can be replayed on another machine. Alembic manages these here.

Alembic
The Python migration tool used by this project. `alembic upgrade head` applies every pending migration in order.

Service layer
Plain Python modules holding the business rules: "a session may only move from scheduled to active", "a late arrival is marked late, not present". Keeping these out of the route files makes them testable on their own.

Business rule
A condition the domain insists on. Examples: one attendance record per student per session; a student may not be marked in a session they are not enrolled in.

Status code / error envelope
Every API error in this project has the same shape: `{"error": {"code": "session_overlap", "message": "…", "details": …}}`. The `code` is stable for programs to branch on; the `message` is written for a human.

RBAC (Role-Based Access Control)
Access rules attached to roles — admin, faculty, student — rather than to individual people. Ownership checks (is this faculty member teaching this course?) are layered on top.

Authentication vs authorisation
Authentication = "who are you?". Authorisation = "may you do this?". The project checks them in that order, always server-side.

Middleware
Code that runs around every request. Here it adds a request ID, sets security headers and disables caching on API responses.

Rate limiting
Capping how many requests one client may make in a period, to blunt brute-force attacks and accidental floods.

2.5 Database words

Database
An organised store of related information that survives a restart. This project uses PostgreSQL, stored in files on disk.

PostgreSQL
An open-source relational database. "Relational" means it stores data in **tables** of rows and columns and can relate them through foreign keys.

Table
A named grid of related facts — one table per noun (`students`, `attendance_records`). This project has 17 of them.

Row (record)
One instance of the thing a table describes: one student, one attendance record.

Column (field)
One attribute of that thing: a student's name, an attendance record's status.

Primary key
The column (or set of columns) that uniquely identifies a row. This project uses an auto-incrementing `BIGINT` called `id`.

Foreign key
A column in one table that points at the primary key of another, enforcing that the referenced row actually exists.

Constraint
A rule the database itself enforces, regardless of which application code is running. A unique constraint on `(session_id, student_id)` is what makes double-marking impossible even under concurrent requests.

Index
An extra structure that makes lookups fast, at the cost of slower writes and more disk.

Schema
The overall structure of the database — which tables exist, their columns, types and relationships. A migration changes the schema.

Enum
A value drawn from a fixed list. This project stores them as text with a `CHECK` constraint, which is easier to migrate than PostgreSQL's native enum type.

ERD (Entity-Relationship Diagram)
A picture of the tables and how they link. `docs/diagrams.md` contains one.

JSONB
A column that stores a small JSON document. Used here for quality measurements and audit snapshots, where the shape may vary.

Audit log
An append-only record of privileged actions: who did what, to which object, when, and what changed. Written here with before/after snapshots but never with secrets.

2.6 AI, security and encryption words

Artificial intelligence / machine learning
Programs that learn patterns from examples instead of following hand-written rules. Here the "examples" are the model weights, downloaded ready-made rather than trained by this project.

Model / weights
The learned parameters that make up a trained network. Stored as a file. Here: YuNet, SFace and MiniFASNet in the ONNX format.

ONNX
(Open Neural Network Exchange) A vendor-neutral file format for neural networks, so a model trained with one framework can run with another. This is why the project can run models without a GPU or a deep-learning framework installed.

ONNX Runtime
The library that executes ONNX models. The CPU execution provider is used here.

OpenCV
(Open Source Computer Vision Library) A library of image-processing tools. This project uses it to decode JPEG frames, run the face detector and recogniser, and measure image quality.

Face detection
Finding *where* the faces are in an image and how big they are. It does not know *who* they are.

Face recognition / embedding
Turning a face into a vector of numbers — a "fingerprint" of the face's shape — so that two pictures of the same person produce nearby vectors. SFace produces 128 numbers per face.

Cosine similarity
A measure of how aligned two vectors are: 1 means identical direction, 0 means unrelated, −1 means opposite. Because both vectors are normalised, the result is a number between −1 and 1 and does not depend on vector length.

Threshold
The cut-off above which a similarity is considered a match. Choosing it is a trade-off: too low and different people match, too high and the same person fails to match.

Liveness detection
Checking that a face in front of the camera is a real, live person rather than a photograph, a video replay or a mask. Also called **PAD** (Presentation Attack Detection).

Presentation attack
An attempt to fool a biometric system with a fake — a printed photo, a phone screen showing a video, a silicone mask.

Hashing (password hashing)
A one-way function. You cannot reverse a hash back to the password, which is exactly why passwords are stored as hashes rather than encrypted.

Salt
Random data mixed into a password hash so that two identical passwords produce different hashes, defeating precomputed lookup tables.

Argon2id
The password hashing algorithm used here (RFC 9106). It is deliberately slow and memory-hungry so that guessing millions of passwords per second is expensive.

Encryption (AES-256-GCM)
Turning readable data into ciphertext with a key, so that a stolen database file does not reveal the biometric templates. *GCM* is an authenticated mode: it also detects tampering.

Nonce / IV
A random value, unique per encryption, mixed into the ciphertext so that encrypting the same data twice produces different output.

AEAD associated data
Extra context that is authenticated but not encrypted. Here it binds each template to its student, model and version, so a ciphertext copied into another student's row will not decrypt.

Key rotation
Keeping old encryption keys usable for a while while encrypting new data with a new key, so data can be re-encrypted without downtime. The template rows store which key was used.

Threat model
A written list of who might attack the system and how, used to decide what to defend. See `docs/privacy-security.md`.

Consent
A recorded, informed agreement by the person whose biometric data is collected, including what the data is for and how long it is kept.

2.7 Tooling and infrastructure words

Docker
A tool that packages an application together with everything it needs to run (libraries, runtime, system files) into a portable unit called a **container**. Two people running the same image get the same behaviour.

Container
A running instance of such a package. It shares the host's kernel but has its own filesystem, so it cannot accidentally corrupt the host or another container.

Image
The template a container is created from — the packaged files plus a start command.

Docker Compose
A YAML file describing several containers and how they fit together, started and stopped with one command. This project's `docker-compose.yml` defines `db`, `api` and `web`.

Volume
A storage area that survives container restarts and rebuilds — where the database's files live.

nginx
A high-performance web server. In this project it serves the built React files and forwards `/api` calls to the back end.

Caddy
A web server that obtains and renews HTTPS certificates automatically. Used on the local machine to serve the site over a tunnel.

Cloudflare Tunnel (cloudflared)
A way to expose a service running on your own machine to the internet through an encrypted outbound connection — no port forwarding, no public IP address needed.

Environment variable
A setting read from the environment at start-up rather than hard-coded in the source. Secrets such as the database password and encryption keys live here, never in the repository.

.env file
A local file listing environment variables, kept out of version control by `.gitignore`. The project ships `.env.example` showing the shape without the values.

Health check
An endpoint a container polls to decide whether it is working, so an orchestrator can restart it or delay starting its dependents.

pytest / vitest
Test runners. pytest runs the Python suite; vitest runs the TypeScript suite. Both are configured so a failure gives a readable message rather than a wall of output.

CI / CD
Continuous Integration / Continuous Delivery — automatically running the tests and building the project whenever code is pushed.

> **You now have the vocabulary**
> Everything else in this guide reuses these words. If you get stuck later, come back to this chapter rather than reading a definition in isolation — the definitions are written to be read in order.

CHAPTER 3

What a web application actually is

Before the project itself, a short lesson on the machinery every web application sits on. This is the foundation everything else in the guide rests on.

3.1 The core idea: a program that asks another program

Open any website you use. Underneath the pictures and buttons, the same two-part arrangement happens every time:

1. Your browser sends a **request** to a server somewhere on the internet.
2. The server does some work and sends back a **response**.

The browser program is the **client**; the server program is the **server**. The browser never runs the business rules itself — it asks, and the server answers.

> **WHY SPLIT IT LIKE THIS?**
> Because of trust and reach. The browser runs on a machine the user controls, so anything the browser decides could be tampered with. Attendance decisions, password checks and database writes all happen on the server, where the user has no access. The browser is treated as an untrusted display device that may only ever *ask*.

3.2 The browser's three jobs

In a web application the browser does exactly three things, and knowing which is which makes the rest of the system much easier to reason about:

Job	What it means	In SmartAttend AI
Render	Turn data into pixels the user can see and click.	React draws the dashboard, the attendance tables, the charts and every dialog.
Collect	Gather input — a typed email, a click, a camera frame.	The login form, the roster picker, and the webcam frames posted to the recognition endpoint.
Request	Send data to the server and handle the reply.	Every call goes through one small helper, `frontend/src/lib/api.ts`, which attaches the cookie, adds the CSRF header and converts errors into a consistent shape.

> **The rule this project follows**
> The browser may never decide who someone is, what their attendance status is, whether they are allowed to see a page, or whether a face matched. It renders what the server tells it and asks the server for decisions. This is why pasting an attendance URL into another tab does not let you mark yourself present — the request still travels to the server, which decides on its own terms.

3.3 What an HTTP request looks like

Every interaction is one or more requests. A request for the list of students in one course looks roughly like this:

```
GET /api/courses/3/enrollments?page=1&page_size=20 HTTP/1.1
Host: attendance.example.edu
Accept: application/json
Cookie: sa_session=9f3c…            # proves which signed-in user this is
```

And the server's answer:

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: no-store

{ "items": [ { "id": 12, "full_name": "Asha Rao", "registration_number": "…",
               "biometric_enrolled": true } ], "total": 42, "page": 1, "page_size": 20 }
```

> **WHAT "NO-STORE" MEANS**
>
> Several response headers are set on every API reply. `Cache-Control: no-store` stops intermediaries (and the browser's disk cache) from keeping attendance data where it could later be shown to the wrong person after a sign-out. The full set is in `backend/app/main.py`: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, `Permissions-Policy: camera=(self)` and `Cross-Origin-Opener-Policy`.

3.4 The API: the contract between the two halves

An **API** is the published list of requests a server will accept and the shape of the replies it will send. It is the seam between two programs that may be written in different languages and updated independently.

The reason this matters for a project like ours: the front end and the back end were written by the same person, but they only talk through the API. That means the interface can be replaced without touching the server, or a mobile app could be added without changing the business rules.

The four rules this API follows

1. **Resources have names.** A URL names a thing: `/api/sessions` (all sessions), `/api/sessions/12` (one session), `/api/courses` (all courses).
2. **The method says what to do.** `GET` reads, `POST` creates or triggers, `PATCH` changes part of a thing, and `DELETE` removes it.
3. **Statelessness.** Each request carries everything the server needs to understand it. The server does not remember "we were just talking"; the cookie carries the identity on every single request.
4. **Consistent errors.** Every failure has the same envelope, so the front end can show a sensible message without special-casing each endpoint.

3.5 Why a single-page application

There are two ways to build a web page. The old way is a *multi-page* site: every link causes the browser to request a whole new HTML document, which the server renders. The modern way is a *single-page application*: the browser loads one HTML file plus a JavaScript bundle, and the JavaScript swaps the visible content as the user moves around.

Aspect	Multi-page	Single-page (this project)
Round trips	A full document per page	One document; only JSON data afterwards
Feels like	A website	An application — instant transitions, no page flash
Server work	Render HTML on every request	Send JSON; the browser assembles the screen
Cost	Simple, but slower and repetitive	More front-end code, much better experience

SmartAttend AI is a single-page application. The address bar still changes as you navigate (`/attendance`, `/reports`) — that is what the router does — but the page itself is never reloaded, so filters, scroll position and the live camera keep working.

3.6 Where the code runs

This distinction causes more confusion than anything else in web development, so it is worth being completely clear:

Runs in the browser	Runs on the server	Runs on the user's disk
All of `frontend/src/` React components, routing, the API helper, camera capture, chart rendering, form validation for convenience.	**All of `backend/app/` and `ai_service/`** Authentication, role checks, every business rule, all SQL, all model inference.	**Nothing.** The browser's cache and the OS may hold temporary files, but the application stores no meaningful data on the client. Signing out removes the session cookie and the next request is treated as anonymous.

> **A concrete consequence**
>
> Because all decisions are server-side, you cannot "tinker with the page" to gain anything. A student who opens the browser's developer tools and calls the self-marking endpoint by hand still gets the server's answer: 409 `window_closed`, or 403 if they are locked, or 409 if a record already exists. The interface is a convenience; the server is the authority.

CHAPTER 4

What SmartAttend AI does

The product from the outside in: the problem it addresses, the people who use it, and what each of them can do.

4.1 The problem

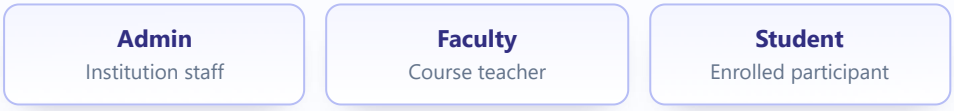
Taking attendance by calling out names is slow, and in a large class it is the part of the lesson most likely to be skipped or rushed. Cameras and face recognition have been suggested as the alternative, and they do solve the speed problem — but they introduce two new problems that ordinary software does not have:

1. **Accuracy.** A recognition system that is wrong is worse than no system, because the wrong mark goes into a permanent record.
2. **Trust.** A camera in a classroom is a surveillance device unless it is designed with consent, deletion and limited retention as first-class requirements.

SmartAttend AI is built around the answer to both: **automate the easy case, and send everything uncertain to a human**. Recognition is treated as a suggestion generator, not as an authority.

4.2 The three actors

The whole system is designed around three roles. Every page and every endpoint is scoped to one of them.



Role	Can do	Cannot do
Admin	Manage people, courses, rosters, sessions and system settings; issue accounts and password resets; see every report; reopen a closed session with a reason; unlock a student from self-marking; read the audit log.	Nothing is blocked from admin by role — but every privileged write is *recorded* with the actor, the time, the reason and the before/after values.
Faculty	Run live attendance for their own sessions; mark attendance manually; review uncertain matches for their own courses; approve or reject correction requests; see reports for their own courses.	Cannot see or touch another teacher's courses, students' accounts, or the global settings. They cannot create a course.
Student	Sign in; enrol their own face (with consent); mark themselves present when a session's self-marking window is open; request a correction of their own record; see their own attendance summary, history and reports; delete their biometric data.	Cannot see anyone else's data, cannot mark anyone else present, cannot open the camera in a live session, cannot change a record directly.

4.3 The four ways attendance gets recorded

The most important design decision in the project: there are **four sources** of an attendance record, and the system always knows which one produced it. A record is never ambiguous about its origin.

Source	How it happens	When it is used
`automated`	The camera recognised the student, the quality check passed, and the similarity cleared the threshold and the ambiguity margin.	The normal path during a live session.
`self_reported`	The student pressed "Mark me present" inside the session's own window.	When the teacher is busy, or when a student arrives without being scanned. It is a deliberate, visible, less-trusted record — and it is marked `late` if it arrives after the late threshold.
`manual`	A member of staff marked it directly, or approved a correction request.	The non-biometric alternative, and the outcome of every reviewed case.
`system`	The session was finalised and the student had no record at all.	Generating `absent` for everyone on the roster who was never marked.

> **Why recording the source is not optional**
>
> If a student's attendance percentage is contested, the first question is always "how was this recorded?". Keeping `source` on every record means the answer is always available and always truthful. It also means a self-reported record can never silently pass as a biometric match in an audit.

4.4 What happens during a normal lecture

Here is the whole product in one paragraph, before we dissect it:

1. An **admin** creates a course, enrols the students, and a **faculty** member schedules a session with a start time and a length.
2. When the lecture begins, the faculty member opens **Live attendance** and starts the session. The camera turns on.
3. Every second the browser captures a frame, checks it locally, and sends it to the server for analysis. The server detects faces, judges quality, runs liveness if enabled, compares against enrolled templates and records a `present` record if the match is confident.
4. Anyone the system is not sure about — an unknown face, a low score, two similar faces — is written to a **review queue** with the evidence attached, and the teacher decides later.
5. A student who was not scanned can press **Mark me present** on their own phone during the session's window, if the teacher allowed self-marking.
6. At the end, the teacher closes and finalises the session. Anyone unmarked becomes `absent`.
7. If a student believes a record is wrong, they file a **correction request** with a reason; the teacher approves or rejects it, and the answer is written to the audit log.

8. Dashboards and reports show attendance rates, trends and exceptions, and can be exported to CSV or PDF.

4.5 The design promises

Four commitments run through every part of the system. They are the reason for a number of otherwise-awkward design decisions:

Promise	What it means in practice
Consent first	No face template can exist without a recorded, versioned consent grant. The database has a foreign key from templates to consents, so the database itself refuses an unauthorised template. A student can delete their templates at any time.
Uncertainty is not punished	An uncertain match never becomes a mark. It becomes a review item. If liveness cannot be evaluated, the result is `not_enabled` or `inconclusive` — never a fabricated pass.
Nothing is hidden	Privileged actions are written to an append-only audit log with before/after values. Students can see and delete their own biometric data. Corrections are visible and justified.
No false metrics	No accuracy claim is made anywhere in the product or its documentation until the evaluation harness has been run on an approved data set. Placeholder screens say they are placeholders instead of showing invented numbers.

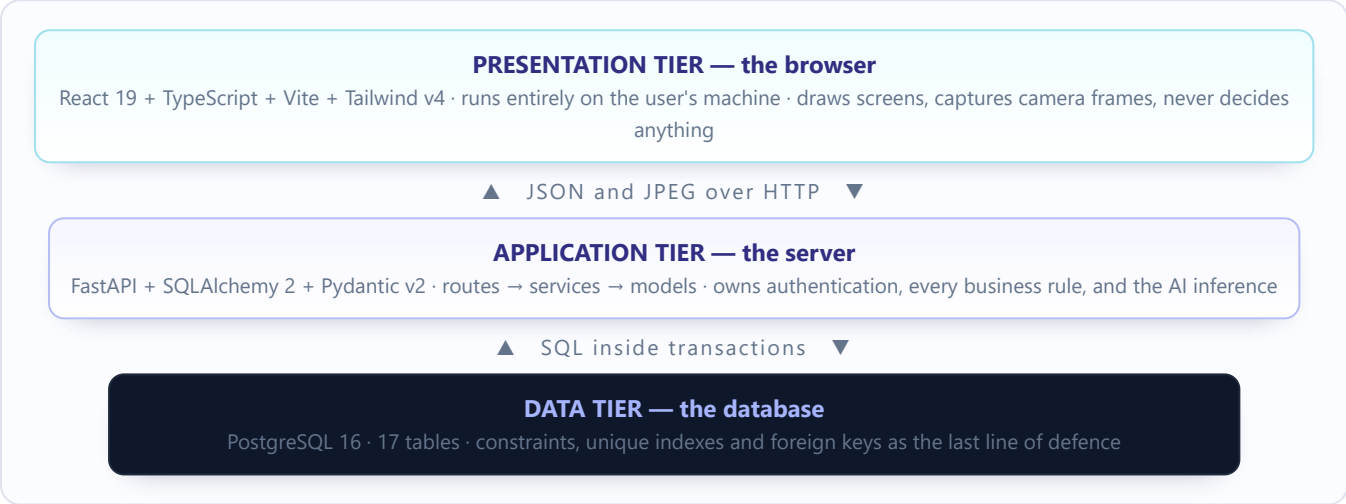
CHAPTER 5

Architecture: the three layers

How the code is divided, why it is divided that way, and what each part is strictly forbidden from doing.

5.1 The whole system on one page

SmartAttend AI is a **three-tier application**. "Tier" and "layer" mean the same thing here: a band of responsibility that sits between two others and only talks to its neighbours.



Sitting beside the application tier is a fourth box that is easy to miss:

ai_service/ — the computer-vision package

A separate Python package with no imports from `backend/`. It receives images and returns numbers. It never touches the database, never knows what a course is, and never decides whether someone is present.

5.2 Why three layers instead of one file

It would be entirely possible to write this whole project as one Python file that also returns HTML. It would even work. The reason not to is that the three parts change for completely different reasons, and separating them means changing one does not force you to touch the others.

Layer	Changes when…	Stays untouched when…
Front end	the design changes, a screen is added, a chart is re-styled, wording changes	the database schema changes, the recognition threshold changes, the password policy changes
Back end	a business rule changes, a new role appears, an integration is added	a button moves, a colour changes, a column is renamed in the UI
Database	a new fact must be remembered, an existing one needs history	anything about how the data is displayed or computed

Layer	Changes when…	Stays untouched when…
AI package	a better model appears, thresholds are re-tuned	anything about attendance policy, users or storage

> **SEPARATION OF CONCERNS**
> The principle that each part of a system should have exactly one reason to change. It is not about tidiness: it is about the size of the blast radius when you edit something. In this project it means a designer can restyle the dashboard without any risk of changing who is marked present.

5.3 The rules each layer must obey

The project's own architecture document states these as explicit "owns / must not" pairs. They are the most useful part of the design, because they turn vague advice ("keep it clean") into checkable rules.

Layer	Owns	Must *not*
Frontend `frontend/`	Rendering, camera capture, frame sampling, UX states (loading, empty, error)	Decide identity, attendance status, eligibility, or permissions
API routes `backend/app/api/`	HTTP shape, validation via Pydantic, authN/authZ dependencies, status codes	Contain SQL or computer-vision logic
Services `backend/app/services/`	Business rules: the session state machine, attendance policy, corrections, reports, consent	Trust any client-provided identity, status or eligibility
Models `backend/app/models/`	Persistence, constraints, relationships, transactions	Contain policy decisions
AI `ai_service/`	Detection, quality checks, alignment, embeddings, similarity, liveness scoring	Write to the database, or know that courses and sessions exist
PostgreSQL	Durable state, uniqueness (one record per student per session), referential integrity	Be reached directly by anything except the ORM layer

5.4 Inside the back end: four folders deep

The application tier is itself layered. A request crosses four directories before it touches the database, and each one has a narrower job than the last.

```
backend/app/
├── main.py        builds the FastAPI app, wires middleware and routers
├── core/          infrastructure: config, database, security, crypto, errors, rate limiting
├── api/           HTTP: dependencies (deps.py) and one route module per resource
│   └── routes/    auth, users, students, faculty, courses, sessions, attendance,
│                  enrollment, recognition, review, corrections, reports, me, system
├── services/      business rules — the only place policy lives
├── models/        SQLAlchemy table definitions
├── schemas/       Pydantic request/response shapes
└── cli.py         command-line maintenance tasks (seed-admin, create-admin, purge-biometrics)
```

> **ROUTE vs SERVICE vs MODEL vs SCHEMA**
>
> **Route** = "here is the URL, the method, and who may call it". **Schema** = "here is the exact shape of the JSON going in and coming out". **Service** = "here is what the rule is". **Model** = "here is where the fact is stored". A request passes through all four, and each stage is testable alone.

A worked example: "start a session"

Follow one click through all four stages. This is the single best way to understand the layering.

Stage	File	What happens there
1. UI	`frontend/src/pages/SessionDetailPage.tsx`	The Start button is shown only when the server said the session is `scheduled`. Clicking calls `api("/sessions/12/start", {method:"POST"})`. The browser does not decide whether starting is legal.
2. Transport	`frontend/src/lib/api.ts`	Adds `credentials: "include"` so the session cookie travels, attaches `X-CSRF-Token` because this is a state-changing method, and converts a non-2xx reply into an `ApiError` the page can display.
3. Route	`backend/app/api/routes/sessions.py`	Declares `POST /sessions/{id}/start`, requires the role dependency, loads the session id from the path, and calls the service. No logic.
4. Auth dependency	`backend/app/api/deps.py`	Reads the `sa_session` cookie, hashes it, looks up the row, checks expiry and idle timeout, checks the CSRF header, and returns the user. If any of that fails the request never reaches the route body.
5. Service	`backend/app/services/session_service.py`	Locks the session row, verifies the caller's faculty owns the course, verifies the status is `scheduled`, sets `started_at`, writes an audit entry.
6. Model / DB	`backend/app/models/attendance.py` → PostgreSQL	The `UPDATE` is issued inside the transaction and committed. Only then does the route serialise the new session back to JSON.
7. Back to UI	`SessionDetailPage.tsx`	Re-reads the session, the status chip turns green, and the Live button appears. The UI changed because the *server said so*, not because the click assumed success.

5.5 Why the AI code is separate but still in-process

A natural question: if `ai_service/` is a clean, independent package, why is it not a separate microservice with its own HTTP port?

Because the reference machine for this project is an **Intel i3-1005G1 laptop with 8 GB of RAM and no GPU**. A second Python process would load OpenCV and ONNX Runtime a second time, adding memory pressure and an HTTP round trip per frame, in exchange for nothing the product actually needs.

> **The boundary is still real**
>
> The package has no imports from `backend/`, takes images and returns dataclasses, and is documented as the seam where a future split would happen. Splitting it out later means wrapping the same `FaceAnalyzer` interface in an HTTP server — a mechanical change, not a rewrite. **Planned**

5.6 What a request looks like end to end

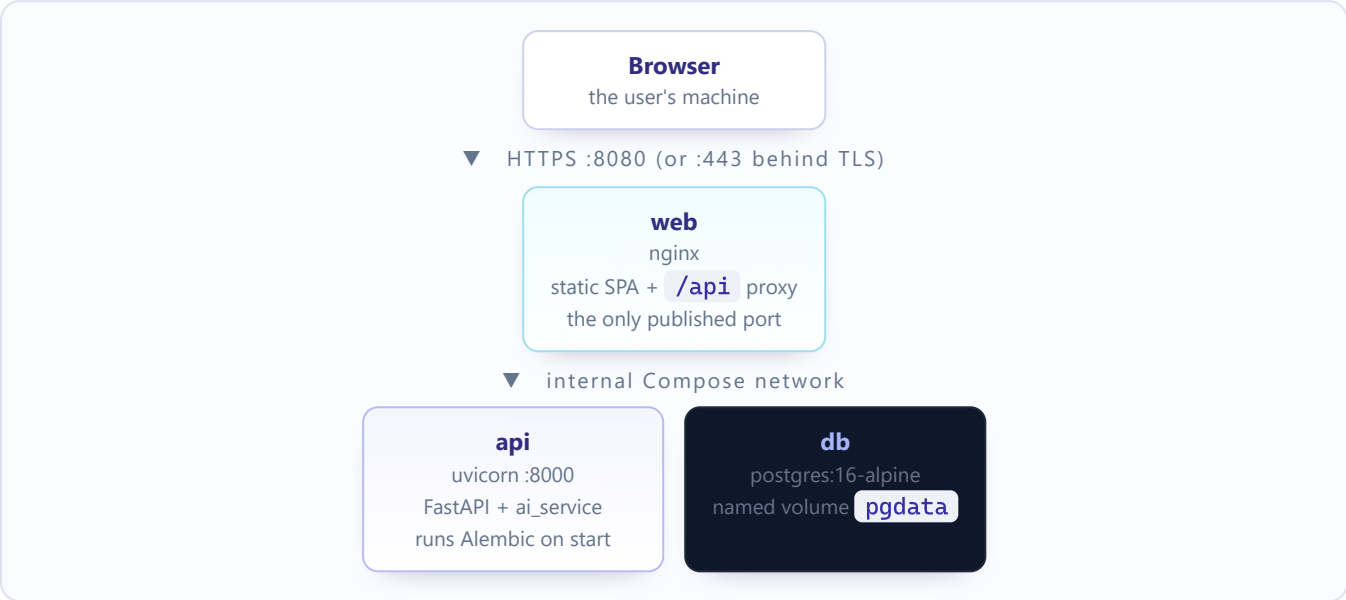
The same journey drawn as a stack, for a live-recognition frame:



> **The frame is gone after the response**
>
> The raw image exists only in request memory. It is never written to disk, never stored in the database, and never logged. What is stored is the *decision* about it — the outcome, the similarity score, the quality measurements, and the timestamps. This is a privacy property, not an optimisation.

5.7 The shape of the running system

Finally, how the pieces are actually deployed. In production this is three containers on one machine; in development it is three processes on a laptop.



Note what is *not* in that diagram: no port for the API and no port for the database. Both are reachable only from inside the private network. The public surface of the entire system is one nginx container that serves static files and forwards `/api`. That is a deliberate reduction in attack surface, and it is also why `CORS_ORIGINS` is empty in production — the browser sees a single origin, so no cross-origin permission is needed at all.

CHAPTER 6

The repository, folder by folder

A map of the 201 files on GitHub, what lives where, and which files are worth opening first.

6.1 By the numbers

201 FILES PUBLISHED	**8,459** LINES OF PYTHON	**12,092** LINES OF TYPESCRIPT	**78** API ENDPOINTS
17 DATABASE TABLES	**97** BACKEND TESTS	**57** FRONTEND TESTS	**12** DOCUMENTATION FILES

The Python figure covers `backend/app/` and `ai_service/` across 76 files; the TypeScript figure covers `frontend/src/` across 67 files. Migrations (551 lines) and the backend test suite (2,372 lines) are counted separately.

6.2 Top level

Path	Type	What it is
`backend/`	directory	The FastAPI application, its tests, its migrations and its Dockerfile.
`frontend/`	directory	The React application, its tests and its Dockerfile.
`ai_service/`	directory	The computer-vision package: detection, recognition, quality, liveness, evaluation harness.
`docs/`	directory	Twelve markdown documents: the specification, the diagrams, the API contract, the database design, deployment, privacy, evaluation and viva preparation.
`docker-compose.yml`	file	Defines the three production containers and how they depend on each other.
`.env.example`	file	The *shape* of the configuration, with placeholder values. Copied to `.env` and filled in locally.
`.env`	file	The real secrets. **Never committed.** Listed in `.gitignore`.
`.gitignore`	file	Tells Git which paths to ignore: virtual environments, build output, model weights, databases, logs, secrets.
`.gitattributes`	file	Normalises line endings so shell scripts keep LF even when checked out on Windows — otherwise the container's entrypoint fails to run.
`.dockerignore`	file	Keeps `.env`, `.venv`, `node_modules` and model weights out of the build context.

Path	Type	What it is
`LICENSE`	file	The MIT licence, covering this repository's own code only.
`README.md`	file	The front door: what the project is, how to run it, and a table linking every document.

What is deliberately absent

There is no `.env`, no `.venv/`, no `node_modules/`, no `dist/`, no `pgdata/`, no log file and no `.tools/` directory in the published repository. The three model weight files are downloaded by a script and verified by checksum, not committed. Before publishing, the staged tree was audited by searching *blob contents* — not just file names — for the live secret values: zero occurrences of the seed admin password, zero of the encryption keys.

6.3 `backend/`

Path	Contents
`app/main.py`	Builds the FastAPI object, registers error handlers, adds the request-context middleware and the CORS middleware, mounts every router under `/api`, and applies the default 600-requests-per-minute limiter. Read this first: it is 88 lines and it tells you the shape of the whole server.
`app/core/`	`config.py` (Pydantic settings, with guards that refuse to start in the wrong environment), `database.py` (engine and session), `security.py` (Argon2id hashing, password policy, token generation), `crypto.py` (AES-256-GCM template cipher with key rotation), `errors.py` (the error envelope), `ratelimit.py` (sliding-window limiter).
`app/api/deps.py`	The authentication dependencies: `get_auth`, `current_user`, `require_roles( ... )`, `check_csrf`, and the pagination helper. 69 lines that guard 78 endpoints.
`app/api/routes/`	Fourteen modules, one per resource. The largest is `sessions.py` with 18 endpoints; the smallest are `me.py` and `review.py` with 2 each.
`app/services/`	Thirteen modules holding the business rules. `session_service.py` owns the state machine and the attendance window; `attendance_service.py` owns marking and finalisation; `recognition_service.py` owns the decide-and-record pipeline; `biometric_service.py` owns consent and templates; `reports_service.py` owns aggregation and exports.
`app/models/`	SQLAlchemy classes for the 17 tables, grouped by domain: `people.py`, `course.py`, `attendance.py`, `biometric.py`, `user.py`, `audit.py`.
`app/schemas/`	Pydantic models for every request and response body.
`migrations/versions/`	Two Alembic scripts: the initial schema, and the student self-marking columns and indexes.
`tests/`	Nine test modules, 97 test functions, running against a separate `smartattend_test` database.

Path	Contents
`cli.py`	Maintenance commands: `seed-admin`, `create-admin`, `seed-demo`, `purge-biometrics`.
`Dockerfile`, `docker-entrypoint.sh`	The API image: non-root user, healthcheck, migrations run before uvicorn starts.

6.4 `frontend/`

Path	Contents
`src/main.tsx`	The entry point: mounts React, wraps the app in the router and the auth provider.
`src/App.tsx`	Every route in the application, with its role guard. 84 lines — the whole navigation structure of the product in one readable file.
`src/pages/`	Nineteen screens. Each is one file; each non-trivial one has a matching `.test.tsx`.
`src/components/ui/`	Twelve small, reusable primitives: Button, Card, Badge, Alert, Field, Select, Switch, Textarea, Modal, PageHeader, Feedback, PasswordField. They are the design system expressed as components.
`src/components/layout/AppShell.tsx`	The frame around every authenticated page: the dark sidebar with its gradient orbs, the role-filtered navigation, the sticky header, and the mobile drawer.
`src/components/`	Bigger composed pieces: `EnrollmentDialog` (the camera enrolment wizard), `ResetLinkDialog`, `RequireAuth` (the guards), and `charts/AttendanceCharts`.
`src/lib/`	Non-visual helpers: `api.ts` (the single HTTP client), `camera.ts` (getUserMedia and frame capture), `types.ts` (the TypeScript mirror of the API), `navigation.ts` (which menu items each role sees), `format.ts`, `attendance.ts`, `reportColors.ts`.
`src/hooks/`	`useCamera` (start/stop with cleanup) and `useDebounced` (delay keystrokes before searching).
`src/context/AuthContext.tsx`	Holds the signed-in user for the whole tree and exposes `login`, `logout` and `refresh`.
`src/index.css`	The design tokens and the custom utility classes — the whole visual identity in 165 lines.
`vite.config.ts`	Dev server on 5173, the `/api` proxy, and the vitest configuration.
`dist/`	The build output. Generated, not committed.
`Dockerfile`, `nginx.conf`	A two-stage build: compile with Node, serve with nginx. The nginx config adds security headers and reverse-proxies `/api`.

6.5 `ai_service/`

Path	Contents
`analyzer.py`	The `FaceAnalyzer` class: loads YuNet and SFace once, serialises inference behind a lock, returns a `FrameAnalysis` of detected faces with quality reports and embeddings. 101 lines and the heart of the CV pipeline.
`quality.py`	Pure-OpenCV checks on the face crop: size, blur (variance of the Laplacian), brightness. Decides whether a face is worth embedding at all.
`matcher.py`	`match_embedding()`: cosine similarity of a probe against a gallery, returning the best student, their score, and the runner-up score so the caller can judge ambiguity.
`liveness/detector.py`	The MiniFASNet wrapper: crop, expand, resize, normalise, softmax, report. **Experimental**
`liveness/convert_minifasnet.py`	One-off converter from the upstream PyTorch checkpoint to ONNX, with a parity check against the PyTorch forward pass.
`models/`	The three `.onnx` weight files. Downloaded by script, checksum-verified, never committed.
`scripts/fetch_models.py`	Downloads the weights and verifies their SHA-256 against the values recorded in `docs/ai-models.md`.
`evaluation/`	The measurement harness: dataset loaders, metric computation, threshold selection. This is the machinery that must be run before any accuracy claim can be made.
`types.py`	The dataclasses that cross the boundary: `FaceBox`, `QualityReport`, `DetectedFace`, `FrameAnalysis`, `QualityPolicy`.

6.6 `docs/` — the twelve documents

Documentation is treated as part of the deliverable, not as an afterthought. Each file has one job and they do not overlap:

File	Lines	Answers the question
`srs.md`	421	What must the system do, and how will we know it is done?
`diagrams.md`	439	What does the system look like from the outside, inside, and in time?
`architecture.md`	155	Why is it divided this way, and what may each layer not do?
`database-design.md`	262	Which tables, which constraints, which invariants?
`api-documentation.md`	222	Which endpoints exist, who may call them, and how do they fail?
`ai-models.md`	93	Which models, from where, under which licence, with which checksum?
`evaluation.md`	159	How would we measure this, and what have we measured so far?

File	Lines	Answers the question
`privacy-security.md`	184	What is the threat model, the consent flow, the retention policy?
`deployment.md`	163	How do we run this for real, back it up and upgrade it?
`ui-wireframes.md`	148	What does each screen contain, and what states can it be in?
`report-material.md`	401	What goes in the written project report, and where is the evidence?
`viva-prep.md`	350	What will the examiners ask, and what is the honest answer?

The consistency rule

The architecture document states it plainly: "*This file is kept consistent with the code; when they disagree, the code wins and this file is a bug.*" That single sentence resolves the most common failure mode of documented projects — documentation that quietly becomes fiction.

6.7 Where to start reading the code

If you want to understand this project by reading it, this order works:

1. `frontend/src/App.tsx` — 84 lines that describe the entire product surface.
2. `frontend/src/lib/api.ts` — 121 lines that show how the two halves are joined.
3. `backend/app/main.py` — the server's skeleton.
4. `backend/app/api/deps.py` — how every request is authenticated and authorised.
5. `backend/app/models/attendance.py` — the data the product is really about.
6. `backend/app/services/session_service.py` — the state machine and the window rules.
7. `backend/app/services/recognition_service.py` — the decide-and-record pipeline.
8. `ai_service/analyzer.py` — where pixels become numbers.
9. `backend/tests/test_sessions.py` — the specification written as executable examples.

CHAPTER 7

The front end — what it is and why

Before any code: what the front end is for, what problem each of its tools solves, and why these particular tools were chosen.

7.1 What the front end is responsible for

The **front end** is the part of the application that runs in the user's browser and that the user can see and touch. In SmartAttend AI it is a React single-page application made of 67 TypeScript files and about 12,000 lines of code.

Its responsibilities are exactly four:

1. **Show state.** Render whatever the server last said — a list of sessions, a roster with attendance colours, a chart of trends, an error message.
2. **Collect input.** Turn a click, a typed search, a chosen date range or a camera frame into a request.
3. **Handle the awkward moments.** Every screen has a loading state, an empty state, an error state and a permission-denied state. Handling all four is most of the work.
4. **Never decide anything.** This is the discipline that makes the rest of the design trustworthy.

> **DECLARATIVE vs IMPERATIVE**
>
> The old way to build an interface was *imperative*: "when the user clicks, find the table row, change its background to green, add a tick icon, update the counter at the top." The React way is *declarative*: you describe what the screen looks like for each possible state, you change the state, and React works out the minimum set of DOM edits. You write the picture, not the procedure.

7.2 The stack, and the problem each piece solves

The front end uses five tools. Each one exists because something was painful without it.

Tool	Version	The problem	What it does instead
React	19.3	Building a screen out of hand-written DOM manipulation does not scale. Every new feature becomes a tangle of "find this element, update that element", and the code drifts out of sync with reality.	Describe the screen as a function of state. React keeps the display and the data in step and re-renders only what changed.
TypeScript	6.0	JavaScript fails at runtime, often in the user's hands. A typo in a property name, a `undefined` where an object was expected, a string passed where a number belongs.	Types checked before the code runs. The whole API surface is mirrored in `src/lib/types.ts`, so a response field that does not exist is a compile error, not a blank screen.

Tool	Version	The problem	What it does instead
Vite	8.3	Raw browser JavaScript cannot import a `.tsx` file, cannot use Tailwind directly, and downloading 60 separate modules is slow.	A dev server with instant hot reload during development, and a production bundler that compiles, tree-shakes and minifies into a few static files.
Tailwind CSS	4.3	Separate stylesheet files grow without bound, names collide, and it becomes impossible to tell whether a class is still used anywhere.	Utility classes written inline, plus a `@theme` block that defines the design tokens once. Deleting a class from the markup deletes it from the build.
React Router	7.18	A single-page app still needs URLs: the back button must work, a link must be shareable, and different roles must see different screens.	Maps paths to components, keeps the address bar honest, and lets guards wrap groups of routes.

Two smaller libraries round it out:

Library	Version	Why
Recharts	3.10	Charts. The dashboard needs a line chart of attendance over time, a bar chart comparing courses, and a donut of status distribution. Recharts draws them from plain arrays as React components, so they resize with the layout and inherit the colour tokens.
lucide-react	1.61	Icons. One consistent stroke weight, tree-shakeable, and each icon accepts `aria-hidden` so screen readers skip decorative glyphs.

7.3 What was deliberately *not* chosen

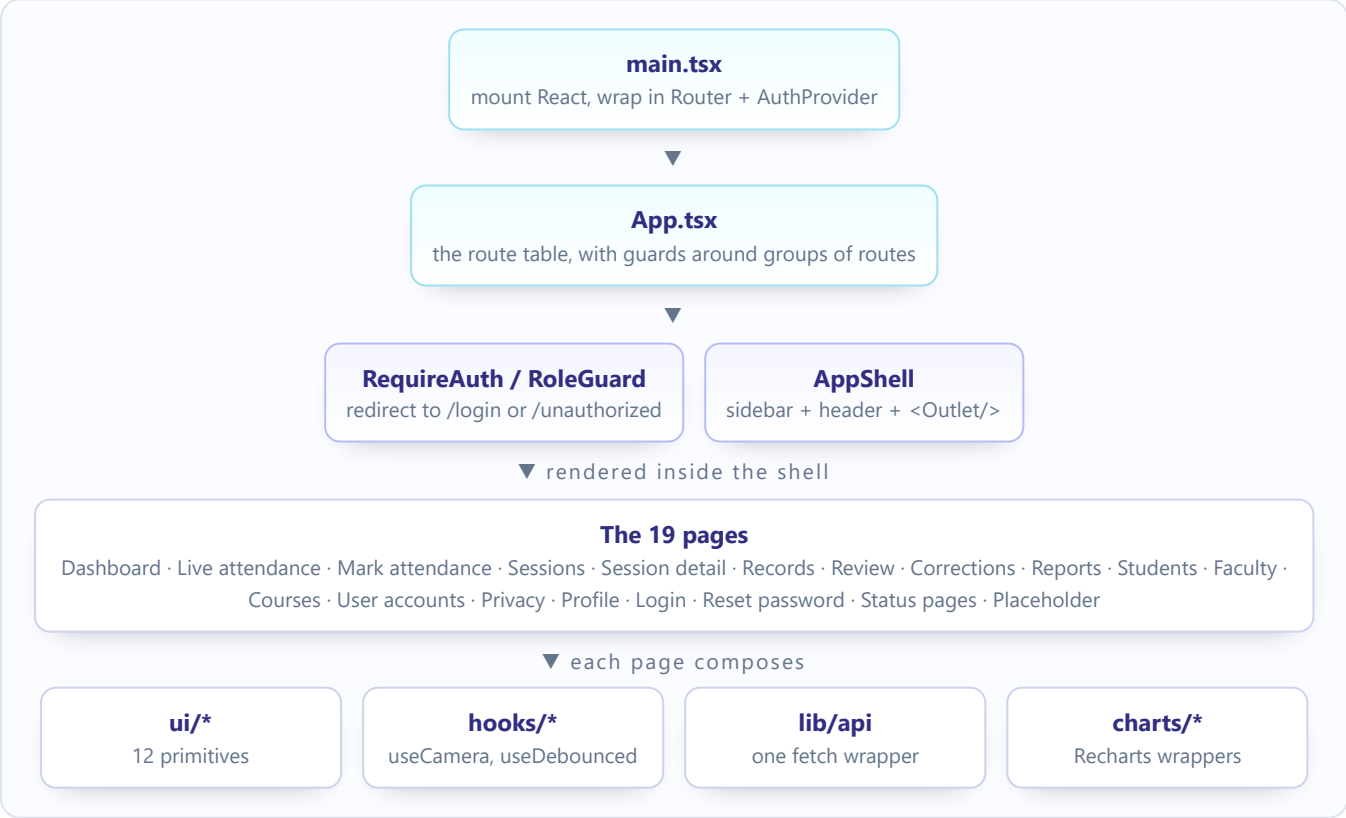
Knowing what a project avoided is as informative as knowing what it used. These are the decisions a viva examiner or a reviewer is most likely to probe.

Popular option	Chosen instead	Reason
Next.js / Nuxt (a meta-framework)	plain Vite + React	Server-side rendering solves a problem this application does not have. The app is behind a login; there is nothing for a search engine to index. A meta-framework would add a server runtime, a build pipeline and a deployment constraint for no benefit.
Redux / Zustand / React Query (state libraries)	React Context plus local `useState`	There is exactly one piece of genuinely global state: the signed-in user. Everything else is per-screen data fetched when the screen mounts. A global store would be ceremony without payoff at this size.
MUI / Ant Design / shadcn (component libraries)	12 hand-written primitives	A heavy library dictates the visual language and drags in dependencies. Twelve small files — Button, Card, Badge, Field, Modal and so on — give the same consistency and stay entirely under our control. The result is 30 KB of our own code rather than a framework.

Popular option	Chosen instead	Reason
Axios	the browser's `fetch`	One 121-line wrapper around `fetch` gives everything needed: the CSRF header, cookie credentials, a typed error object, and file download. A dependency would add nothing.
Client-side face detection (face-api.js, MediaPipe)	send frames to the server	This one is a **security** decision, not a performance one. If matching happened in the browser, the gallery and the decision logic would be inside the user's reach — trivially inspectable and tamperable. Recognition must run where the user cannot see it.
Storing the JWT in `localStorage`	an `HttpOnly` cookie	Anything JavaScript can read, an XSS bug can read. A token in a readable location is a stealable token. The cookie is invisible to page scripts, which is the whole point.

> **The pattern in that table**
>
> Every rejection follows the same test: *does this solve a problem this project actually has?* If not, it is complexity without benefit. This is worth remembering as a habit — the skill is not adding tools, it is refusing to.

7.4 How the pieces fit



7.5 The one-file rule for talking to the server

Every network call in the application goes through a single function in `frontend/src/lib/api.ts`. No page calls `fetch` directly. That one rule buys four things at once:

- **The CSRF header is never forgotten.** Any non-`GET` call picks up the `sa_csrf` cookie and attaches it as a header automatically.
- **Cookies always travel.** `credentials: "include"` is set in one place, so no endpoint can accidentally be called without authentication.
- **Errors are one shape.** Every failure becomes an `ApiError` with a `status`, a machine `code` and a human `message`. Pages can display `error.message` without knowing which endpoint failed.
- **The base URL is configurable.** `VITE_API_BASE_URL` lets the same bundle point somewhere else if the front end is ever served from a different origin.

```
// frontend/src/lib/api.ts
export async function api<T>(path: string, options: RequestOptions = {}): Promise<T> {
  const method = options.method ?? "GET";
  const headers: Record<string, string> = { Accept: "application/json" };
  if (method !== "GET") {
    const csrf = readCookie("sa_csrf");
    if (csrf) headers["X-CSRF-Token"] = csrf;    // the double-submit half
  }
  response = await fetch(`${API_BASE}/api${path}`, {
    method, headers, body,
    credentials: "include",                      // send the session cookie
  });
  …
  if (!response.ok) throw new ApiError(response.status, err?.code ?? "http_error", …);
  return payload as T;
}
```

Why the generic `<T>`

`api<Me>("/auth/me")` tells TypeScript what shape to expect back. Combined with the mirror types in `lib/types.ts`, a page that reads `user.emial` fails the build instead of showing `undefined` in the header.

CHAPTER 8

The front end, in detail

How a page is built, how state flows, how the design system produces the look, and how the camera actually works.

8.1 Anatomy of a component

Here is the smallest complete component in the project, with every part labelled:

```
// frontend/src/components/ui/Badge.tsx  (condensed)
import type { ReactNode } from "react";
import { cx } from "../../lib/format";

type Tone = "brand" | "success" | "warning" | "danger" | "neutral";   // ① a type

const TONES: Record<Tone, string> = {                                  // ② a lookup table
  brand:   "bg-brand-50 text-brand-700 ring-brand-200",
  success: "bg-emerald-50 text-emerald-700 ring-emerald-200",
  warning: "bg-amber-50 text-amber-700 ring-amber-200",
  danger:  "bg-rose-50 text-rose-700 ring-rose-200",
  neutral: "bg-slate-100 text-slate-600 ring-slate-200",
};

export function Badge({ tone = "neutral", children }: {              // ③ props with defaults
  tone?: Tone; children: ReactNode;
}) {
  return (
    <span className={cx(                                             // ④ markup + classes
      "inline-flex items-center gap-1 rounded-full px-2 py-0.5",
      "text-[11px] font-semibold ring-1 ring-inset",
      TONES[tone],
    )}>
      {children}
    </span>
  );
}
```

Used anywhere as `<Badge tone="success">Present</Badge>`. The whole point is that "present is green, absent is red, pending is amber" is written down *once*, so it cannot drift between screens.

The four building blocks of any React component

Block	What it is	Where it appears in this project
Props	Values passed in by the parent. Read-only.	`tone`, `variant`, `loading`, `onClick`.
State	Values the component owns and can change; changing one re-renders.	`const [scanning, setScanning] = useState(false)` in `LivePage.tsx:90`.

Block	What it is	Where it appears in this project
Effects	Work that happens after render — fetching, subscribing, starting a timer — with a declared list of what it depends on.	`useEffect` at `LivePage.tsx:183` starts and stops the camera frame loop.
Render	The JSX returned: a description of the screen for the current state.	Everything visible.

> **The dependency array is the hard part**
>
> `useEffect(fn, [a, b])` runs `fn` after the first render and again whenever `a` or `b` changes. Getting that list wrong causes the two most common React bugs: an effect that runs on every render (infinite loop) or one that never re-runs when it should (stale data). The frame loop in `LivePage` lists seven dependencies because it genuinely needs all seven — session, status, camera readiness and the configured interval among them.

8.2 The 12 UI primitives

Every screen is assembled from the same small kit in `src/components/ui/`:

Component	Purpose	Notable detail
`Button`	Actions	Four variants (primary, secondary, ghost, danger), two sizes, and a `loading` prop that disables the button, shows a spinner and sets `aria-busy` so a double-click cannot submit twice.
`Card`	Raised surface	Applies the `.panel` glass treatment; the `hover` flag adds `.panel-lift`.
`Badge`	Status chips	Five tones; used for attendance status, role labels and session state.
`Alert`	Inline messages	Success / warning / danger banners with an icon; the destination of every `ApiError.message`.
`Field`	Labelled input	Wires `htmlFor` / `id` so clicking the label focuses the input, and renders help and error text with the right ARIA relationship.
`PasswordField`	Password input	The show/hide eye toggle, plus a live strength hint driven by the same rules the server enforces.
`Select`	Dropdown	Native element, styled — better on mobile and with a keyboard than a custom listbox.
`Switch`	On/off	Used for the self-marking toggle and the liveness policy. A real `role="switch"` button, not a styled checkbox.
`Textarea`	Long input	Used for correction reasons and review notes, where the server requires 5–500 characters.
`Modal`	Dialog	Focus trap, Escape to close, and a backdrop click that does not destroy in-progress input.

Component	Purpose	Notable detail
`PageHeader`	Title + actions	Every page's top band, so titles and primary buttons always sit in the same place.
`Feedback`	Loading / empty / error	The three states every data screen must handle, so nobody forgets the empty case.

8.3 The design system: how the look was made

The interface was redesigned once during the project, from a flat grey layout to the current "aurora" treatment. The requirement was depth and colour without losing the professional, institutional feel — a system that records attendance should not look like a game.

Step 1: tokens, not hard-coded values

Tailwind v4 lets the palette be declared in CSS. Everything else in the app references those names:

```
/* frontend/src/index.css */
@theme {
  --color-brand-500: #6c63f4;     /* aurora indigo — the primary surface */
  --color-accent-400: #22d3ee;     /* aqua — gradient stop and live-status highlight */
  --shadow-surface: 0 1px 1px rgba(15,23,42,.04), 0 4px 10px -4px rgba(15,23,42,.08),
                    0 18px 40px -26px rgba(15,23,42,.3);
  --shadow-lift:    0 2px 4px rgba(15,23,42,.05), 0 24px 44px -22px rgba(79,70,229,.45);
}
```

Declaring `--color-brand-500` is what makes `bg-brand-500`, `text-brand-700` and `ring-brand-200` exist. Declaring `--shadow-lift` makes `shadow-lift` a class. The palette is therefore a single source of truth, and re-theming is a CSS edit rather than a search-and-replace across 67 files.

Step 2: depth as a small set of named effects

Class	Effect	How it achieves it
`.panel`	A physical slab floating above the page	Five stacked shadows: a bright inset line along the top edge (the lit bevel), a dark inset line along the bottom (the unlit edge), then three outer shadows at increasing blur. Plus a translucent white gradient and `backdrop-filter: blur(14px) saturate(140%)`, so the coloured background bleeds through.
`.panel-edge`	A hairline of brand colour along the top	A `::before` pseudo-element, 2px tall, with a horizontal gradient that fades to transparent at both ends.
`.panel-lift`	Raise on hover	`translateY(-3px)` plus a deeper shadow, eased with a `cubic-bezier(.22,1,.36,1)` curve — fast out, slow settle.

Class	Effect	How it achieves it
`.press-3d`	Buttons that feel physical	A lit top face via `--shadow-control` (an inset white highlight over a coloured drop), and on `:active` a 2px translate with a 1.5% scale-down.
`.orb`	Soft coloured glow behind dark surfaces	A circle with `filter: blur(58px)` and `pointer-events: none`, positioned absolutely. Three of them sit behind the sidebar.
`.text-aurora`	Gradient text for headings	A linear gradient clipped to the glyphs with `background-clip: text` and `color: transparent`.

> **Why "3D" here is not real 3D**
>
> No WebGL, no Three.js, no perspective transforms on the content. The depth is entirely *shadow and light* — the same trick a flat icon set uses to look raised. That is a deliberate choice: it costs nothing to render, works identically on a 5-year-old laptop, prints correctly, and does not fight with text legibility. A genuine 3-D scene in an attendance table would be slower and harder to read.

Step 3: motion that can be switched off

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: .01ms !important;
    transition-duration: .01ms !important;
    scroll-behavior: auto !important;
  }
  .panel-lift:hover { transform: none; }
}
```

This respects an operating-system setting used by people for whom animation is genuinely unpleasant or triggers vestibular problems. It costs four lines and is the right thing to do.

8.4 Routing and permissions in the browser

The entire navigation structure of the product is `App.tsx`, and it reads almost like prose:

```
// frontend/src/App.tsx
<Routes>
  <Route element={<GuestOnly />}>                 // signed-out users only
    <Route path="/login" element={<LoginPage />} />
    <Route path="/reset-password" element={<ResetPasswordPage />} />
  </Route>

  <Route element={<RequireAuth />}>                // anyone signed in
    <Route element={<AppShell />}>                 // sidebar + header
      <Route path="/dashboard" element={<DashboardPage />} />
      <Route path="/profile"    element={<ProfilePage />} />

      <Route element={<RoleGuard roles={["admin","faculty"]} />}>
        <Route path="/live"      element={<LivePage />} />
        <Route path="/sessions"  element={<SessionsPage />} />
        <Route path="/review"    element={<ReviewPage />} />
        <Route path="/reports"   element={<ReportsPage />} />
      </Route>

      <Route element={<RoleGuard roles={["admin"]} />}>
        <Route path="/students"    element={<StudentsPage />} />
        <Route path="/admin/users" element={<AdminUsersPage />} />
      </Route>

      <Route element={<RoleGuard roles={["student"]} />}>
        <Route path="/privacy"         element={<PrivacyPage />} />
        <Route path="/mark-attendance" element={<MarkAttendancePage />} />
      </Route>
    </Route>
  </Route>
</Routes>
```

The guards are three tiny components. `RequireAuth` handles three cases in order:

```
if (status === "loading") return <FullPageSpinner label="Checking your session" />;
if (!user) return <Navigate to="/login" replace state={{ from: location.pathname }} />;
if (roles && !roles.includes(user.role)) return <Navigate to="/unauthorized" replace />;
return <Outlet />;
```

These guards are for experience, not for security

Client-side routing can be bypassed by anyone who types a URL or edits JavaScript. The guard exists so a student does not briefly see an admin table and get a confusing error — it is a usability feature. The **security** is the `Depends(require_roles("admin"))` on the server endpoint, which cannot be bypassed. If you remember one thing from this chapter, remember that distinction.

8.5 State: where data lives in the browser

Kind	Where	Example
Global, slow-changing	`AuthContext`	The signed-in user. Read by the shell, the guards and half a dozen pages; changed only on login, logout or refresh.
Per-screen server data	Local `useState` in the page	`roster`, `sessions`, `frame.results`. Fetched on mount, re-fetched after a mutation.
Per-screen UI state	Local `useState`	`scanning`, `drawerOpen`, `markBusy`, the selected tab.
URL state	`useSearchParams`	The session id on `/live?session=12`, filters on the reports page — so a screen can be shared or bookmarked.
Ref (not reactive)	`useRef`	The `<video>` element and the `MediaStream`. They must survive re-renders without triggering one.

`AuthContext` is small enough to read whole, and it is the clearest illustration of React Context:

```
// frontend/src/context/AuthContext.tsx
const refresh = useCallback(async () ⇒ {
  try {
    const me = await api<Me>("/auth/me");
    setUser(me); setStatus("authenticated");
  } catch {
    setUser(null); setStatus("anonymous");     // a failed /auth/me simply means "not signed in"
  }
}, []);

useEffect(() ⇒ { void refresh(); }, [refresh]);   // runs once when the app mounts
```

On a cold page load the app does not know whether you are signed in — the answer is in a cookie the JavaScript cannot read. So it asks the server. Until the answer arrives, `status` is `"loading"` and the guards render a spinner rather than bouncing you to the login page. That one detail is what makes a hard refresh of a deep link work correctly.

8.6 The camera: from webcam to server

This is the most unusual part of the front end and worth following carefully.

Opening the camera

```
// frontend/src/lib/camera.ts
stream = await navigator.mediaDevices.getUserMedia({
  video: { width: { ideal: 1280 }, height: { ideal: 720 }, facingMode: "user" },
  audio: false,
});
video.srcObject = stream;
await video.play();
```

- The browser shows a permission prompt. Nothing is captured until the user allows it.
- Errors are translated into sentences: `NotAllowedError` becomes "Camera permission was denied. Allow it in the browser and try again." A raw DOMException is never shown.
- `audio: false` matters — requesting the microphone too would change the permission banner and alarm people for no reason.

Capturing a frame

```
const canvas = document.createElement("canvas");
canvas.width = video.videoWidth; canvas.height = video.videoHeight;
canvas.getContext("2d").drawImage(video, 0, 0, canvas.width, canvas.height);
canvas.toBlob(resolve, "image/jpeg", 0.9);   // encode in memory; nothing uploaded from here
```

A `<canvas>` is an in-memory bitmap the page can draw into. Copying the current video frame into it and encoding as JPEG gives a small binary blob — typically 40–120 KB at 720p — ready to post.

The sampling loop

```
// frontend/src/pages/LivePage.tsx:183-224
useEffect(() => {
  if (!scanning || !sessionId || session?.status !== "active") return;
  const interval = Math.max(600, settings?.frame_interval_ms ?? 1000);
  const timer = setInterval(async () => {
    const blob = await captureFrame(video);
    const body = new FormData();
    body.append("frame", frameFile(blob));
    body.append("session_id", String(sessionId));
    const response = await api<RecognizeResponse>("/recognition/recognize",
      { method: "POST", body });
    setFrame(response);          // draws boxes over the preview
    response.results.forEach(pushFeed);
  }, interval);
  return () => clearInterval(timer);   // cleanup: the loop stops when any dependency changes
}, [scanning, sessionId, session?.status, camera.active, camera.videoRef,
    settings?.frame_interval_ms, loadSelected, pushFeed]);
```

Three things to notice:

1. **The server owns the interval.** `frame_interval_ms` comes from `/api/settings/client`, so an administrator can tune sampling without a rebuild. The `Math.max(600, …)` floor stops a mis-set value from flooding the API.
2. **The loop cannot start on a session that is not active.** The guard is on `session?.status !== "active"` — and the server independently refuses the same request with a 400. Two checks, one for experience and one for truth.
3. **Cleanup is explicit.** Returning `clearInterval` from the effect is how React is told to stop the timer when the page unmounts or a dependency changes. Without it, navigating away would leave a loop posting frames forever.

Drawing the boxes

The response includes a bounding box for each detected face. The page overlays a scaled `<div>` on the preview for each one, coloured by outcome — emerald for `recognized`, amber for `low_confidence`, rose for `liveness_failed`. The scaling matters: the video element is displayed at a CSS size that differs from its pixel size, so box coordinates are multiplied by the ratio.

> **"Attendance recorded" only when it really was**
>
> The overlay text is driven by `attendance.committed === true`, which the server sets only after the database transaction commits. If two frames catch the same student at the same moment, one insert wins and the other returns `committed: false` with outcome `duplicate`. The interface therefore cannot claim a success the database did not have.

8.7 Charts

`src/components/charts/AttendanceCharts.tsx` wraps Recharts in three components — a trend line, a course-comparison bar set, and a status donut. Two rules keep them honest:

- Every colour comes from `lib/reportColors.ts`, so a status is the same colour in the chart, the table and the badge.
- Empty data renders the empty state, not a zeroed chart. A graph with no points is a question about the data, not a flat line at zero.

8.8 Accessibility and the small usability work

Not glamorous, but part of the build:

Practice	Where
Decorative icons hidden from screen readers	Every lucide icon carries `aria-hidden`.
Icon-only buttons labelled	`aria-label="Sign out"`, `aria-label="Open navigation"` in `AppShell.tsx`.
Visible keyboard focus	`:focus-visible { outline: 2px solid var(--color-brand-500) }` in the base layer.
Labels bound to inputs	`Field` generates matching `id` / `htmlFor`.
Busy state announced	`aria-busy` on a loading `Button`, which is also disabled so a form cannot double-submit.
Mobile layout	The sidebar becomes a drawer under `lg:`; the header gains a menu button; the grid collapses to one column.
Plain-language errors	`ApiError` messages are written for a person, not a log file: "This session is closed for attendance", not "409".

8.9 Building and serving the front end

```
cd frontend
npm install            # download the declared dependencies
npm run dev            # Vite dev server on :5173, proxying /api to :8000
npm test               # vitest: 57 tests in jsdom
npm run lint           # oxlint
npm run build          # tsc -b (type check) then vite build → dist/
```

`npm run build` runs the TypeScript compiler first, so a type error stops the build rather than shipping. The output in `dist/` is a handful of static files: an `index.html`, hashed JavaScript bundles, and CSS. Those files are what nginx (or, on this machine, Caddy) serves.

> **One operational consequence worth knowing**
>
> Because the production server serves `dist/` directly from disk, rebuilding the front end makes the change live on the next page load with no restart. Knowing that saves a confusing debugging session after a deploy.

CHAPTER 9

The API — what it is and why

The contract between the two halves of the system: 78 endpoints, one error shape, one authentication story, and a small number of rules applied consistently.

9.1 What an API actually is

> **API — APPLICATION PROGRAMMING INTERFACE**
> A published agreement about how one program may ask another for things. It specifies the address, the method, the shape of the data going in, the shape of the data coming back, and the ways it can fail. It is a *contract*: both sides are allowed to rely on it, and breaking it is a real event, not a style preference.

The word is used loosely on the internet, so here is the precise thing in this project: a set of HTTP endpoints under `/api`, served by FastAPI, that the React app calls. There is no other way for the front end to affect the world. It cannot touch the database, cannot run the face models, and cannot decide anything.

9.2 Why the boundary is worth the effort

Benefit	What it bought this project
Two languages, one seam	The front end is TypeScript, the back end is Python. JSON is the only thing they share, so each side can be rewritten without touching the other.
Free documentation	FastAPI reads the type annotations and generates an interactive reference at `/api/docs`, plus a machine-readable OpenAPI document at `/api/openapi.json`. It is always current because it is derived from the code, not written by hand.
Testability	The 97 backend tests call the API directly with an in-process HTTP client. No browser is involved, so a full attendance scenario runs in milliseconds.
A place to enforce rules	Authentication, role checks, ownership, rate limits and validation all sit at one boundary rather than being repeated in every screen.
A future second client	A mobile app or a kiosk could be added against the same contract with no server change.

9.3 The conventions

Consistency is the whole value of a contract. These are the rules every one of the 78 endpoints follows:

Topic	The rule
Base path	Everything lives under `/api`. That single prefix is what lets nginx forward all application traffic to one upstream.
Format	JSON in and out, except image uploads (`multipart/form-data`) and exports (CSV/PDF file attachments).

Topic	The rule
Authentication	The `sa_session` cookie. There is no API key, no bearer token, and no token in a URL.
CSRF	Every unsafe method (`POST` , `PUT` , `PATCH` , `DELETE`) requires an `X-CSRF-Token` header matching the `sa_csrf` cookie. Login and password reset are exempt because they are the requests that establish a session.
Pagination	`?page=1&page_size=20` , maximum 100 per page, always returning `{items, total, page, page_size}` .
Filtering	Query parameters, not bodies: `?course_id=3&status=active&date_from=2026-01-01` .
Times	ISO-8601 with an explicit timezone, always UTC on the wire. The institution's local timezone is a display concern.
Errors	One envelope, described in §9.5.
Trust	Identity, role, attendance status and eligibility are derived server-side. A client-sent `student_id` is never believed for a decision about who that client is.

9.4 The endpoint map

78 endpoints grouped by resource. The role column uses **A** admin, **F** faculty (own courses only), **S** student (own data only).

Group	Count	Roles	Representative endpoints
Health & settings	4	mixed	`GET /health` · `GET/PATCH /settings` · `GET /settings/client`
Authentication	5	mixed	`POST /auth/login` · `POST /auth/logout` · `GET /auth/me` · `POST /auth/change-password` · `POST /auth/reset-password`
Users	5	A	`GET/POST /users` · `PATCH /users/{id}` · `POST /users/{id}/reset-link` · `POST /users/{id}/password`
Students & faculty	13	A	list, create, read, edit, deactivate, create account, set password
Courses & enrolment	7	A, F	`GET /courses/{id}/enrollments` · `POST …/enrollments` (bulk, idempotent) · `DELETE …/enrollments/{sid}`
Consent & biometrics	5	A, S(self)	`GET /privacy/policy` · `POST /enrollment/{id}/consent` · `POST /enrollment/{id}` · `POST /biometrics/{id}/delete`
Recognition & liveness	5	A, F, S(self)	`POST /recognition/detect` · `POST /recognition/recognize` · `GET /recognition/events` · `POST …/events/{id}/resolve` · `POST /liveness/verify`
Sessions & attendance	18	A, F, S	the largest group — see §9.6
Corrections	6	A, F, S	submit, list, read, approve, reject, withdraw
Student self-service	2	S	`GET /me/summary` · `GET /me/attendance`

Group	Count	Roles	Representative endpoints
Reports	5	A, F	`/reports/summary` · `/trends` · `/activity` · `/attendance` · `/export`
Audit	1	A	`GET /audit` with actor, action, entity and date filters

Where the counts come from

`sessions.py` alone declares 18 route handlers; `students.py` and `courses.py` declare 7 each; `review.py` and `me.py` declare 2. You can count them yourself with `grep -c '@router\.' backend/app/api/routes/*.py`.

9.5 Errors: one shape for every failure

Every error the API produces, from any layer, arrives at the client in this envelope:

```
{
  "error": {
    "code": "session_overlap",
    "message": "Another session for this course overlaps the requested time.",
    "details": { "conflicting_session_id": 14 }
  }
}
```

The two fields have different audiences, and that is the whole design:

- `code` is **for programs**. Stable, lowercase, snake_case, never reworded. A client can branch on it: `if (e.code === "consent_required")` opens the consent screen.
- `message` is **for people**. Written as a sentence a student or a teacher can act on. It may be rephrased at any time without breaking anything.

The mechanism is a single exception class and four handlers registered in `backend/app/core/errors.py`:

```
class AppError(Exception):
    def __init__(self, status: int, code: str, message: str, details: Any = None): …

@register_error_handlers
  AppError              → its own status and code
  RequestValidationError→ 422 "validation_error", with the offending fields in details
  StarletteHTTPException→ mapped to a code (401 unauthenticated, 403 forbidden, 404 not_found…)
  Exception             → 500 "internal_error" with a generic message
```

Why the catch-all hides its details

An unexpected exception is logged with its traceback and a request id, but the client only ever sees "An unexpected error occurred." A stack trace in a response tells an attacker which libraries and versions are in use, and can leak file paths or query fragments. The request id lets the user report the problem and lets staff find the exact log line — without revealing anything.

Status codes in use

Code	Meaning here	Example
`200`	OK	Any successful read.
`201`	Created	`POST /sessions` returns the new session.
`204`	No content	A delete where there is nothing worth returning.
`400`	Business rule violated	`invalid_transition` — closing a session that was never started.
`401`	Not signed in	No cookie, expired cookie, or idle-timed-out cookie.
`403`	Signed in, not allowed	A student calling an admin endpoint; a failed CSRF check.
`404`	Not found — *or* not yours	A faculty member asking for another teacher's session gets 404, not 403.
`409`	Conflict	`session_overlap`, `already_enrolled`, `window_closed`, `pending_exists`.
`413`	Upload too large	A frame bigger than `MAX_UPLOAD_BYTES` (2 MB by default).
`415`	Unsupported media	Uploading something that is not a JPEG or PNG.
`422`	Validation failed	`weak_password`, `invalid_time_range`, `invalid_window`.
`429`	Rate limited	More than 10 login attempts per minute from one IP.
`503`	Dependency unavailable	The face models are missing from disk.

> **WHY 404 INSTEAD OF 403?**
>
> Because 403 confirms existence. If a teacher probing random session ids gets 403 for some and 404 for others, they have learned which sessions exist without being allowed to see any of them. Returning 404 for both cases leaks nothing. This is an *information-disclosure* decision, and it is the kind of thing that separates a designed API from an accidental one.

9.6 The session endpoints in detail

The largest group, and the one that carries the product's rules:

Method	Path	Roles	Purpose
GET	`/sessions`	A, F	List with `course_id`, `status` and date filters, plus per-session counts.
POST	`/sessions`	A, F	Schedule a session. Validates the window and rejects overlaps.
GET	`/sessions/mine`	S	This student's sessions, each with their own record status and whether self-marking is open.

Method	Path	Roles	Purpose
GET/PATCH	`/sessions/{id}`	A, F	Read; edit only while `scheduled`.
POST	`/sessions/{id}/start`	A, F	`scheduled → active`.
POST	`/sessions/{id}/pause` · `/resume`	A, F	Pause and resume the camera window.
POST	`/sessions/{id}/close`	A, F	Stop accepting marks.
POST	`/sessions/{id}/finalize`	A, F	Generate `absent` for unmarked roster members. Frozen afterwards.
POST	`/sessions/{id}/cancel`	A, F	Only from `scheduled`.
POST	`/sessions/{id}/reopen`	A	`closed → active`, reason mandatory, audited.
GET	`/sessions/{id}/roster`	A, F	Eligible participants with current status.
GET/POST	`/sessions/{id}/attendance`	A, F	Read records; write a manual mark with a reason.
PATCH	`/sessions/{id}/self-marking`	A	Turn student self-marking on or off for the whole session.
PATCH	`/sessions/{id}/participants/{sid}/self-marking`	A, F	Lock or unlock one student, with a reason.
POST	`/sessions/{id}/self-mark`	S	Mark themselves present. 409 if the window is closed, they are locked, or a record exists.

9.7 Rate limits

An in-process sliding-window limiter (`backend/app/core/ratelimit.py`) sits in front of everything. Each key holds a deque of timestamps; old entries are popped, and the request is refused if the deque is still full.

Route group	Limit	Keyed by
`/auth/login`	10 per minute	IP, plus a separate 15-minute account lockout after 5 consecutive failures
`/auth/reset-password`	10 per 15 minutes	IP
`/recognition/*`, `/liveness/*`	240 per minute	user id — one frame per second is 60, so this allows four concurrent stations
everything else	600 per minute	IP

> **The honest limitation**
>
> The counters live in the process's memory, which is correct for the single-worker deployment this project targets and wrong for a horizontally scaled one. Raising `UVICORN_WORKERS` multiplies every limit by the number of

> workers. The documented fix is a shared store such as Redis. **Planned** — and stated in both the API contract and the deployment guide rather than left as a surprise.

9.8 Self-documenting API

Because FastAPI generates its own reference from the type annotations, the running system always carries an accurate description of itself:

- `/api/docs` — Swagger UI. Try requests against the live server, see real responses, see the exact schema of every request body.
- `/api/openapi.json` — the machine-readable OpenAPI document, the input to a code generator or an integration test harness.

The docs page is public by design and exposes no secrets: it lists endpoints and shapes, never data. This is one of the few places a beginner can watch the contract directly instead of reading about it.

CHAPTER 10

The back end, in detail

FastAPI, the service layer, the state machine, and the rules that make attendance trustworthy.

10.1 What FastAPI is and why it was chosen

WEB FRAMEWORK

The layer that turns an HTTP request into a function call and a return value into an HTTP response, and handles everything in between: routing, parsing, validation, serialisation, error conversion. You write the rules; the framework moves the bytes.

FastAPI feature	What it does for this project
Type-driven validation	A route declares `body: SessionIn`; Pydantic checks the JSON against that model before the function body runs. Bad input never reaches business logic.
Response models	`response_model=Page[SessionOut]` filters the output to declared fields. A field you forget to declare cannot leak — which is exactly the protection you want around biometric data.
Dependencies	`Depends(require_roles("admin"))` composes authentication, authorisation, database sessions and pagination as declarable preconditions.
Generated docs	The interactive reference at `/api/docs` comes free from the same annotations.
Async	Long database waits do not block other requests, which matters when one teacher's browser is posting frames every second.

Why not Django or Flask? Django bundles an admin interface, an ORM and a template system that this project does not use, because the front end is a separate SPA. Flask is lighter but has no built-in validation or serialisation, so those would be hand-rolled — which is FastAPI's main feature. The deciding factor was **Pydantic integration**: this project's most important property is that nothing unvalidated reaches a decision, and FastAPI makes that the default path rather than a discipline.

10.2 Request lifecycle, in code

Every request passes through the same seven stages. Knowing the order explains most behaviour you will ever need to debug.

1. **Middleware** (`main.py`) assigns or echoes an `X-Request-ID`, then on the way out adds the security headers and `Cache-Control: no-store`.
2. **CORS** decides whether a cross-origin browser request is allowed at all. In production the list is empty because the SPA and API share one origin.
3. **Rate limit** — the default 600/min per IP is a dependency on the parent router, so it applies before any route body.

4. **Authentication dependency** (`deps.get_auth`) reads the cookie, hashes it, finds the session row, checks expiry and idle timeout, and enforces the CSRF header on unsafe methods.
5. **Authorisation dependency** — `require_roles( ... )`, or the narrower `staff_scope` / `student_or_forbidden` helpers in `backend/app/api/access.py`.
6. **Validation** — Pydantic parses the body, path and query parameters.
7. **Route body** → **service** → **ORM** → **commit**, then serialisation back to JSON.

10.3 Authentication, precisely

This is the part most worth understanding in detail, because it is where projects usually get things wrong.

Logging in

```
// backend/app/services/auth_service.py (condensed)
user = find_by_email(db, email)
if user is None:
    verify_password(_DUMMY_HASH, password)     ① burn the same time as a real check
    raise AppError(401, "invalid_credentials", "Email or password is incorrect")
if user.locked_until and user.locked_until > now():
    …same generic 401…                          ② never say "you are locked out"
if not verify_password(user.password_hash, password):
    user.failed_login_attempts += 1
    if user.failed_login_attempts ≥ 5: user.locked_until = now() + 15 min
    raise AppError(401, "invalid_credentials", "Email or password is incorrect")

token = secrets.token_urlsafe(32)               ③ 256 bits of randomness
db.add(AuthSession(user_id=user.id, token_hash=sha256(token), expires_at=…))
response.set_cookie("sa_session", token, httponly=True, samesite="lax",
secure=settings.cookie_secure)
response.set_cookie("sa_csrf",  new_token(),   samesite="lax", secure=settings.cookie_secure)
```

Five details, each of which is a defence:

Detail	What it prevents
The same message for "no such user" and "wrong password"	Account enumeration. An attacker cannot discover which email addresses are registered by watching the responses.
Hashing a dummy password when the user does not exist	A timing side channel. Argon2id takes tens of milliseconds; if the "unknown email" path returned instantly, response time alone would reveal whether an account exists.
Only `sha256(token)` stored in the database	A database leak becoming a session hijack. The plaintext token exists only in the browser's cookie jar.
`HttpOnly` on the session cookie	Theft by injected JavaScript. A cross-site scripting bug cannot read a cookie the page cannot see.
Absolute expiry *and* idle expiry	A session left open in a staff room. `SESSION_TTL_MINUTES=480` caps total life; `SESSION_IDLE_MINUTES=60` ends it after an hour of inactivity.

"I pasted the admin URL in another tab and I was still signed in — is that a bug?"

No, and it is worth understanding why, because this exact question came up during development. The session token is read **only** from the `sa_session` cookie (`backend/app/api/deps.py:32`). Nothing in a URL identifies you. Opening `/admin/users` in a second tab works because the browser automatically attaches the cookie for that origin — the same reason a bookmark still works after you sign in. If the second tab were a different browser profile, the same URL would bounce to the login page.

The only token that ever travels in a URL is the single-use, 24-hour password-reset link, which is a deliberate exception: it must be pasted into an email client and opened on a phone.

CSRF, in one paragraph

A malicious page can make your browser send a request to `/api/students` with your cookies attached — the browser does not care who initiated it. What a malicious page *cannot* do is read the value of your `sa_csrf` cookie, because of the same-origin rule. So the server requires that value to be echoed in an `X-CSRF-Token` header. Our own front end does that automatically in `lib/api.ts` ; an attacker's page cannot. The comparison uses `hmac.compare_digest` , a constant-time function, so the check cannot be probed byte by byte.

10.4 Authorisation: roles plus ownership

Roles alone are not enough. A faculty member is a legitimate, powerful user — and must not be able to touch another teacher's course. So every privileged read or write passes a second question: *is this yours?*

```
// backend/app/api/access.py
def staff_scope(user: User = Depends(current_user)) → User:
    if user.role_name not in (ROLE_ADMIN, ROLE_FACULTY):
        raise forbidden()
    if user.role_name == ROLE_FACULTY and user.faculty is None:
        raise forbidden("Your account is not linked to a faculty profile")
    return user

def can_manage_course(user: User, course: Course) → bool:
    if user.role_name == ROLE_ADMIN:
        return True
    return (user.role_name == ROLE_FACULTY
            and course.faculty_id is not None
            and course.faculty_id == faculty_id_or_none(user))
```

The pattern in the route layer, repeated deliberately:

```
scope = None if actor.role_name == ROLE_ADMIN else faculty_id_or_none(actor)
stmt  = session_service.search_sessions(db, … , faculty_scope=scope)
```

SCOPE-PUSHING

Rather than fetch everything and filter in Python, the ownership restriction is pushed into the SQL query as an extra condition. Two benefits: it is impossible to forget the filter on one code path, and the database never returns rows the caller was not entitled to see.

10.5 The service layer

Thirteen modules, 3,449 lines, and the only place business rules live.

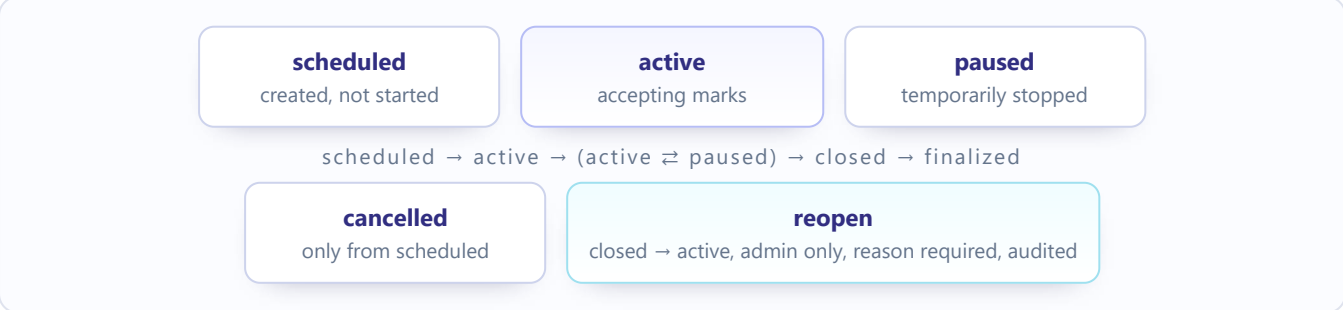
Module	Lines	Owns
`reports_service.py`	726	Every aggregate: dashboard cards, trend series, ten report types, CSV and PDF export.
`session_service.py`	410	The session state machine, window calculation, overlap detection, roster snapshot, self-marking.
`biometric_service.py`	361	Consent, sample validation, template encryption, duplicate-identity check, deletion.
`people_service.py`	319	Student and faculty records, deactivation, portal accounts.
`recognition_service.py`	307	The frame pipeline: analyse, liveness, match, decide, record, log the event.
`attendance_service.py`	239	Manual marking, counts, finalisation, percentage rules.
`correction_service.py`	237	Correction requests and their one-transaction approval.
`course_service.py`	223	Courses and enrolment, including roster sync into open sessions.
`auth_service.py`	181	Login, session resolution, logout, password changes, reset tokens.
`review_service.py`	171	The exception queue and reviewer decisions.
`settings_service.py`	114	Application settings with validated ranges.
`biometric_policy.py`	57	The consent notice text and its version.
`audit_service.py`	51	The append-only audit writer, with sensitive keys stripped.

WHY NOT PUT THE RULES IN THE ROUTE?

Because a route function can only be tested through HTTP — you need a client, cookies, a database and a request. A service function is called with plain arguments. `_validate_window` can be tested with four datetimes and no infrastructure at all. Keeping policy out of the HTTP layer is what makes 97 fast tests possible.

10.6 The session state machine

A session moves through a fixed set of states, and only certain moves are legal. Representing that as data rather than as scattered `if` statements is what stops a whole category of bug.



```
// backend/app/services/session_service.py
def _transition(db, session_id, *, allowed: tuple[str, ...], new_status: str, action: str, …):
    session = _lock(db, session_id)                    ① SELECT … FOR UPDATE
    if session.status not in allowed:                  ② the only legal source states
        raise AppError(400, "invalid_transition",
                       f"Cannot {action} a {session.status} session",
                       {"current_status": session.status, "allowed_from": list(allowed)})
    …
    audit(db, action=…, before={"status": old}, after={"status": new_status}, reason=…)
```

- ① **Row lock.** `SELECT … FOR UPDATE` means two teachers clicking "start" simultaneously queue instead of both reading `scheduled` and both writing `active`.
- ② **Declarative guard.** Every transition names its allowed sources. There is no way to reach `active` from `finalized` because no transition lists it.
- ③ **Audit.** The reason, actor, timestamp and both states are recorded in the same transaction.

10.7 The attendance window — a rule worth getting exactly right

This is the single most misunderstood part of the system, including by the person who wrote it (there is a documentation correction in the project history for precisely this). State it precisely:

> **Both thresholds are measured from the START of the session**
>
> `late_after = scheduled_start + late_after_min` (default 10 minutes)
>
> `attendance_closes_at = scheduled_start + window_min` (default 30 minutes)
>
> The window is *not* measured from the end. A one-hour lecture starting at 10:00 has an attendance window that closes at **10:30**, not 11:30. If you need the window to cover the whole class, set `window_min` explicitly when scheduling — the demo session in this deployment uses `window_min: 60` for exactly that reason.

```
// backend/app/services/session_service.py:40
def _validate_window(start, end, late_min, window_min):
    if end ≤ start:
        raise AppError(422, "invalid_time_range", "The session must end after it starts")
    if late_min > window_min:
        raise AppError(422, "invalid_window", "The late cutoff must fall inside the attendance
window")
    return start + timedelta(minutes=late_min), start + timedelta(minutes=window_min)
```

The database enforces the same ordering independently, as a `CHECK` constraint:

```
CHECK (scheduled_start ≤ late_after ≤ attendance_closes_at)
```

> **DEFENCE IN DEPTH**
>
> The rule is written twice — once in Python, once in the schema. If a future code path forgets the check, the database refuses the row. Application bugs should degrade into rejected writes, not into corrupt history.

10.8 Overlap protection

Two sessions for the same course cannot run at once, because a student could not be in both and the roster would become ambiguous:

```
def _assert_no_overlap(db, course_id, start, end, exclude_id=None):
    # conflicts with any session in scheduled / active / paused whose time range intersects
    raise AppError(409, "session_overlap", "Another session for this course overlaps the requested time.")
```

Note the status filter: `closed`, `cancelled` and `finalized` sessions do not block a new scheduling, because they are history.

10.9 Student self-marking

The most recent feature added to the system, and a good example of a rule with three independent guards.

The motivation: a student who walks in while the teacher is mid-explanation should not be marked absent for the teacher's busyness. So a session can open a self-marking window.

The guards (all evaluated server-side, in `session_service.self_marking_state`):

1. **Opt-in per session.** Self-marking is off unless an admin turned it on with `PATCH /sessions/{id}/self-marking {enabled: true}`.
2. **Time-bounded.** Open only between `scheduled_start` and `attendance_closes_at`. Outside it: 409 `window_closed`.
3. **Per-student lockable.** An admin or the owning faculty member can lock one student out — `self_marking_locked_at` plus a reason — without disturbing the rest of the cohort. That is the answer to "they might mark and walk out".

And the honesty measures:

- The record is written with `source = self_reported`, permanently distinguishable from a biometric match or a staff mark.
- Marking after `late_after` produces `late`, not `present`.
- An existing record is never overwritten: a second self-mark returns the first record rather than creating a duplicate. The unique index on `(session_id, student_id)` makes that impossible even under a race.
- `GET /sessions/mine` returns `self_marking_open`, `self_marking_message` and `self_marking_closes_at`, so the interface can explain exactly why a button is absent instead of leaving the student guessing.

10.10 Configuration that refuses to be wrong

Settings come from the environment through a Pydantic model with validators that make dangerous configurations impossible rather than merely discouraged:

```
// backend/app/core/config.py
@model_validator(mode="after")
def _isolate_test_database(self):
    if self.app_env == "test":
        if not self.test_database_url:
            raise ValueError("TEST_DATABASE_URL must be set when APP_ENV=test")
        if self.test_database_url == self.database_url:
            raise ValueError("TEST_DATABASE_URL must differ from DATABASE_URL")
        self.database_url = self.test_database_url
    return self

@model_validator(mode="after")
def _production_hardening(self):
    if self.app_env == "production" and not self.biometric_encryption_keys.strip():
        raise ValueError("BIOMETRIC_ENCRYPTION_KEYS must be set when APP_ENV=production")
    return self
```

The first one is the reason the test suite physically cannot wipe development data: run the tests with `APP_ENV=test` and no separate test database and the process refuses to start. The second is the reason a production deployment can never quietly store biometric templates in the clear.

> **Fail at start-up, not at 09:00 during a lecture**
>
> Both checks run when the settings object is first built, which is before the server binds its port. A misconfiguration is a crash in front of an operator with a terminal, not a silent data loss in front of a class.

10.11 The audit trail

Every privileged write goes through `audit_service`, which appends a row with the actor, their role, the action, the entity, before and after snapshots, an optional reason, the IP and the request id. Two properties matter:

- **Append-only.** There is no update or delete path for audit rows, and no API exposes one. The only read endpoint is admin-only.
- **Sanitised.** The helper strips known sensitive keys — password fields, tokens, embeddings — before serialising a snapshot. An audit log that leaks credentials would be a worse problem than the one it documents.

CHAPTER 11

The database

Seventeen tables, the constraints that make attendance impossible to fake, and the reasoning behind every non-obvious choice.

11.1 What the database is for here

PostgreSQL is the system's memory. Everything that must survive a restart, a crash or a redeploy lives in these 17 tables. The application tier is deliberately stateless — any request can hit any worker — which is only possible because all durable facts are here.

Three properties make PostgreSQL the right choice for a project like this:

Property	Why it matters for attendance
Constraints are enforced by the database	A unique index on `(session_id, student_id)` makes a duplicate attendance record impossible even if two concurrent requests both pass every Python check. The rule is not "remember to check"; it is "the storage engine will refuse".
Real transactions	Writing a record and its evidence event must succeed together or not at all. Partial commits would create attendance with no explanation.
It is boring and everywhere	Free, well documented, installable on any institution server, and no examiner has ever asked "but is PostgreSQL reliable?". Choose boring technology for the layer that holds the permanent record.

> **Why not SQLite?**
>
> SQLite would run with zero setup, which is genuinely attractive for a student project. It loses on two fronts: concurrent writes lock the whole file, which is exactly the pattern of a live session where several requests write at once; and it has no `SELECT … FOR UPDATE`, no `ON CONFLICT … RETURNING` with the same semantics, and no `TIMESTAMPTZ`. The state machine in §10.6 depends on all three.

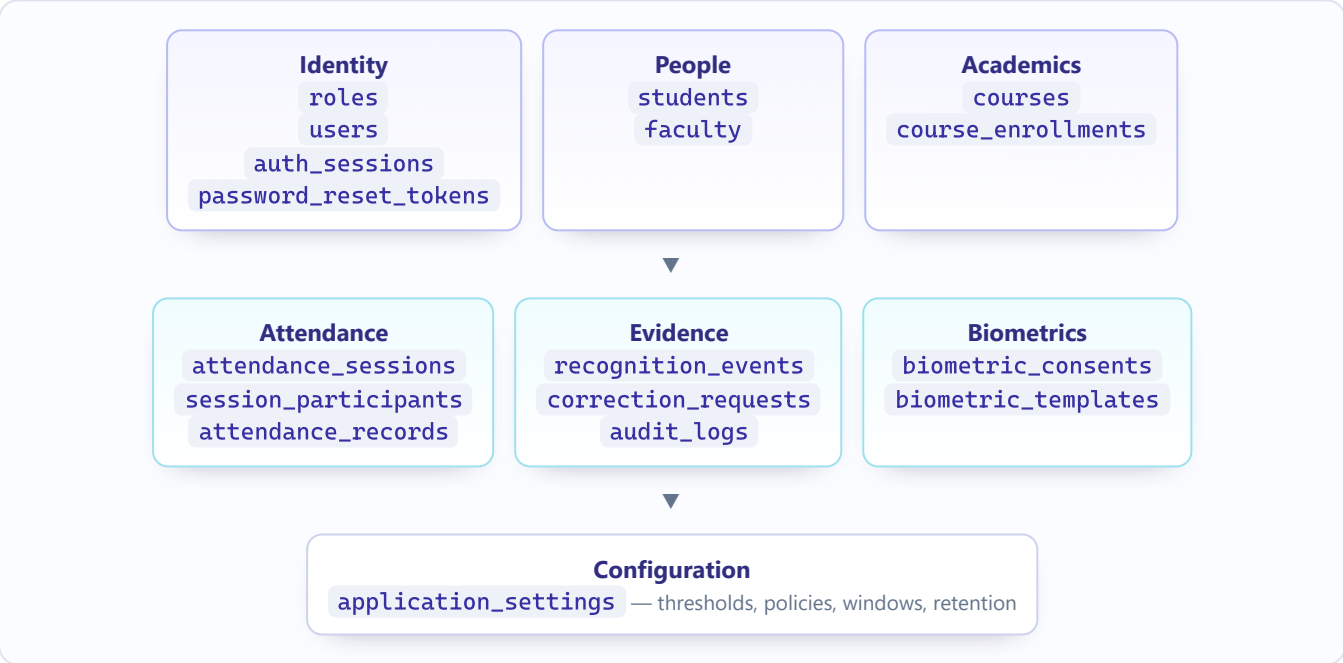
11.2 The conventions

Convention	Reason
Primary keys are `BIGINT GENERATED BY DEFAULT AS IDENTITY`	A single auto-incrementing integer per table. Simple, indexable, and never reused.
Timestamps are `TIMESTAMPTZ`, written in UTC	Storing local times creates ambiguity at daylight-saving boundaries and makes ordering across timezones wrong. The institution's timezone (`APP_TIMEZONE`) is applied only when interpreting "today" or formatting for display.
`created_at` / `updated_at` on every mutable table	Forensics and debugging. When a record looks wrong, the first question is when it changed.

Convention	Reason
Enumerations are `VARCHAR` + `CHECK`, not native PG enums	Adding a value to a native enum is an awkward migration; adding one to a `CHECK` list is a one-line swap. Portability and reversibility beat purity here.
Attendance foreign keys use `ON DELETE RESTRICT`	History is never silently deleted. Students and faculty are *deactivated* by the API; there is no delete path at all.
One exception: `biometric_templates` cascades on student deletion	The opposite policy would be a privacy bug. If a student's record is removed, their face templates must go with it automatically.

11.3 The entity map

The 17 tables fall into six clusters. Reading them as clusters is much easier than reading them one by one:



11.4 The core relationships

In words, the chain that produces an attendance record:

1. A **course** exists and has a **faculty** member assigned.
2. Students are linked to the course by **course_enrollments** rows — the roster.
3. The faculty member schedules an **attendance_session** for that course.
4. At creation, the roster is copied into **session_participants**. This snapshot is the eligibility list for that one session.
5. Each recognised, marked or self-marked student gets one **attendance_record**: one row per (session, student), ever.
6. Every automated attempt — successful or not — writes a **recognition_event**, which the record points at as its evidence.

7 Any privileged change writes an **audit_log** row.

> **WHY SNAPSHOT THE ROSTER AT ALL?**
> Because enrolments change. If a student drops a course in week 8, attendance from week 3 must still be explainable — "you were on this class list on that date". Copying the roster into `session_participants` freezes the eligibility question per session. Closed, cancelled and finalized sessions keep their snapshot forever; only `scheduled`, `active` and `paused` sessions are re-synced when the enrolment list changes.

11.5 The tables that matter most

`attendance_records` — the product's actual output

Column	Type	Note
`session_id`, `student_id`	FK, RESTRICT	**UNIQUE together.** This single constraint is the duplicate guard for the whole system.
`status`	VARCHAR + CHECK	`present`, `late`, `absent`, `pending_review`, `excused`.
`source`	VARCHAR + CHECK	`automated`, `manual`, `system`, `self_reported` — how the mark came to exist.
`marked_at`	TIMESTAMPTZ	The timestamp used to decide present vs late.
`recognition_event_id`	FK, SET NULL	The evidence. Nullable because a manual mark has none.
`marked_by_user_id`	FK	Who, for manual marks.
`reviewer_id`, `correction_reason`, `corrected_at`	FK / TEXT / TS	The latest correction. Full history is in the audit log, not here.

`recognition_events` — the audit of the AI

One row per analysed face that led to a decision (cooldown-suppressed repeats are not stored). It records what the model thought *and* what the system did about it:

Columns	Content
`outcome`	`recognized`, `unknown`, `low_confidence`, `poor_quality`, `no_face`, `multiple_faces`, `liveness_failed`, `duplicate`, `not_eligible`, `error`
`matched_student_id` / `candidate_student_id`	Who it decided, and who it was unsure about. Visible to reviewers only — never in a student-facing response.
`similarity`, `second_similarity`, `threshold_used`	The numbers behind the decision, so a reviewer can see how close it was.
`quality` (JSONB)	Blur, brightness, face size. No pixels.

Columns	Content
`liveness_status`, `liveness_score`, `liveness_model`	`not_required`, `not_enabled`, `passed`, `failed`, `inconclusive`, `error`.
`model_name`, `model_version`, `processing_ms`	Provenance and timing, so a later model swap is traceable.
`review_status`, `reviewed_by_user_id`, `reviewed_at`, `review_note`	The human follow-up.

There is no image column anywhere in this table

That is the point. The system keeps the *decision and its measurements*, not the photograph. Retaining raw camera frames of a classroom would be the single biggest privacy problem a system like this could have, and it is avoided structurally rather than by policy.

`biometric_templates` — the sensitive one

Column	Content
`student_id`	FK with `ON DELETE CASCADE` — the one cascade in the schema, for privacy.
`consent_id`	FK with `RESTRICT`. **A template cannot exist without a consent row.** The database enforces this; no application bug can bypass it.
`model_name`, `model_version`, `embedding_dim`	Provenance. A template from SFace 2021dec is not comparable with one from a future model, and the row says which it is.
`ciphertext`	AES-256-GCM over the float32 embedding bytes. Not the face. Not an image. 512 bytes of numbers, encrypted.
`nonce`, `key_id`	The per-row random value and which key encrypted it, enabling rotation.
`quality` (JSONB)	Blur/brightness/size of the original sample — metadata about the photo, not the photo.

`application_settings` — the tuning knobs

Key/value rows with JSONB values, seeded by the migration and editable only by an admin through a validated, audited endpoint:

Key	Default	Meaning
`recognition_threshold`	0.42	Cosine similarity needed for a match. Deliberately above OpenCV's suggested 0.363 — conservative by design.
`low_confidence_margin`	0.07	Scores in `[threshold − 0.07, threshold)` become `low_confidence` and go to review rather than being rejected outright.

Key	Default	Meaning
`ambiguity_margin`	0.05	If best minus runner-up is below this, the system is not sure *which* person it saw, so it does not decide.
`liveness_policy`	`"off"`	`off` · `flag` (mark confident identities, queue failures for review) · `enforce` (only mark on a pass).
`liveness_threshold`	0.80	MiniFASNet "real" probability needed to pass.
`frame_interval_ms`	1000	How often the browser sends a frame.
`recognition_cooldown_s`	30	Suppress repeat events for the same person in the same session, so one student does not generate 60 rows while sitting there.
`max_faces_per_frame`	5	More than this in one frame is ambiguous, so it is reported as `multiple_faces` and nothing is marked.
`min_face_px` / `blur_threshold` / `brightness_min` / `brightness_max`	80 / 60.0 / 50 / 210	The quality gates from §12.3.
`default_late_after_min` / `default_window_min`	10 / 30	Both measured from the session **start**.
`faculty_can_correct`	true	Whether faculty may correct records for their own courses.
`biometric_retention_days`	30	Templates are purged this many days after a student is deactivated.

11.6 How the concurrency is handled

The interesting race in this system is two frames arriving at the same instant, both recognising the same student. The answer is to let the database arbitrate:

```
INSERT INTO attendance_records (session_id, student_id, status, source, …)
VALUES (…)
ON CONFLICT (session_id, student_id) DO NOTHING
RETURNING id;
```

- If a row comes back, *this* request created the record. Write the event, commit, tell the client `committed: true`.
- If nothing comes back, someone else already did it. Store the event with outcome `duplicate` and tell the client `committed: false`.

No explicit locking is needed for that case, and it is correct under any isolation level. Session state transitions do need a lock, which is what `SELECT … FOR UPDATE` provides.

11.7 Migrations: how the schema changes safely

A migration is a versioned, replayable script that changes the schema. This project has two: `20261025_0001_initial_schema.py` (everything, plus the seeded roles and default settings) and `20261026_0002_student_self_marking.py` (the self-marking columns and their indexes). Running `alembic upgrade head` applies whatever is pending, in order.

```
cd backend
.venv/Scripts/alembic upgrade head     # apply all pending migrations
.venv/Scripts/alembic revision --autogenerate -m "add something"
```

> **The rule about rollbacks**
>
> Rolling back the *code* does not roll back an applied *migration*. The deployment guide says so explicitly, and the operational answer is: take a backup before upgrading, and if you must go back, restore the backup or write a forward migration. Treating schema changes as one-way is the mature assumption, not a limitation of the tool.

11.8 What "attendance percentage" means here

A deceptively contentious number in any institution, so it is defined exactly:

```
attended  = count(records with status in {present, late})
eligible  = count(finalized sessions where the student is on the roster) − count(excused)
percentage = attended / eligible × 100        (shown as "—" when eligible is 0)
```

- Sessions that are not yet finalized are **excluded**. Their records are shown in history but marked provisional. Including them would let a half-taken session drag a percentage down.
- `pending_review` in a finalized session counts as **not attended** until a human resolves it. Uncertainty never silently becomes credit.
- `excused` is removed from the denominator rather than counted as a miss.
- A zero denominator renders as an em dash, not as 0% and not as 100%.

CHAPTER 12

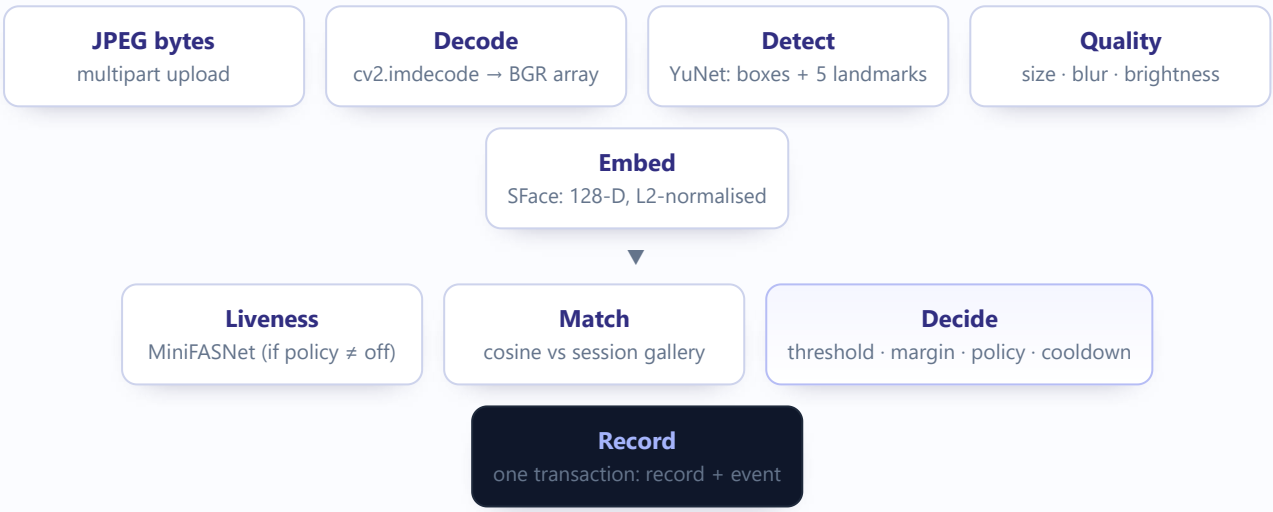
The AI service — face recognition and liveness

How a camera frame becomes a decision, which models do the work, why they were chosen, and exactly what is and is not claimed about them.

12.1 The pipeline in one line

Frame → detect faces → reject bad ones → turn each good one into 128 numbers → compare those numbers against the enrolled templates → decide → record or queue.

Everything in this chapter is an elaboration of that sentence.



Each row runs left to right; the flow continues on the row below.

> **THE MOST IMPORTANT BOUNDARY**
>
> The first five boxes are `ai_service/` — it produces numbers. The last two are `backend/app/services/recognition_service.py` — it makes decisions. The AI package never writes to the database, never knows what a course is, and never decides whether someone is present. That separation is what allows the model to be replaced, tested and audited independently.

12.2 The models and why these ones

Job	Model	Licence	Size	Why
Face detection	YuNet (2023mar)	MIT	227 KB	Tiny, millisecond-fast, and built into OpenCV as `cv2.FaceDetectorYN` — no extra native dependency to install on Windows.

Job	Model	Licence	Size	Why
Face recognition	SFace (2021dec)	Apache-2.0	37 MB	Also built into OpenCV as `cv2.FaceRecognizerSF`. Produces a 128-dimensional embedding, which is small enough to compare hundreds of templates per frame on a laptop CPU.
Liveness **Experimental**	MiniFASNetV2 (2.7_80x80)	Apache-2.0	1.7 MB	Converted from the official Silent-Face-Anti-Spoofing checkpoint by a script in the repo, with a parity check against the PyTorch forward pass.

The rejected option, and why

InsightFace `buffalo_l` was considered and refused

It is the most popular open face-recognition pack and would probably score better. It was rejected for two reasons, and the first is decisive: **its pretrained weights are released for non-commercial research only.** A project that publishes a licence and invites reuse cannot ship a component that forbids it. The second reason is practical — the Python package needs a C++ build toolchain on Windows, which makes the project harder to run for exactly the people it is written for.

This is a good example of a licence being an *engineering* constraint, not a legal formality. Choosing a slightly weaker model you may actually use beats choosing a stronger one you may not.

Every model file is downloaded by `ai_service/scripts/fetch_models.py`, which verifies its SHA-256 checksum against the values recorded in `docs/ai-models.md`, and the weights are never committed. The `--check` flag re-verifies them later.

WHY A CHECKSUM MATTERS

A SHA-256 is a fingerprint of a file's contents. Recording it means "the model I evaluated is byte-for-byte the model you downloaded" is a checkable claim rather than a hope. If a weight file is silently swapped — a different mirror, a corrupted download, a malicious substitute — the verification fails and the script says so.

12.3 Quality gating: refusing to look at a bad face

Before spending time on an embedding, each detected face is measured. This is `ai_service/quality.py` in full:

```
def assess(image, box, policy) -> QualityReport:
    face = crop(image, box)
    gray = cv2.cvtColor(face, cv2.COLOR_BGR2GRAY)

    face_px = box.shortest_side
    if face_px < policy.min_face_px:            issues.append("face_too_small")   # < 80 px

    blur = cv2.Laplacian(gray, cv2.CV_64F).var()
    if blur < policy.blur_threshold:            issues.append("blurry")           # < 60.0

    brightness = gray.mean()
    if brightness < policy.brightness_min:      issues.append("too_dark")         # < 50
    elif brightness > policy.brightness_max:    issues.append("too_bright")       # > 210

    return QualityReport(not issues, tuple(issues), blur, brightness, face_px)
```

Two of these deserve explanation, because they are the only place in the project where a computer-vision idea is doing real work:

> **VARIANCE OF THE LAPLACIAN — how "blurry" is measured**
>
> The Laplacian is a convolution filter that highlights edges: it responds strongly where brightness changes sharply. A sharp face has many crisp edges — eyes, nose, mouth — so the *variance* of that response is high. Blur smooths edges away, so the variance collapses toward zero. One line of OpenCV gives a usable sharpness score with no model at all. Below 60, this project declines to embed the face.

> **FACE SIZE — why 80 pixels is a floor, not a preference**
>
> SFace expects a 112×112 aligned crop. Upscaling a 40-pixel face to 112 does not add information; it invents it smoothly. The embedding will be dominated by interpolation artefacts and the comparison becomes noise. Rejecting the sample is more honest than scoring it.

If quality fails, the outcome is `poor_quality` and no embedding is computed — visible to the operator as a reason ("too dark"), not as a silent miss.

12.4 Embeddings and cosine similarity

The step that makes recognition possible at all:

1. YuNet returns a bounding box plus **five landmarks** — two eyes, nose tip, two mouth corners.
2. Those landmarks define an affine transform that rotates, scales and translates the face so the eyes land in fixed positions. This is **alignment**, and it matters enormously: a face tilted 20° embeds very differently from the same face upright.
3. SFace runs on the aligned 112×112 crop and emits 128 floating-point numbers.
4. The vector is **L2-normalised** — divided by its own length so it lies on the unit sphere. Now "how similar" is purely about direction.

```
// ai_service/analyzer.py
aligned   = self._recognizer.alignCrop(image, row)
embedding = normalize(self._recognizer.feature(aligned))
```

> **COSINE SIMILARITY, INTUITIVELY**
>
> Imagine each face as an arrow pointing somewhere in a 128-dimensional space. Two photos of the same person point in nearly the same direction; two different people point in noticeably different directions. Cosine similarity measures the angle between the arrows: 1.0 is identical direction, 0 is unrelated, −1 is opposite. Because the vectors are normalised, cosine similarity reduces to a simple dot product — which is why `np.dot` is the entire matcher.

Matching against the gallery is `ai_service/matcher.py`, and it is worth reading in full because it is only 14 lines of logic:

```
def match_embedding(embedding, gallery) → Match:
    """gallery maps student_id → list of template vectors"""
    probe = normalize(embedding)
    per_student = {}
    for student_id, templates in gallery.items():
        for template in templates:
            score = float(np.dot(probe, normalize(template)))
            if score > per_student.get(student_id, -1.0):
                per_student[student_id] = score
    ranked = sorted(per_student.items(), key=lambda x: x[1], reverse=True)
    return Match(ranked[0][0], ranked[0][1], ranked[1][1] if len(ranked) > 1 else None)
```

- **Max over templates.** A student has 3–5 enrolled samples; their score is the best of those. Faces legitimately vary with lighting, angle and expression.
- **The runner-up is returned too.** Not an optimisation — a safety feature. The gap between first and second is what reveals ambiguity.

12.5 The decision rule

This is where the AI package stops and the back end takes over. The rule, stated exactly:

```
recognized     ⟺  score ≥ threshold  AND  (best − second_best) ≥ ambiguity_margin
low_confidence ⟺  threshold − low_confidence_margin ≤ score < threshold
                   OR the margin to the runner-up is too small
unknown        ⟺  everything else
```

With the shipped defaults (`threshold = 0.42` , `low_confidence_margin = 0.07` , `ambiguity_margin = 0.05`):

Score	Runner-up	Outcome	What happens
0.71	0.34	`recognized`	Attendance recorded, source `automated` .

Score	Runner-up	Outcome	What happens
0.45	0.43	`low_confidence`	Nothing recorded. Event queued for review with both candidates attached.
0.39	0.12	`low_confidence`	Inside the review band below the threshold — a human decides, rather than the system guessing "no".
0.21	0.18	`unknown`	Not on this roster. Recorded as an event, no attendance.

WHY THE AMBIGUITY MARGIN EXISTS

A high absolute score does not mean the system identified the right *person* — it may mean it matched a face shape that several enrolled people share (siblings, twins, or simply similar faces in a small class). The margin asks the second question that matters: "is this clearly better than the alternative?" Two criteria, one about identity and one about discrimination, are strictly safer than either alone.

These scores are not probabilities

A cosine similarity of 0.62 does not mean "62% likely to be correct". It is a normalised dot product in an embedding space, and no calibration to probability is claimed anywhere in the project. Treating it as a probability is the most common misunderstanding of face recognition, and the documentation names it explicitly.

12.6 The session gallery

Comparing a probe against every enrolled template in the institution would be slow and wrong — the student may not even be in this class. So each session works against a **gallery**: the decrypted templates of the students on that session's roster, cached in RAM.

- Templates are decrypted once per session, held only in process memory, and never written to disk in plaintext.
- The comparison set is the roster, so a student from another course cannot match into this session even if their face is a perfect match.
- That is also an eligibility rule: a confident match with a student who is *not* on the roster produces `not_eligible`, not an attendance record.

12.7 Liveness Experimental

Liveness answers a different question from recognition. Recognition asks "is this face the same person as that template?". Liveness asks "is this a real, present human, or a photograph of one?". A system can be excellent at the first and completely fooled by the second — a printed photo held up to the camera is a high-quality face.

The implementation is MiniFASNetV2, a small convolutional network:

```
1. Take the detected face box and expand it by a factor of 2.7 (clamped to the frame).
2. Resize to 80×80, convert to CHW float32, scale by 1/255, then (x - 0.5) / 0.5 → range [-1, 1].
3. Run ONNX Runtime on CPU. The output is a 3-class softmax.
4. Class 1 is "real". Its probability IS the liveness score.
5. score ≥ liveness_threshold (0.80) → passed.   score < 0.5 → failed.   between → inconclusive.
```

The expansion factor matters: a tight crop of just the face carries few artefacts, while the region *around* the face is where the evidence of a screen bezel, a printed border or a moiré pattern lives.

The three policies

Policy	Behaviour	Use when
`off` *(default)*	No liveness check. `liveness_status` is recorded as `not_required`.	Until the model has been evaluated on the institution's own bona fide and attack samples.
`flag`	Confident identities are marked; failures and inconclusive results are queued for human review.	A reasonable pilot. It surfaces how the model behaves without changing outcomes.
`enforce`	Only a passing check produces a mark. An unavailable model produces no mark and an `error` outcome.	Only after measurement. This is an institutional decision, not a checkbox.

It never fakes a pass

If the ONNX file is absent, the API returns `not_enabled` — and under `enforce`, that means no attendance is recorded. During the model conversion smoke check, a synthetic flat-colour frame scored **0.008** and was reported as `failed`. A placeholder implementation would have returned a plausible-looking number; this one returned the correct answer for a picture that is obviously not a face. That is the evidence, in the repository, that real inference is happening.

What this model cannot do

Stated in `docs/ai-models.md` rather than hidden in a footnote:

- It is **single-frame**. It sees one still image. Blink, head-pose and texture-flow cues available across a video sequence are unavailable to it.
- It is **RGB-only**. There is no depth camera and no infrared channel, so a printed photo at the right distance is genuinely harder to distinguish than with a 3-D sensor.
- It is **known to be weaker against high-quality replay attacks** — a phone screen showing a video — and against capture devices unlike the one it was trained on.
- It does not and cannot defeat every presentation attack. No liveness system does, and any product claiming otherwise is selling something.

12.8 The evaluation harness

Because no accuracy claim is made, the machinery to make one has to exist. It does, in `ai_service/evaluation/`:

Script	What it does
`recognize_dataset.py`	Runs the recognition pipeline over an approved local dataset and writes per-pair scores.
`select_threshold.py`	Sweeps candidate thresholds and reports the resulting false-accept and false-reject rates, so the operating point is chosen from a curve rather than picked from a blog post.
`liveness_dataset.py`	The equivalent for presentation-attack detection: bona fide versus attack samples, APCER and BPCER.
`metrics.py`	Shared metric computation.

THE TWO ERRORS, IN PLAIN TERMS

False accept — the system marks someone present who is not the person it thinks it saw. This is the dangerous one: it corrupts a record and, at scale, is how attendance fraud works.

False reject — the system fails to recognise a legitimate student. Annoying, visible, and self-correcting because the student is standing there and can be marked manually.

The threshold trades one against the other. Raising it to 0.55 reduces false accepts and increases false rejects. There is no free lunch, which is why the default is the conservative 0.42 and why a review queue exists at all.

Current status: nothing has been measured

`docs/evaluation.md` states plainly that no accuracy metrics have been produced for this project yet, because that requires a consented dataset of real participants. The harness is ready; the data is not. `README.md` says the same thing on the front page. This is the most important honesty property of the whole project, and the one most likely to be asked about in a viva.

12.9 Performance on the reference hardware

The reference machine is an Intel i3-1005G1 with 8 GB of RAM and no GPU — an ordinary student laptop, chosen deliberately so the project does not quietly require hardware nobody has.

Step	Design estimate (CPU)
YuNet detection at 640×480	~10–25 ms
SFace embedding per face	~15–30 ms
MiniFASNet liveness per face	~10–20 ms
Frame sampling default	1 frame per 1000 ms

"Design estimate" is doing real work in that phrase

> These are figures expected from models of this size on a CPU of this class, not measurements taken on this project's machine. The document says so in the sentence immediately after the table. A measured latency would go in the evaluation report, alongside the accuracy figures — from the same instrumented run.

Two engineering consequences follow from the CPU-only constraint. First, `FaceAnalyzer` loads each model exactly once and wraps inference in a `threading.Lock`, because OpenCV's detection objects are not safe to call concurrently — serialising is cheaper than debugging the alternative. Second, one uvicorn worker is the default, since the rate limiter and the recognition cooldown both live in process memory.

CHAPTER 13

Security and privacy

The seven principles the design is built on, the threat model, and what the system explicitly does not claim to defend against.

13.1 Why this chapter exists at all

Most projects treat security as a checklist performed at the end. This one could not, because it stores something that cannot be replaced if it leaks. A stolen password can be changed; a stolen face cannot. And a system that marks students absent incorrectly has a real effect on their eligibility for exams.

So security and privacy are design inputs, not a coat of paint. Seven principles are stated at the top of `docs/privacy-security.md`, and each one has a mechanism behind it:

Principle	What it means	The mechanism that enforces it
1. Consent before biometrics	No face template may exist without a recorded, versioned agreement.	`biometric_templates.consent_id` is a `NOT NULL` foreign key to `biometric_consents`. The database rejects the row. This is not a policy the code follows; it is a fact the schema makes impossible.
2. Data minimisation	Store the least that can do the job.	128 encrypted floats per enrolment, and metadata about frames — never the frames themselves. No image column exists anywhere in the schema.
3. Purpose limitation	Recognition marks attendance for a scheduled session. Nothing else.	Recognition requires a session id, the session must be `active`, and the caller must own the course. There is no endpoint that searches all students by face.
4. Human control	Uncertainty never becomes a record without a person deciding.	The review queue, the `low_confidence` outcome, and `pending_review` counting as not-attended until resolved.
5. Least privilege	Each role sees the minimum it needs.	Role dependencies plus ownership checks; scope-pushed queries; 404 instead of 403 so existence is not leaked.
6. Defence in depth	No single control is trusted to hold.	RBAC + CSRF + session expiry + rate limits + encryption + security headers + audit. Each layer assumes the one before it might fail.
7. Honesty	Say what is not true as clearly as what is.	Liveness labelled experimental; a missing model yields `not_enabled`, never a pass; no accuracy figure published until measured.

13.2 What is stored, and for how long

Data	Where	Retention
Password	`users.password_hash`	Argon2id hash only. Plaintext is never stored, never logged, never returned.
Session token	Browser cookie + `auth_sessions.token_hash`	480 minutes absolute, 60 minutes idle. Revoked on logout, password change, reset or deactivation. The raw token is never stored server-side.
Reset token	`password_reset_tokens.token_hash`	24 hours, single use. Issuing a new link cancels earlier ones.
Consent	`biometric_consents`	Kept for accountability, even after withdrawal — the record that someone consented, and later withdrew, is itself evidence.
Face templates	`biometric_templates.ciphertext`	Until deleted, or purged 30 days after the student is deactivated.
Attendance	The attendance tables	Institutional academic-record policy. Never cascaded away.
Recognition metadata	`recognition_events`	Scores, outcomes, quality numbers, model provenance. No pixels.
Raw camera frames	**Request memory only**	**Discarded after the response. Never on disk, in the database, in a log or in a cache.**

13.3 The threat model

A **threat model** is a written list of who might attack the system, how, and what would stop them. Its value is that it forces you to name what you have *not* defended, which is the part most security writing skips. This is the project's, condensed:

Threat	Mitigation	Residual risk — stated honestly
Spoofed photo or screen at the camera	Optional MiniFASNet in `flag` or `enforce`; staff supervision; an absent pass never marks anyone.	**High.** Single-frame RGB PAD is experimental; a high-quality replay may pass.
Wrong identity matched	Conservative 0.42 threshold, ambiguity margin, low-confidence review, never auto-mark on uncertainty.	False accepts and rejects exist. Accuracy is unmeasured.
A classmate marking on someone's behalf	Liveness signal, staff presence, audit trail, per-student self-marking lock.	Not eliminated. There is no exam-grade identity proofing.
Password guessing	Argon2id, 5-failure lockout for 15 minutes, 10 logins/min per IP, generic error messages.	Distributed attacks; no multi-factor authentication (out of scope).
CSRF	Double-submit token on every state-changing route, constant-time comparison.	A successful XSS would defeat it; React escaping and a strict CSP reduce XSS.

Threat	Mitigation	Residual risk — stated honestly
Session theft	`HttpOnly` / `Secure` cookies, server-side revocation, idle timeout, only the hash stored.	Malware on the client machine reads the cookie jar.
Database theft	Encrypted templates bound by AEAD to their student; hashed tokens and passwords.	Attendance and user metadata are *not* encrypted at rest.
Loss of the encryption keys	Documented, separate backup requirement.	Every template becomes unreadable and all students must re-enrol.
Insider misuse	Role scoping, append-only audit, no gallery endpoint, reviewer-only candidate fields.	An administrator with both database and key access is highly privileged. This is true of every system.
Log leakage	The audit helper strips sensitive keys; no frames or embeddings in logs.	Free-text reason fields are written by humans and need care.
Spreadsheet formula injection	CSV cells beginning with `=`, `+`, `-`, `@`, tab or CR are prefixed with an apostrophe.	A user who overrides that protection in Excel still takes the risk.
Model supply-chain tampering	SHA-256 verification in `fetch_models.py`; licences and provenance pinned in the docs.	Compromise of the download host before verification.

> **Read the third column**
>
> Most project reports list mitigations and stop. A threat model whose residual-risk column is empty is not a threat model, it is marketing. The first row of this table says the anti-spoofing measure may not work — and that sentence is in the repository, on the front page of the documentation index, before anyone asks.

13.4 Encryption of biometric templates, in full

The whole mechanism is 63 lines in `backend/app/core/crypto.py`. It is worth walking through because it illustrates four distinct ideas at once.

```
class TemplateCipher:
    def __init__(self, spec: str):
        # spec = "k1:base64…,k2:base64…"  – comma-separated key_id:secret pairs
        for part in spec.split(","):
            key_id, _, b64 = part.partition(":")
            raw = base64.b64decode(b64, validate=True)
            if len(raw) ≠ 32 or not key_id:
                raise BiometricKeyError(f"key {key_id!r} must be 32 bytes")
            self._keys[key_id] = AESGCM(raw)
            self._active = self._active or key_id        # the first key encrypts new data

    def encrypt(self, student_id, model_name, model_version, embedding):
        nonce = os.urandom(12)                           # fresh random per template
        data  = np.asarray(embedding, dtype=np.float32).tobytes()
        ct    = self._keys[self._active].encrypt(nonce, data, _aad(student_id, model_name,
model_version))
        return EncryptedTemplate(self._active, nonce, ct)

def _aad(student_id, model_name, model_version):
    return f"smartattend:template:{student_id}:{model_name}:{model_version}".encode()
```

Idea	What it buys
AES-256-GCM	Symmetric encryption with authentication. GCM detects tampering: flip a bit in the ciphertext and decryption raises an error rather than returning a subtly wrong embedding.
Fresh 96-bit nonce	Two students who enrol the identical sample get different ciphertexts. Reusing a nonce with the same key in GCM is a serious cryptographic failure, which is why it is generated at encryption time and stored per row.
AEAD associated data	The ciphertext is bound to its student and model. An attacker — or a buggy migration — that copies a row onto another student's record produces a decryption failure, not a successful impersonation.
Multiple keyed by id	Rotation. The first key in the list encrypts new data; every listed key can decrypt. Old rows keep working while new ones use the fresh key.

> **Why not hash the embeddings instead?**
>
> Hashing is one-way, and the system genuinely needs the numbers back in order to compare them. Hashing is for secrets you only ever verify (passwords); encryption is for data you must use (templates). Confusing the two is one of the most common security mistakes in student projects.

13.5 Security headers, and what each one stops

Set on every response by the middleware in `backend/app/main.py`:

Header	Value	Purpose
`X-Content-Type-Options`	`nosniff`	Stops the browser guessing that an uploaded file is executable script — the classic path from "user uploads an image" to "user runs code".

Header	Value	Purpose
`X-Frame-Options`	`DENY`	Stops another site embedding this one in an `<iframe>`, which is how click-jacking works: a transparent overlay makes a student click "delete my biometrics" while they think they clicked something else.
`Referrer-Policy`	`no-referrer`	When a user follows a link out, the destination does not learn the exact internal URL they came from.
`Permissions-Policy`	`camera=(self), microphone=(), geolocation=()`	Camera access is limited to this origin and cannot be delegated to embedded third-party frames. The microphone and location are switched off entirely — the application has no use for either.
`Cross-Origin-Opener-Policy`	`same-origin`	Isolates this page's browsing context from unrelated ones.
`Cache-Control`	`no-store` on `/api/*`	Attendance data is never written to a shared or on-disk cache.

13.6 The access-control matrix

Who may do what, in one table. **A** admin, **F** faculty on their own course, **S** student on their own data.

Action	A	F	S	Anon
View own profile, change own password	✓	✓	✓	—
Manage users, settings, read the audit log	✓	—	—	—
Manage students, faculty, courses	✓	read own	—	—
Record consent, enrol, delete templates	✓	—	✓ own only	—
Create, start, close sessions	✓	✓	—	—
Run recognition	✓	✓	—	—
Self-mark during an open window	—	—	✓	—
Review events, manual marks, approve corrections	✓	✓	—	—
Request a correction	—	—	✓	—
View attendance history	all	own courses	own	—
Reports and exports	all	own courses	—	—
`GET /health`	✓	✓	✓	✓ (no secrets)

13.7 What is *not* protected

The document's own list of known limitations, reproduced because it is the most useful part of any security write-up:

- No multi-factor authentication, no single sign-on, no multi-tenant isolation.
- No field-level encryption of attendance metadata — only biometric templates are encrypted at rest.
- Audit records are append-only *through the application*. A database administrator with direct access can modify rows; there is no cryptographic tamper evidence.
- Rate limiting and recognition cooldown assume one API worker.
- Liveness is experimental and uncertified; it must never be the sole basis for a high-stakes decision.
- Demographic bias in the recognition model is unmeasured. This is a real, known property of face recognition generally, and a human review path is therefore mandatory rather than optional.
- No automated data-subject-access workflow, no breach-notification tooling, no consent-expiry re-prompting.

> **Why listing your weaknesses is a strength**
>
> An examiner's job is to find the boundary of what you understand. A project that volunteers "our anti-spoofing is single-frame RGB and may not survive a good replay attack, and here is the evaluation that would settle it" demonstrates more understanding than one that claims the feature works. Writing the limitation down is the evidence that you know where the system stops.

CHAPTER 14

End-to-end journeys, step by step

Six complete scenarios traced from a click to a committed row, with every check named at the moment it fires.

14.1 Setting up an institution (one time)

1. **Deploy.** `docker compose up -d --build`. The API container runs `alembic upgrade head` before starting, creating all 17 tables, seeding the three roles and the default settings.
2. **Create the first admin.** `docker compose exec api python -m app.cli create-admin --email … --name "…"`. The command prompts twice for a password, validates its strength, stores only an Argon2id hash and writes an audit entry. There is no default account and no password in the repository — creating one requires shell access to the container, which is the authorisation.
3. **Add people.** Admin → Faculty → create. Admin → Students → create, with registration number, department and programme. Each can be given a portal account from the same screen, either by setting a password directly or by issuing a one-time reset link shown once.
4. **Add a course.** Code, name, department, term, and the faculty member who teaches it. A course without an assigned teacher cannot have sessions scheduled — the API answers `400 course_has_no_faculty`.
5. **Enrol students.** Bulk-select on the course page; the endpoint is idempotent, so enrolling the same student twice is a no-op rather than an error.

14.2 Enrolling a face, with consent

This is the flow where privacy actually happens, so each step is traced.



Step	What happens	Checks that fire
1	The student opens My account → Face registration. The notice is displayed: purpose, alternatives, retention, who can access, the right to withdraw.	The current `policy_version` is read.

Step	What happens	Checks that fire
2	The student confirms. A row is written to `biometric_consents` with `capture_method: self_service`.	`participant_confirmed` must be `true`; a stale `policy_version` is rejected so nobody agrees to old wording.
3	The camera opens on a click. Each preview frame goes to `/recognition/detect`, which returns the box and the quality report — nothing is stored.	`no_face`, `multiple_faces`, `blurry`, `too_dark`, `too_bright`, `face_too_small`. The student gets a reason, not a silent failure.
4	3–5 accepted samples are posted as multipart.	Every sample is **re-validated server-side**. Client-side acceptance is a convenience, never a credential.
5	Embeddings are computed, checked for internal consistency, and compared against other students.	`inconsistent_samples` if the five samples do not cluster (a strong signal that someone enrolled the wrong face, or several faces). `possible_duplicate_identity` if a very close match already exists under a different registration number.
6	Templates are encrypted with AES-256-GCM and stored. The images are gone.	`consent_required` blocks the whole flow if step 2 did not happen — and the foreign key blocks it again at the database.

The duplicate-identity check is the interesting one

It exists because the failure mode is not "the system is inaccurate" but "two records claim one face". If a student enrols twice under different registration numbers, attendance becomes unresolvable. Catching that at enrolment, when it can be fixed by a human looking at two files, is far cheaper than discovering it in a dispute at the end of term.

14.3 A live session, from the staff side

1. **Schedule.** Courses → the course → Students → Schedule. Pick a title, a start and an end, and optionally `window_min` and `late_after_min`. The server checks the window ordering and rejects overlaps with a 409.
2. **Open Live attendance.** `/live?session=12`. The page loads the session, the roster and the client settings (frame interval, liveness policy, timezone) in parallel.
3. **Start.** `POST /sessions/12/start` → the state machine locks the row, verifies the transition, sets `started_at`, writes the audit entry. The status chip turns green.
4. **Enable the camera.** A click, then the browser permission prompt. The `<video>` element plays the stream locally — no frames are sent yet.
5. **Scan.** Pressing Scan starts the interval loop (`LivePage.tsx:183`). Each tick captures a JPEG and posts it.
6. **Per frame.** The server runs the pipeline from §12.1 and returns one `FaceResult` per face: box, outcome, similarity, liveness, and whether a record was committed.

7. **Feedback.** Boxes are drawn over the preview, coloured by outcome. A rolling feed lists recent results. The roster updates live with green ticks.
8. **Manual fill.** Anyone present but not recognised is marked manually with a reason. `source: manual`, and the reason is mandatory.
9. **Close.** The loop stops. Closing is a transition, not a client-side flag.
10. **Review.** Anything that landed in the exception queue is resolved now or later — `mark_present`, `mark_late` or `dismiss`, with a 5–500 character note.
11. **Finalize.** Everyone on the roster with no record gets `absent` with `source: system`. After this the session is frozen; changes require an audited correction.

> **What happens if two frames catch the same student**
>
> The first insert wins. The second hits the unique index, `ON CONFLICT DO NOTHING` returns no row, and the event is stored with outcome `duplicate` and `committed: false`. The cooldown setting (30 s) suppresses most of these before they reach the database, but the constraint is the guarantee — the cooldown is only politeness.

14.4 A student's own session

1. The student signs in and sees **Mark attendance** in the sidebar (the only navigation item unique to that role).
2. The page calls `GET /sessions/mine`. Each session comes back with the student's own record status and three self-marking fields: `self_marking_open`, `self_marking_message` and `self_marking_closes_at`.
3. Sessions are grouped into **Open now**, **Upcoming** and **Earlier sessions**. The grouping is computed from the server's flags plus the clock — a closed session never appears as upcoming even if its scheduled end is still in the future.
4. Inside the window, one button: **Mark me present**. The countdown to `attendance_closes_at` is shown so the student can see the deadline.
5. `POST /sessions/{id}/self-mark`. The server re-checks everything: is self-marking enabled for this session, is the clock inside the window, is this student locked, does a record already exist.
6. The answer is a record with `source: self_reported` and status `present` or `late` depending on `late_after`. The UI shows what the server decided, not what the click hoped for.

And the control that answers "but they could mark and leave": on the session detail page, each participant row has a lock toggle. Locking one student sets `self_marking_locked_at` with a reason, and their `self_marking_open` becomes `false` with a message. Nobody else in the cohort is affected. Unlocking restores them. Both actions are audited.

14.5 A correction request

Step	Actor	Detail
Submit	Student	`POST /corrections` with the session, the status requested (`present`, `late` or `excused`) and a reason of 5–500 characters. If there is no record at all, the request type becomes `manual_attendance` rather than `correction`.
Guard	Server	One pending request per (student, session), enforced by a partial unique index. A second submission gets 409 `pending_exists`.
Review	Faculty or admin	The queue shows the request, the reason, the current record and the session context. Only their own courses.
Decide	Faculty or admin	Approve applies the record change *and* writes the audit entry in one transaction. Reject leaves the record untouched and stores the reviewer's note.
Withdraw	Student	A pending request can be withdrawn by its author.
See	Student	The outcome and the note are visible in their own list.

14.6 A report and an export

1. The Reports page sends filters as query parameters: report type, date range, course, granularity.
2. `reports_service` builds the aggregate *within the caller's scope* — an admin sees everything, a faculty member sees only their own courses. The scope is a SQL condition, not a post-filter.
3. The response carries server-defined `columns` with `key`, `label` and `align`, plus `rows`. The browser renders exactly what the CSV and PDF exporters will render, so the on-screen table and the downloaded file can never disagree.
4. Grouped reports include a `summary` with per-status totals and `rate = (present + late) / marked`.
5. Export reuses the same filters and scope, caps at 10,000 rows (409 `export_too_large` beyond that), and returns a file attachment with `Content-Disposition`. CSV cells that begin with a formula character are prefixed with an apostrophe. PDFs are landscape A4 tables generated with reportlab.

CHAPTER 15

Testing

154 automated tests, what they check, how they are written, and why the test database is a hard requirement rather than a convention.

15.1 What tests are for

> **AUTOMATED TEST**
> A piece of code that exercises other code and fails loudly if the behaviour is wrong. It runs in seconds, needs no human, and — the important part — runs the same way every time. Its real value is not proving the code works today; it is proving that a change you make tomorrow did not break something that used to work.

Suite	Count	Runner	What it covers
Backend	97	pytest + httpx	Every endpoint and every business rule, against a real PostgreSQL database.
Frontend	57	vitest + Testing Library	Component behaviour and page logic in a simulated browser (jsdom).

15.2 Backend tests

Nine modules, each owning one area of the product:

Module	What it proves
`test_auth_and_users.py`	Login success and failure, lockout after five failures, session revocation on logout and password change, CSRF enforcement, role gates, admin-set passwords.
`test_academics.py`	Course creation, bulk enrolment idempotency, dropping a student, roster synchronisation into open sessions while closed sessions stay frozen.
`test_sessions.py`	The whole state machine: every legal transition, every illegal one, window computation, overlap rejection, reopen requiring a reason.
`test_self_marking.py`	The window boundaries, the per-student lock, the never-overwrite rule, and that `source` is `self_reported`.
`test_biometric.py`	Consent gating, sample validation, encryption round-trip, the AEAD binding, duplicate-identity detection, deletion.
`test_review_and_corrections.py`	The exception queue, resolution paths, the one-pending-request rule, approval applying record and audit atomically.
`test_reports.py`	Every report type, scope restriction, the CSV formula-injection guard, the export row cap.

Module	What it proves
`test_hardening.py`	Security properties: rate limits, upload size and type, headers, information disclosure through 404 vs 403.
`test_settings.py`	Validated ranges, the client-facing subset, audit on change.
`test_evaluation_metrics.py`	The metric maths in the evaluation harness — that APCER and BPCER are computed correctly, independent of whether any data has been measured yet.

What a test looks like

```
// backend/tests/conftest.py provides fixtures: a client, a database, and logged-in users
def test_self_mark_is_recorded_late_after_the_cutoff(client, session_with_window, as_student):
    # push the clock past late_after but stay inside the window
    resp = client.post(f"/api/sessions/{session_with_window.id}/self-mark",
                       headers={"X-CSRF-Token": as_student.csrf})
    assert resp.status_code == 200
    body = resp.json()
    assert body["status"] == "late"                  ① the rule, not just the code
    assert body["source"] == "self_reported"         ② provenance preserved

    again = client.post(f"/api/sessions/{session_with_window.id}/self-mark", …)
    assert again.json()["record_id"] == body["id"]   ③ the second call returns the first record
```

Three assertions, three separate guarantees. A test that only checked `status_code == 200` would pass on a system that marked everyone present.

15.3 The test database rule

The most valuable safety property in the whole test setup is not a test at all — it is a start-up check:

```
if self.app_env == "test":
    if not self.test_database_url:
        raise ValueError("TEST_DATABASE_URL must be set when APP_ENV=test")
    if self.test_database_url == self.database_url:
        raise ValueError("TEST_DATABASE_URL must differ from DATABASE_URL")
```

Running the suite truncates tables. Without this guard, a mistyped environment variable would let a test run wipe real attendance data. With it, the process exits before connecting. The protection is structural: there is no configuration under which the tests can touch the development database.

> **Real database, not a mock**
>
> The tests run against actual PostgreSQL. That matters for this project specifically, because the guarantees that protect attendance live in the database: the unique index, the `CHECK` constraints, `ON CONFLICT … RETURNING`, `SELECT … FOR UPDATE`. A mocked database would let every one of those rules be broken while all the tests stayed green.

15.4 Frontend tests

Fifteen test files, written with React Testing Library — which means they interact with the rendered DOM the way a user would, rather than calling internal functions:

```
// frontend/src/pages/MarkAttendancePage.test.tsx
it("keeps a closed session out of the upcoming list even before its end time", async () => {
  const futureEnd = new Date(Date.now() + 60 * 60 * 1000).toISOString();
  stubFetch([ session({
      id: 9, title: "Closed but not finished", status: "closed",
      self_marking_open: false,
      self_marking_message: "This session is closed for attendance",
      scheduled_end: futureEnd,
  }) ], {});

  renderPage();
  expect(await screen.findByText(/closed but not finished/i)).toBeInTheDocument();
  expect(screen.queryByText(/upcoming/i)).not.toBeInTheDocument();   ← the actual bug
  expect(screen.getByText(/earlier sessions/i)).toBeInTheDocument();
});
```

This test exists because of a real defect: a session that had been closed early but whose scheduled end was still in the future appeared under "Upcoming", so a student could see a session they could never join. The fix was a status check in the grouping logic; the test is what stops the regression.

File	Tests	Focus
`lib/api.test.ts`	9	The transport layer: CSRF header on unsafe methods, never on GET, error envelope parsing, network failure mapping, download filename extraction.
`MarkAttendancePage.test.tsx`	5	Grouping, the countdown, the locked state, the closed-session regression.
`LoginPage`, `LivePage`, `AttendancePage`	4 each	Form validation, the frame loop's guards, filter behaviour.
`SessionDetailPage`, `ReportsPage`, `ProfilePage`, `AdminUsersPage`, `ResetPasswordPage`	3 each	The main CRUD screens and their permission states.
the remaining 8 files	1–2 each	Dashboard, courses, corrections, review, privacy, and the two interactive components.

15.5 Running everything

```
# backend — needs the test database and the venv
cd backend
.venv/Scripts/python -m pytest

# frontend
cd frontend
npm test          # vitest run — 57 tests
npm run lint      # oxlint
npm run build     # tsc -b then vite build — type errors fail the build
```

A real-world flake, and what was learned from it

On the reference laptop the frontend suite intermittently failed one test on a cold run. The cause was not the code: it was a 5-second default timeout on a CPU-bound machine where jsdom setup alone could take most of it. The fix was raising `testTimeout` to 30 seconds in `vite.config.ts`, with a comment saying why. That is the correct resolution for a resource timeout and the *wrong* resolution for a genuine race — telling the two apart is most of what debugging tests involves.

15.6 What testing does not do

Tests verify that code behaves as written. They cannot verify that it behaves as *intended* when the intent was wrong, and they cannot measure model accuracy at all — that needs a consented dataset and the harness in `ai_service/evaluation/`. Two of the most important claims in this project are outside what any test can prove:

- That the recognition threshold of 0.42 is a good operating point. Only measurement on real data can say that; the tests only prove the code *uses* the configured threshold correctly.
- That the liveness model resists attacks. The tests prove the pipeline handles every verdict correctly, including `not_enabled`. Whether the verdicts are right is an empirical question that has not been answered.

CHAPTER 16

Running it: development and deployment

From a fresh laptop to a working URL — the two ways this system runs, every command in order, and what each one is actually doing.

16.1 Two ways to run the same code

Everything in this project can run in two shapes. They use the *same* backend code, the *same* frontend code and the *same* database schema. What differs is who serves the files, where the processes live, and how the browser reaches them.

DEVELOPMENT MODE VS PRODUCTION MODE

Development mode means: two servers running side by side on your own machine, one for the frontend with hot reload, one for the API with auto-restart on save. Speed of editing is the priority; the arrangement is nobody's idea of tidy.

Production mode means: the frontend is compiled once into static files, a single web server serves those files *and* forwards `/api` to the backend, cookies are marked secure, and none of the tooling that helps you edit is present. Correctness and a single origin are the priority.

Question	Development	Production
Who serves the frontend?	Vite dev server, on the fly, from source files	nginx or Caddy, from compiled `dist/` files
Which port do you open?	`http://localhost:5173`	`http://localhost:8080`, or your public HTTPS URL
How does `/api` reach the backend?	Vite's built-in proxy rewrites it to `127.0.0.1:8000`	The web server reverse-proxies it to the API container or process
Do you see your edits immediately?	Yes — hot module replacement, plus `--reload` on the API	No — you rebuild and redeploy
Are cookies HTTPS-only?	No, `COOKIE_SECURE=false`	Yes, `COOKIE_SECURE=true`
Can it be exposed to the internet?	Not safely — no TLS, debug tooling, permissive defaults	That is what it is for

REVERSE PROXY

A web server that sits in front of another server and forwards requests to it. The browser talks only to the proxy; the backend is invisible. This project uses a reverse proxy for three separate jobs at once: **(1)** serve the static frontend, **(2)** forward `/api/*` to uvicorn, **(3)** add security headers the backend does not set. Because both the page and the API arrive from the same origin, the browser sends the session cookie with API calls automatically — no CORS negotiation at all.

16.2 Prerequisites, and what each one is

Software	Version used	What it is and why this project needs it
Python	3.10+	The language the backend, the AI package and the operational commands are written in. This project was developed against 3.10 — the minimum matters because the code uses modern type-hint syntax (`int | None`) that older Pythons reject.
Node.js	20+	A JavaScript runtime that lets JavaScript run outside a browser. The frontend is written in TypeScript and must be compiled with Vite, which is a Node program. Node also runs the test runner and the linter.
PostgreSQL	16	The database server. In this workspace it is a *portable* copy — plain binaries unzipped into `.tools/pgsql/` with no installer and no Windows service. That choice means the database lives entirely inside the project folder and can be copied or deleted without touching the machine.
Git	any recent	Version control. Not required to run the app, but required to clone it and to see its history.
Docker	24+ with Compose v2	Only for the containerised deployment path in 16.9. The local path in 16.3–16.6 needs no Docker at all.

Why a portable PostgreSQL?

A normal PostgreSQL install registers a Windows service that starts at boot and owns a shared data directory. A portable copy instead runs only when you start it, with its data in `.tools/pgdata`. For a college project this is the honest arrangement: nothing is left running in the background, nothing is left behind after the demo, and the whole state of the system is two folders you can copy onto a USB stick.

16.3 Step one — the `.env` file

Configuration is everything that changes between one machine and another: passwords, ports, the public URL, how long a session lasts. Putting it in the source code would mean editing code to deploy, and committing secrets to Git. So it lives in a single plain-text file named `.env` at the repository root, which the backend reads at start-up and which is **never committed**.

```
cp .env.example .env        # start from the template; then replace every CHANGE_ME

# generate a database password (URL-safe, so it can be pasted into a URL)
python -c "import secrets;print(secrets.token_urlsafe(32))"

# generate a biometric encryption key: a key id, a colon, then 32 random bytes in base64
python -c "import secrets,base64;print('k1:' +
base64.b64encode(secrets.token_bytes(32)).decode())"
```

ENVIRONMENT VARIABLE

A named value supplied to a program by the operating system rather than by its source code. `DATABASE_URL` is not written inside any Python file; the code asks the environment for it and crashes at start-up if it is missing. This

is what makes the same image runnable on your laptop and on a server without a single code change. A `.env` file is simply a convenient way to load a batch of them before the program starts.

Variable	Example	What breaks if it is wrong
`APP_ENV`	`development`	Set to `production` and the development seeding commands refuse to run — by design. Set to `test` and the app refuses to start unless a separate test database is configured.
`APP_TIMEZONE`	`Asia/Kolkata`	"Today" and every session date are computed in this zone. Wrong zone means a 10:00 lecture is stored as 04:30 and the attendance window opens at the wrong moment.
`DATABASE_URL`	`postgresql+psycopg://…`	The app cannot connect. The `+psycopg` part tells SQLAlchemy which driver to use — omit it and it looks for a driver that is not installed.
`TEST_DATABASE_URL`	a *different* database	The test suite is blocked from running. It must differ from `DATABASE_URL` so that a test run can never delete real attendance.
`FRONTEND_BASE_URL`	`http://localhost:5173`	Password-reset links are built from this. If it points at a host that no longer exists, every reset link is dead — the single most common deployment mistake in this project.
`CORS_ORIGINS`	`http://localhost:5173`	In development the API must accept calls from port 5173. In production it is left empty, because page and API share one origin.
`SESSION_TTL_MINUTES` / `SESSION_IDLE_MINUTES`	`480` / `60`	Absolute lifetime and idle timeout. Too short and staff are logged out mid-lecture; too long and a shared machine stays signed in.
`COOKIE_SECURE`	`false` locally	`true` over plain HTTP means the browser silently refuses to send the cookie and login appears to "not stick". `false` over HTTPS works but gives up protection.
`BIOMETRIC_ENCRYPTION_KEYS`	`k1:base64…`	The API exits at start-up. Lose it and every stored face template becomes permanently unreadable — students must re-enrol.
`MODEL_DIR`	`../ai_service/models`	Recognition reports the models as missing; enrolment and live scanning fail with an explicit "model unavailable" rather than a simulated result.
`MAX_UPLOAD_BYTES`	`2097152`	The largest accepted frame (2 MB). Raise it and a hostile client can make the server decode huge images.
`SEED_ADMIN_EMAIL` / `SEED_ADMIN_PASSWORD`	development only	Used once by `seed-admin`. Left empty in production; the first real administrator is created interactively instead.

The one rule about `.env`

It contains the database password, the biometric encryption key and the seed administrator's password. It is listed in `.gitignore`, it is excluded from Docker images, and it is never pasted into a chat, a screenshot or a report. The

repository ships `.env.example` — the same keys with every secret replaced by `CHANGE_ME` — so that a reader knows what to fill in without ever seeing a real value.

16.4 Step two — get the AI models

The model files total roughly 40 MB. They are third-party artefacts with their own licences, so they are **downloaded at install time** and never committed to Git or baked into an image. The fetch script downloads each file and verifies its SHA-256 checksum before accepting it.

```
python ai_service/scripts/fetch_models.py           # YuNet + SFace, with checksum verification
python ai_service/scripts/fetch_models.py --check   # re-verify later, after a copy or a backup restore
```

Resulting files in `ai_service/models/`:

File	Size	Role
`face_detection_yunet_2023mar.onnx`	227 KB	Finds faces and five landmarks (two eyes, nose tip, two mouth corners). Tiny and fast — it runs on the CPU for every frame.
`face_recognition_sface_2021dec.onnx`	37 MB	Turns an aligned face into 128 numbers — the "fingerprint" that is compared and stored. This file is the bulk of the download.
`minifasnet_v2_2.7_80x80.onnx`	1.7 MB	**Experimental** Liveness check. Optional. Converted once from the upstream Apache-2.0 checkpoint; absent by default, and its absence produces `not_enabled`, never a false pass.

ONNX

Open Neural Network Exchange — a file format for trained machine-learning models. Its value here is portability: the same `.onnx` file runs through *onnxruntime* on Windows, Linux or a container, with no GPU, no CUDA, no framework install and no C++ compiler. That single decision is why this project can run on an ordinary college laptop.

16.5 Step three — the backend

```
cd backend
python -m venv .venv                              # create an isolated Python environment
.venv/Scripts/pip install -r requirements-dev.txt  # install pinned dependencies
.venv/Scripts/alembic upgrade head                 # create/upgrade every table
.venv/Scripts/python -m app.cli seed-admin         # development: first administrator from .env
.venv/Scripts/python -m uvicorn app.main:app --reload --port 8000
```

Each line deserves one sentence of explanation, because each one fails in a different way:

Command	What it does, in plain terms
`python -m venv .venv`	Creates a **virtual environment**: a private folder holding its own copy of Python's package installer and, later, its own packages. Without it, installing this project's dependencies would pollute the whole machine and could break another project that needs a different version of the same library.
`pip install -r requirements-dev.txt`	Installs the exact versions listed in the requirements file — FastAPI, SQLAlchemy, Pydantic, psycopg, Argon2, cryptography, OpenCV, onnxruntime, numpy, plus pytest and its plugins. Pinning versions is why a machine that works today still works in six months.
`alembic upgrade head`	Applies every database **migration** in order, up to the latest one ("head"). On an empty database this creates all 17 tables; on an existing one it applies only what is missing. This is how the schema evolves without anyone hand-editing SQL.
`python -m app.cli seed-admin`	Reads `SEED_ADMIN_EMAIL` and `SEED_ADMIN_PASSWORD`, hashes the password with Argon2id, and creates one administrator so you can log in. It refuses to run when `APP_ENV=production`.
`uvicorn app.main:app --reload`	Starts the API. `app.main:app` means "the object named `app` inside `app/main.py`". `--reload` watches the source and restarts on every save; it is for development only and is deliberately absent from production.

The API documentation is then at `http://127.0.0.1:8000/api/docs`. Uvicorn itself is the **ASGI server** — the program that speaks HTTP to the outside world and hands each request to the FastAPI application. FastAPI defines behaviour; uvicorn makes it reachable.

16.6 Step four — the frontend

```
cd frontend
npm install      # install the packages listed in package.json
npm run dev      # Vite dev server on http://localhost:5173
```

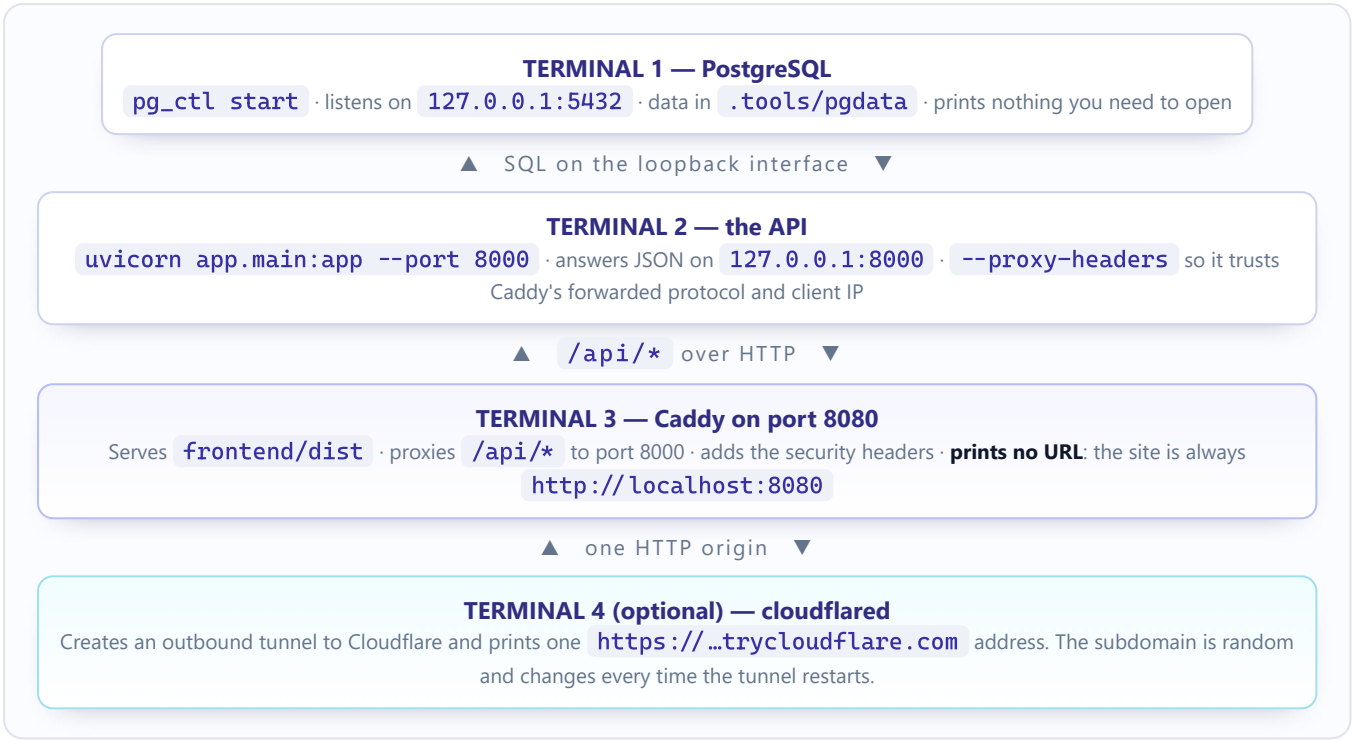
Open `http://localhost:5173`. The Vite dev server rewrites any request beginning `/api` to `http://127.0.0.1:8000`, so from the browser's point of view there is only one origin. That is exactly the arrangement production uses, which means the app behaves the same in both — with one Windows-specific exception:

> **Use `localhost`, not `127.0.0.1`, for the dev server**
>
> On this machine Vite binds to the IPv6 loopback address. Opening `http://127.0.0.1:5173` (IPv4) produces a connection refused error that looks like the server is down when it is running fine. `http://localhost:5173` resolves correctly. The production URL, `http://localhost:8080`, is served by Caddy and has no such quirk.

16.7 The other production shape used here: laptop + Caddy + tunnel

Docker is the textbook deployment, and section 16.9 covers it. The deployment that was actually used for the live demonstration is simpler and costs nothing: run the same three processes directly on the laptop, put **Caddy**

in front of them as the reverse proxy, and expose the result through a **Cloudflare Quick Tunnel** when a public link is needed.



The exact commands, run from the folder that contains both `smartattend-ai/` and `.tools/`:

```
# 1 — database (leave this window open)
.tools/pgsql/bin/pg_ctl.exe -D .tools/pgdata -l .tools/pg.log ^
  -o "-p 5432 -c listen_addresses=localhost" -w start

# 2 — API, in production mode, from backend/
.venv/Scripts/python -m uvicorn app.main:app --host 127.0.0.1 --port 8000 ^
  --proxy-headers --forwarded-allow-ips=127.0.0.1,::1

# 3 — web server, from .tools/ (serves ../smartattend-ai/frontend/dist)
caddy run --config Caddyfile --adapter caddyfile

# 4 — optional public link
cloudflared tunnel --url http://localhost:8080 --no-autoupdate
```

WHY `--proxy-headers` IS NOT OPTIONAL

Behind Caddy, every request appears to come from `127.0.0.1`, and every request appears to be plain HTTP. Without `--proxy-headers` and `--forwarded-allow-ips`, the API would rate-limit *everyone as one IP address* and would believe an HTTPS request was insecure — which breaks secure-cookie logic and makes the access log useless. The flag means "trust these specific upstream addresses when they tell you the real client IP and scheme", and it is restricted to the loopback addresses so a remote client cannot spoof them.

Which terminal prints which URL

Caddy prints no URL at all. Do not go looking for one in its output — the local site is simply `http://localhost:8080`. The only program that prints an address to open is `cloudflared`, and its address is

a random `https://<three-words>.trycloudflare.com` hostname that changes on every restart. If the public URL changes, `FRONTEND_BASE_URL` in `.env` must be updated and the API restarted, or password-reset links will point at the dead host.

A Quick Tunnel is a genuinely free way to get an HTTPS URL, but it is a demonstration tool, not infrastructure: the hostname is not chosen by you, cannot be renamed, has no uptime guarantee, and the laptop must stay awake for the link to work.

16.8 Stopping everything, in the right order

Shutdown order matters because each layer expects the one below it to still exist. Shutting the database down first turns every in-flight API request into a 500 error and can leave connections dangling.

1. **Stop the API** (Ctrl+C in its terminal, or end the process if it was started detached).
2. **Stop Caddy.** It can no longer proxy to anything, so stop it next.
3. **Stop the tunnel.** Otherwise Cloudflare keeps sending requests to a dead port.
4. **Stop PostgreSQL *last***: `.tools/pgsql/bin/pg_ctl.exe -D .tools/pgdata stop -m fast`. The `fast` mode disconnects clients, rolls back open transactions and shuts down cleanly — it does not skip the write-ahead log, so committed data is safe.

Nothing is lost by shutting down

All application state lives in PostgreSQL, whose files are on disk under `.tools/pgdata`. Sessions are rows; face templates are encrypted rows; audit entries are rows. Stopping the processes ends the service, not the data. Starting the four processes again restores the exact state — including the model templates, so students do not re-enrol.

16.9 The containerised deployment

The repository also ships a three-service Compose stack, which is what "deployment" normally means: the same thing, packaged so that a server needs only Docker.

Service	Image / build	Responsibility and exposure
`db`	`postgres:16-alpine`	Stores everything in a named volume `pgdata`. No ports published — unreachable from outside the Compose network. Health-checked with `pg_isready`.
`api`	built from `backend/Dockerfile`	Runs `alembic upgrade head` then uvicorn. Models are mounted read-only from the host, never copied into the image. Starts only when the database reports healthy. Health check calls `/api/health` and expects `status: ok`.
`web`	built from `frontend/Dockerfile` (Node build stage → nginx)	Serves the compiled SPA and proxies `/api` to `api:8000`. **The only service with a published port** (`WEB_PORT`, default 8080).

```
docker compose up -d --build      # build images and start in the background
docker compose ps                 # wait until api and web both say "healthy"
docker compose exec api python -m app.cli create-admin --email you@example.edu --name "Your Name"
curl -fsS http://localhost:8080/healthz
curl -fsS http://localhost:8080/api/health
```

> **CONTAINER, IMAGE, VOLUME, HEALTHCHECK**
> An **image** is a frozen filesystem plus a start command — the recipe. A **container** is one running instance of that image. A **volume** is storage that lives outside the container, so deleting and recreating the container does not delete the data; `docker compose down` keeps `pgdata`, while `down -v` destroys it. A **healthcheck** is a command Docker runs inside the container on a timer; other services can be told to wait for `condition: service_healthy`, which is why the API does not crash while PostgreSQL is still starting.

There is no default account and no password in the repository, so the first administrator is created interactively against the running container. That command prompts for the password twice because it works in production, where a seeded password from `.env` would be a security hole.

16.10 HTTPS in front of the stack

The bundled nginx does **not** terminate TLS. Your institution's reverse proxy — Caddy, Traefik, or a gateway — does, and then two settings must change:

```
PUBLIC_URL=https://attendance.example.edu
COOKIE_SECURE=true
```

Then restart the API. `COOKIE_SECURE=true` marks the session and CSRF cookies HTTPS-only, so a browser will never send them over plain HTTP and a downgrade attack cannot steal a session. Leave it `false` only for a local HTTP demonstration. Never publish the `api` or `db` ports directly.

16.11 Backups, restores and upgrades

Two things must be backed up, separately and both encrypted: the **database** and the **biometric encryption keys**. Backing up the database without the keys produces a file full of undecryptable template rows; backing up the keys without the database produces nothing at all. Backing them up *together* defeats the point of encrypting templates at rest.

```
# logical backup in PostgreSQL's custom compressed format
pg_dump -Fc -U smartattend -d smartattend -f backup-2026-10-28.dump

# restore into an empty database
pg_restore -d smartattend --clean --if-exists backup-2026-10-28.dump

# inside Compose
docker compose exec -T db pg_dump -U smartattend smartattend > smartattend-$(date +%F).sql
```

> **LOGICAL VS PHYSICAL BACKUP**

> A **physical** backup copies the database files byte for byte; it is fast and exact but tied to the same PostgreSQL major version and platform. A **logical** backup (`pg_dump`) writes SQL or a portable archive of the data and schema; it is slower and larger but restores into a different version or a different machine. `-Fc` chooses the custom archive format, which is compressed and can restore individual tables.

Upgrading is: take a backup, rebuild the changed images (`docker compose build api web`), and bring the stack up (`docker compose up -d`). Migrations run automatically at start-up. One honest warning belongs here: **rolling the code back does not roll the database back.** A migration that dropped a column cannot be undone by checking out the previous commit; you restore the backup, or you write a forward migration that repairs the data.

16.12 The retention job

Privacy commitments expire, so a scheduled job must enforce them. Students deactivated longer than the configured retention period (`biometric_retention_days` , default 30) must have their templates deleted and their consent withdrawn.

```
python -m app.cli purge-biometrics --dry-run   # prints how many would be affected; changes
nothing
python -m app.cli purge-biometrics --yes       # actually deletes; prints counts only
```

The command prints counts, never names, e-mails or template bytes, so its output is safe to keep in a log. Run the dry-run first, then schedule the second command — for example as a protected scheduled task on the host, or a cron job in the container — and record the outcome in the institution's retention log.

16.13 Troubleshooting: symptoms and their real causes

Symptom	What is actually happening
API exits immediately at start-up	Almost always a missing or malformed `BIOMETRIC_ENCRYPTION_KEYS` (the format is `key_id:base64` of exactly 32 bytes) or wrong database credentials. The process fails loudly on purpose — a server that starts without encryption keys would be worse.
Login succeeds, then the next click is unauthenticated	The cookie was not stored or not sent. Over HTTPS with `COOKIE_SECURE=false` it is still sent; over HTTP with `COOKIE_SECURE=true` the browser silently drops it. Check the scheme first, the cookie flags second.
Every state-changing request returns `403 csrf_failed`	The `X-CSRF-Token` header is missing or does not match the `sa_csrf` cookie. A common cause in scripts is parsing the `Set-Cookie` header incorrectly: split on `;` *before* splitting on `=` , otherwise the "token" includes `; Path=/` .
`422 weak_password` when an admin sets a password	The policy requires at least 10 characters and at least one letter and one digit. The error lists every failed rule, not just the first.
Recognition says the models are missing	`MODEL_DIR` points somewhere else, or `fetch_models.py` was never run on this machine. In Compose the model directory is mounted read-only, so downloading inside the container is not the fix.

Symptom	What is actually happening
Camera never opens, or permission is reported "denied" with no prompt	An embedded or automation browser view can refuse camera access without asking. Perform enrolment and live scanning in a normal Chrome or Edge window after allowing the camera for the site. This is a browser-permission issue, not an application bug.
`413` when enrolling or scanning	The request body exceeded the limit at two places: `MAX_UPLOAD_BYTES` in the API and `client_max_body_size` in nginx. Both must allow the frame.
Reset links point at `localhost`	`FRONTEND_BASE_URL` (or `PUBLIC_URL` under Compose) still holds the development value. Change it to the real origin and restart the API.
A 60-minute lecture stops accepting attendance after 30 minutes	Not a bug: `attendance_closes_at` is `scheduled_start + window_min`, measured from the *start*. Raise `window_min` when scheduling if the class should stay open longer.
Test suite refuses to start	`TEST_DATABASE_URL` is unset or equal to `DATABASE_URL`. The guard exists so a test run can never truncate the development database.
The frontend suite fails randomly on a slow machine	jsdom set-up exceeds the default 5-second per-test timeout. `vite.config.ts` raises it to 30 seconds. A timeout on a slow CPU is not the same thing as a race condition — but both look identical until you read the failure.

CHAPTER 17

How this project was built

The actual order of work, including the parts that were changed after the "finished" version, and the defects that were found by running it.

17.1 Nine phases, each with something demonstrable at the end

The project was built in nine phases. The organising rule was that no phase ended with "mostly done": each one produced something that could be started, clicked and checked. This matters because a face-recognition system has an unpleasant property — its parts are individually easy to fake. A login screen that does not authenticate anything still looks like a login screen. A "recognition" button that returns a random result still looks like recognition. Building in vertical slices forces every interface to be genuinely connected early.

Phase	Scope	What could be demonstrated at the end of it
1	Architecture, schema, API contract, model and UI design documents	Twelve design documents: the SRS, the 17-table schema, the endpoint list, the AI-model choice with its licence review, and wireframes. The decisions that are expensive to change later were made here, on paper.
2	Database, authentication, CSRF, RBAC, accounts, settings, audit	You could log in as an administrator, be rejected with the wrong password, be locked out after repeated failures, and see an audit entry for the account you created.
3	React shell, login page, route guards, account pages, admin user console	A real interface with a sidebar that hides what your role cannot use — and an API that refuses those things anyway.
4	Students, faculty, courses, sessions, attendance records	The complete academic skeleton: create a course, put students on it, schedule a lecture, mark attendance by hand.
5	Recognition engine, enrolment, live camera	Enrol a face, then open a session camera and have a name appear with a similarity score.
6	Liveness integration, exception review, corrections	An uncertain match becomes a review item instead of a mark; a student can dispute an attendance record.
7	Dashboards, reports and exports	Charts that show real aggregates, and CSV/PDF exports with formula-injection defence.
8	Deployment, broader test coverage	Docker images, a Compose stack with health checks, and a test suite covering every module rather than the happy path only.
9	SRS, diagram pack, evaluation harness, report material, viva preparation	The documents a degree project is judged on — plus the measurement tooling needed to answer "how accurate is it?" truthfully.

VERTICAL SLICE

A thin feature cut through every layer of the system: database column, service rule, API route, frontend screen. The opposite is a **horizontal** build — all the database first, then all the services, then all the screens — which feels

> efficient and delays every integration problem to the end, when they are most expensive. The nine phases above are vertical slices with widening scope.

17.2 The verification habit

Each phase ended the same way: run the tests, build the frontend, and exercise the feature in a real browser. Two habits came out of that and stayed:

- **Never trust a green build as proof of a working feature.** TypeScript proves types line up; it cannot tell you a form posts to the wrong endpoint or that a table renders an empty state on a successful response.
- **Prefer the real database to a mock.** The backend tests run against a separate PostgreSQL database, not an in-memory substitute, because half of this system's guarantees are database constraints — the `NOT NULL` consent foreign key, the window `CHECK`, the `ON CONFLICT` upsert. A mock would happily accept code that the real schema rejects.

17.3 What changed after the first "finished" version

The nine phases produced a working system. Then it was actually used, and using it changed it. These are the changes that came afterwards, in the order they were made — they are part of the design, not noise around it.

1. Course → students → session became one flow

What was wrong: creating a course, adding students, and scheduling a lecture were three separate screens you had to know to visit in the right order. Nothing forced a relationship between them, so it was possible to schedule a session for a course with an empty roster and get a camera that recognised nobody.

What changed: creating a course opens its roster, every course row gained a **Schedule** action, and the roster dialog gained **Schedule class** (title, date, start, end) which jumps straight to the new session. More importantly, changing a course's enrolment now *syncs into that course's open sessions* — a student added after the session was scheduled is still recognised during it. Finalised and cancelled sessions keep their frozen roster, because history must not silently rewrite itself.

> **THE DESIGN TENSION IN ONE SENTENCE**
>
> Live sessions must follow the current roster; closed sessions must preserve the roster they had. That is why `session_participants` is a snapshot that is re-synced only while the session is `scheduled`, `active` or `paused`.

2. Administrators can set a password directly

What was wrong: a student who had never signed in could only be helped by issuing a 24-hour single-use reset link and waiting for them to use it. On a demonstration day that is a queue.

What changed: an admin can set any user's password from Admin → Users, and any student's password from Admin → Students → Password. For a student without a portal account, setting a password creates the account in the same action. Reset links still exist and still cancel each other; they are simply no longer the only path. The admin-set value passes through the same strength policy and the same Argon2id hashing — the shortcut is operational, not cryptographic.

3. Every password field got a show/hide toggle, and the admin types the branch

What was wrong: only the login page had the eye button, so the password an administrator typed *for someone else* — the one they then had to read aloud — was invisible. Separately, the Add Student and Add Faculty forms pre-filled `Computer Science` and `B.Sc. Computer Science`, quietly asserting a branch on behalf of an institution that had not said so.

What changed: a shared `PasswordField` component with an accessible `aria-label` replaced the bespoke one, and the department/program fields now start empty with a greyed example, refusing to submit with a plain-language message. The general lesson is worth stating: a default value in a form is a claim about the world, and most of them are false.

4. Students can mark themselves present, and staff can switch that off per student

What was requested: let a student mark their own attendance during the session's own time slot, with the ability to close it for a specific student who abuses it — attends for a minute and leaves.

What was built: an opt-in switch per session (admin only), a per-student lock (admin or the course's faculty), and a self-marking endpoint that is honest about what it is:

- **Off by default.** A session does not accept self-marking unless an admin enabled it.
- **Time-bounded.** Open only between `scheduled_start` and `attendance_closes_at`; `present` vs `late` comes from `late_after`, exactly as with every other source.
- **Labelled.** The record is written with `source = self_reported`, so nobody can later mistake it for a camera match.
- **Non-destructive.** It never overwrites an existing record — a camera match, a manual mark or a staff decision always wins.

The student screen also had to explain absence of a button, so `GET /sessions/mine` returns `self_marking_open`, `self_marking_message` and `self_marking_closes_at` — "why can't I see the button" is a support question this design answers in the payload.

5. The interface was deliberately made less flat

What was wrong: the first UI worked and looked like a settings page. For a demonstration to an audience, a system that looks unfinished invites doubt about whether it *is* finished.

What changed: four shared CSS utilities rather than decorative one-offs — `.panel` (frosted surface, inset highlight, layered coloured shadow), `.panel-edge` (a two-pixel gradient hairline), `.panel-lift` (hover lift), `.press-3d` (press response) — plus aurora background gradients, a dark sidebar, gradient-clipped numbers and gradient chart fills. Because these are *utilities*, a change to the depth language changes every screen at once, which is the difference between a design system and a pile of styles.

17.4 Defects that only showed up when the thing was running

Three bugs from this period are worth recording, because each one is a category rather than an incident.

Defect	Cause	Lesson
A closed session appeared under "Upcoming" for students	The student page grouped sessions by comparing the scheduled end time with now. A session closed early but still *scheduled* in the future has an end time in the future, so it was filed as upcoming — but it could never be joined.	Group by **status first**, then by time. Time is a proxy for state, and the proxy broke. A regression test now asserts the exact case: closed, end in the future, must not appear as upcoming.
Chart gradients silently rendered as flat colour	Recharts renders its children inside an SVG tree; a `<defs>` block placed inside that tree was not resolvable, so `fill="url(#…)"` fell back to nothing.	Library-internal DOM assumptions are not obvious from the component API. The fix was a sibling `<svg>` holding the definitions, referenced by id.
The attendance window closed halfway through a 60-minute lecture	Nothing was broken — the documentation and the reader's mental model were. `window_min` is measured from the session *start*, so the default 30 closed attendance at 10:30 for a lecture ending at 11:00.	When a correct system surprises its operator, the fix is in the wording and in explicit values, not in silencing the behaviour. The docs now say "measured from the start" and the demo sessions set `window_min` explicitly.

17.5 Publishing, and the secret audit that preceded it

Publishing the repository was treated as a security event rather than a formality, because this project's worst failure mode is not a broken build — it is a committed encryption key or a committed face image. Before the first push:

1. **A local repository was initialised and `.gitignore` verified** before any file was staged, so `.env`, the model weights, the database directory, the virtual environments and generated biometric data were never in scope.
2. **The staged tree was audited for real secret values.** Every live secret from `.env` — the database password, the encryption key, the seed admin password — was searched for across all files that would be pushed. Result: zero occurrences.
3. **Third-party artefacts were checked for licence compatibility.** YuNet is MIT, SFace and MiniFASNet are Apache-2.0; the weights are downloaded at install time and are not in the repository. The project's own code is released under the MIT licence, and the README states the distinction explicitly so nobody assumes the MIT grant covers the models.
4. **The remote was created and `main` pushed** to `github.com/Abhishek-stacked/smartattend-ai`, then the public page was checked — the licence badge, the README rendering, the file tree.

> **Why the secrets audit is reproducible, not reassuring**
>
> "I don't think I committed anything sensitive" is a feeling. Searching the exact secret strings across the exact set of files to be pushed is a test, and it has a number as an answer. The same habit applies to screenshots, reports and the demonstration database export: secrets travel through documents far more often than through code.

17.6 What would be done differently

- **Measure before polishing.** The evaluation harness should have existed before the threshold was chosen; instead the threshold was a conservative design choice and the harness arrived in phase 9. Accuracy numbers should drive the operating point, not the other way round.
- **Write the "what this does not do" section first.** The limitations chapter was the easiest to write and the most valuable to read. Writing it at the start would have prevented at least one round of over-claiming in the UI.
- **One configuration surface from day one.** The window semantics, the timezone handling and the reset-link base URL all caused confusion for the same reason: the value existed in more than one place in the operator's head.
- **Test the grouping and the empty states.** The bugs that reached a human were almost all "correct data, wrong place on screen" — the category of defect that unit tests of services never catch.

CHAPTER 18

Limitations, honest answers and next steps

What this system demonstrably does, what it does not, and how to describe the difference without either overselling or apologising.

18.1 Why limitations get their own chapter

A project that lists only its features is telling you what it hopes. A project that lists its limits is telling you what it knows. For a biometric system the second is more important than the first, because the failure modes are asymmetric: a wrong attendance mark is an inconvenience, while an over-confident identity claim in a high-stakes setting is a harm. The same applies to a demonstration: an examiner who finds an unmentioned limitation trusts nothing else you said.

> **CLAIM VS EVIDENCE**
>
> A **claim** is a sentence about behaviour: "it recognises faces", "it detects spoofing". **Evidence** is something you can reproduce that would be different if the claim were false: a test that fails when the code changes, or a measured number on a described dataset. This project separates the two deliberately — features are claims backed by code, accuracy is a claim that currently has *no* evidence, and the documentation says so in those words.

18.2 What is measured, and what is merely implemented

Statement	Status	What would turn it into evidence
The API enforces role permissions on every endpoint	Verified	Already evidenced: the test suite asserts 403/404 for each forbidden role, and the code path is single-sourced in `api/access.py`.
Templates cannot exist without consent	Verified	Evidenced by a database constraint, not by willpower: a `NOT NULL` foreign key rejects the insert.
Copied template rows fail to decrypt	Verified	Evidenced by AEAD associated data binding ciphertext to student, model name and model version.
Recognition reaches a stated accuracy	Unmeasured	A consented evaluation set and the harness in `ai_service/evaluation/`: false-accept and false-reject rates at the chosen threshold.
Liveness resists presentation attacks	Unmeasured	A documented attack set (printed photo, phone screen, replay video) with per-attack pass rates. None of this exists yet.
The system is fair across demographics	Unmeasured	Disaggregated error rates by group on a consented dataset. Not attempted, and therefore not claimed.
The limits hold under multiple API workers	Not supported	A shared counter store (Redis) or per-worker sharding. In-process rate limiting is correct only for a single worker.

Statement	Status	What would turn it into evidence
The Docker stack has been run end to end	Static only	A real `docker compose up` on a machine with Docker available. On the machine where this was built, Docker was unavailable, so Compose was validated by reading, not by running.

Read that last row again

It is tempting to write "deployment tested". It was not. The images, health checks, mount paths and environment wiring were reviewed, and the equivalent arrangement was run directly on the host — but the containerised path itself is unexecuted. Stating that plainly costs one sentence and prevents a much worse conversation later.

18.3 Liveness: the most important limitation

Liveness detection asks: is the face in front of the camera a live person, or a presentation of one — a printed photograph, a phone screen, a recorded video? This project integrates MiniFASNet as an **optional, experimental** signal with three policies: off, flag, and enforce.

- **A single RGB frame is weak evidence.** The model sees one image at one instant. High-quality replay on a good screen can pass; a genuine face in poor light can fail. Both directions of error are real.
- **Off by default is the honest default.** Turning on enforcement by default would convert an unmeasured model into a gate that blocks real students.
- **No pass is ever fabricated.** If the model file is absent or errors, the result is `not_enabled` or an explicit unavailability — never a silent success.
- **Human supervision is the actual control.** The system's defence against impersonation in a classroom is a member of staff present who can see who is standing at the camera, plus an audit trail. Liveness is a hint, not an identity proof.
- **It must never be the sole basis for a high-stakes decision.** Anything consequential — an exam, a disciplinary matter — needs a human process.

18.4 Security limitations

Not implemented	Consequence, stated plainly
Multi-factor authentication	A stolen password is enough to sign in, subject to lockout and rate limits. For staff accounts with access to biometric data, MFA is the single highest-value addition.
SSO / institutional identity	Accounts are local to this application. There is no federation, so account lifecycle is manual.
Tenant isolation	One institution per deployment. Two colleges cannot share one instance without seeing each other's data; the correct answer is two deployments.
Field-level encryption of attendance metadata	Templates are encrypted; names, roll numbers and attendance rows are not. A database dump exposes who attended what.
Tamper-evident audit log	Audit rows are append-only *through the application*. A database administrator can modify them. External log shipping or hash chaining would fix this.

Not implemented	Consequence, stated plainly
Distributed rate limiting	The counter lives in process memory. One worker: correct. Several workers: each has its own budget, so effective limits multiply.
Web application firewall	No WAF, no bot management. Protection is upload limits, rate limits, strict headers and validation — good hygiene, not a shield.
Automatic key rotation tooling	The keyring format supports rotation (first key encrypts, all keys decrypt), but no migration command re-encrypts existing rows. Rotating the only key without a plan makes templates unreadable.

18.5 Privacy limitations

- **No automated data-subject request workflow.** Access and correction are possible through the UI; a formal DSAR process with deadlines and logging is an institutional procedure, not a feature here.
- **No breach-notification tooling.** The incident plan exists as guidance in the documentation; executing it is human work.
- **No consent re-prompt on policy change.** Consent is versioned and stale versions are rejected at enrolment, but existing students are not asked to re-consent when the wording changes. They will be asked the next time they enrol.
- **Attendance history survives biometric withdrawal.** That is deliberate — it is academic record, not biometric data — but it means "delete my face" is not "delete my record", and the distinction must be explained to participants rather than buried.
- **Institutional responsibility is not transferred.** The software provides mechanisms; the lawful basis, the notice, the retention schedule and the regulator relationship belong to the deploying institution. The policy text shipped in `biometric_policy.py` is a template, not legal advice.

18.6 Operational limitations

Constraint	What it means in practice
The demonstration laptop must stay awake	The free deployment runs on the machine itself plus a tunnel. Sleep, a closed lid or a lost Wi-Fi connection takes the public URL down. There is no always-on free alternative that was acceptable here.
The public hostname is random and temporary	A Cloudflare Quick Tunnel prints a new `trycloudflare.com` address each time it starts, and that address cannot be renamed. Password-reset links embed the base URL, so they must be regenerated after a tunnel change.
One API worker	Recognition is CPU-bound and the cooldown/rate limit are in-process. Throughput is roughly one recognition at a time per worker; a classroom of 60 students is a queue, not a burst.
CPU-only inference	No GPU path. That was a deliberate constraint (works on an ordinary laptop, no CUDA, no build tools) and it caps performance. The numbers in the performance section are design estimates, not measurements.

Constraint	What it means in practice
No mobile application	The web app is responsive, and the camera API works in mobile browsers, but there is no native app, no offline mode and no background sync.
No face login	Face recognition is for *classroom attendance only*. Sign-in is e-mail plus password. This is a frequent misunderstanding worth correcting early: adding biometric authentication would be a different threat model and a different consent.
Camera access depends on the browser	Enrolment and live scanning need a real browser window with camera permission. Embedded or automation views may refuse without prompting, which looks like a broken app and is not one.

18.7 Questions you will be asked, and honest answers

> **"HOW ACCURATE IS IT?"**
>
> "The pipeline is implemented and the threshold is a conservative design choice; accuracy has not been measured yet. The evaluation harness exists so that the next step is measurement rather than argument. Until then I will not quote a percentage."

> **"CAN A PHOTOGRAPH FOOL IT?"**
>
> "A single-frame RGB liveness model can be fooled by a good replay, and it can also reject a real face in bad light. It is shipped as experimental and off by default. The real control in a classroom is supervision: a staff member is present, uncertain matches go to a human review queue, and no uncertain identity is ever marked automatically."

> **"WHAT IF IT MARKS THE WRONG STUDENT?"**
>
> "Three mechanisms make that unlikely and one makes it recoverable. A similarity threshold rejects weak matches; an ambiguity margin rejects two candidates that are too close; a low-confidence band sends the event to review instead of marking. And every record is visible to the student, who can raise a correction request — the correction is audited, and the original value is preserved in the audit snapshot."

> **"IS THIS LEGAL?"**
>
> "The software implements consent capture, data minimisation, retention and access control. Whether a particular deployment is lawful depends on the institution's basis, notice and jurisdiction — for example India's DPDP Act or the GDPR. The included policy text is a template, and the deploying institution remains responsible for it. I can describe the mechanisms; I cannot give a legal opinion."

> **"WHAT HAPPENS IF THE LAPTOP DIES MID-LECTURE?"**
>
> "Attendance already written is committed in PostgreSQL and survives. Sessions in progress can be resumed; a session can be reopened by an administrator with a reason, and that action is audited. What is lost is the live camera stream, not the data — and staff can mark attendance manually while the system is down."

> **"WHY NOT JUST USE AN EXISTING ATTENDANCE APP?"**

> "Because the interesting part is not the CRUD — it is the boundary between a model's opinion and an institutional decision. Building it means every uncertain match, every consent record and every correction is explicit and auditable. That is also what makes it a project worth assessing rather than a product to buy."

> **"WHAT IS THE MOST IMPRESSIVE PART?"**
> "The refusal to fake things. Missing models produce an explicit unavailability, uncertain matches produce a review item, unmeasured accuracy produces a blank where a number would be, and absent liveness produces `not_enabled`. Every shortcut that would have made the demo look better was a lie about behaviour, and those were the ones deliberately not taken."

18.8 Next steps, in the order they are worth doing

#	Change	Effort	Why it comes at this position in the list
1	Measure recognition: build a consented evaluation set and run the harness to select a threshold	A few days	Everything else is tuning. Until false-accept and false-reject rates exist, the 0.42 threshold is a guess with good reasoning behind it, and no claim about accuracy can be made.
2	Evaluate liveness as an attack study: printed photo, screen replay, video replay, per-attack pass rate	A week	Second because it is the only way the experimental label can ever be removed. Without it, "enforce" mode is a coin-flip that can lock out real students.
3	MFA for administrator and faculty accounts	Days	Highest-value security addition: staff accounts can see and delete biometric data, and passwords are currently the only factor.
4	Shared rate-limit and cooldown store, enabling more than one API worker	Days	Removes the throughput ceiling honestly, instead of silently multiplying the limits by adding workers.
5	Key-rotation command that re-encrypts templates under a new key id	Days	Turns the documented rotation procedure into an executable one, which matters the first time a key is suspected compromised.
6	Hash-chained or externally shipped audit log	Days	Closes the "a DBA can edit history" gap. Valuable, but only after the roles that write the log are themselves protected by MFA.
7	Consent re-prompt when the policy version changes	Hours	Small, and it converts a privacy principle into something the system actively enforces rather than something a human remembers.
8	Run the Compose stack on a machine with Docker and record the result	Hours	Cheap to do and it upgrades one documented "static only" caveat into a fact.
9	Accessibility and mobile pass: focus order, zoom, small-screen camera flow	Days	Quality of life for the students who will actually use it on a phone at a classroom door.

18.9 How to describe this project without overselling it

One sentence that survives scrutiny, because every clause is checkable:

> **A DEFENSIBLE SUMMARY**
>
> "A full-stack attendance system where facial recognition proposes a match, an explicit rule set decides, and a human resolves anything uncertain — with consent, encryption, audit and retention designed in, and accuracy and liveness explicitly not yet measured."

What to avoid: "AI-powered accurate face recognition" (the accuracy is unmeasured), "anti-spoofing secured" (experimental, single-frame), "enterprise-grade" (single institution, no MFA, one worker), "GDPR compliant" (a mechanism is not a compliance certification). Each of those swaps a checkable claim for an uncheckable one, which is exactly the trade this project refuses to make everywhere else.

CHAPTER 19

Appendices

Lookup material: where each thing lives, every configuration value, every command, every error code, and the ten ideas worth carrying away.

A. Where to look for what

File	What it holds
`backend/app/main.py`	Application factory: middleware order, router mounting, security headers, start-up checks.
`backend/app/core/config.py`	Every setting, its default and its validation — including the window and timezone checks that fail at start-up rather than at 09:00.
`backend/app/core/security.py`	Argon2id hashing, password-strength rules (≥10 characters, letter and digit), token generation and comparison.
`backend/app/core/crypto.py`	AES-256-GCM template encryption, the associated-data binding, and the keyring parsed from `BIOMETRIC_ENCRYPTION_KEYS`.
`backend/app/core/errors.py`	The error envelope and the exception handlers that produce it for application, validation, HTTP and unexpected errors.
`backend/app/core/ratelimit.py`	The in-process sliding-window limiter and the `rate_limit` dependency factory.
`backend/app/core/database.py`	Engine, session factory, the `get_db` dependency and transaction boundaries.
`backend/app/api/deps.py`	Session resolution *from the cookie only*, CSRF verification, `current_user`, role gates, pagination.
`backend/app/api/access.py`	Object-level authorisation: staff scope, student scope, `can_manage_course`, `can_view_session`.
`backend/app/api/routes/*.py`	Twelve route modules: auth, users, students, faculty, courses, enrollment, sessions, attendance, recognition, review, corrections, reports, me, system.
`backend/app/services/*.py`	Thirteen service modules — every business rule, transaction and audit call. Routes are thin by design.
`backend/app/models/*.py`	The 17 tables as SQLAlchemy models, with constraints, indexes and the one deliberate cascade.
`backend/migrations/versions/`	Two migrations: the initial schema, and the student self-marking columns.
`backend/app/cli.py`	`seed-admin`, `seed-demo`, `create-admin`, `purge-biometrics`.
`backend/tests/*.py`	Ten test modules plus the fixtures and the test-database guard in `conftest.py`.

File	What it holds
`ai_service/analyzer.py`	Detection plus alignment: YuNet → five landmarks → 112×112 aligned crop.
`ai_service/matcher.py`	128-float embedding and cosine similarity between two vectors, with normalisation.
`ai_service/quality.py`	Blur and size checks — variance of Laplacian and the minimum face size.
`ai_service/liveness/detector.py`	MiniFASNet preprocessing and the off / flag / enforce policies.
`ai_service/scripts/fetch_models.py`	Download with SHA-256 verification; `--check` to re-verify.
`ai_service/evaluation/`	The measurement harness: dataset loading, recognition metrics, liveness metrics, threshold selection.
`frontend/src/App.tsx`	The complete route tree and where guards wrap it.
`frontend/src/lib/api.ts`	One transport: base URL, credentials, CSRF header on unsafe methods, error-envelope parsing, downloads.
`frontend/src/lib/camera.ts`, `hooks/useCamera.ts`	`getUserMedia` lifecycle and canvas frame capture.
`frontend/src/context/AuthContext.tsx`	Who is signed in, which role, and the cold-load request that resolves an existing cookie.
`frontend/src/components/RequireAuth.tsx`	The client-side route guard — UX only; the server decides.
`frontend/src/components/layout/AppShell.tsx`	Sidebar, navigation filtering by role, page frame.
`frontend/src/components/ui/`	Twelve primitives: Button, Card, Field, Select, Textarea, Switch, Modal, Alert, Badge, Feedback, PageHeader, PasswordField.
`frontend/src/pages/`	Twenty-eight route screens and their colocated tests.
`frontend/src/index.css`	Tailwind v4 `@theme` tokens and the shared depth utilities (`.panel` , `.panel-lift` , `.press-3d` , `.orb`).
`Caddyfile` (workspace)	Port 8080, static root, `/api` proxy with forwarded headers, and the security header block.
`docker-compose.yml`	Three services, one published port, health checks, model mount, migration-on-start.

B. Configuration reference

Variable	Typical value	Meaning
`APP_ENV`	`development`	`development` | `test` | `production` . Controls seeding commands, test-database enforcement and debug behaviour.

Variable	Typical value	Meaning
`APP_TIMEZONE`	`Asia/Kolkata`	Zone used for "today" and session dates. Compose defaults to `UTC` — set it explicitly.
`FRONTEND_BASE_URL`	`http://localhost:5173`	Base for password-reset links. Must match where users actually open the site.
`CORS_ORIGINS`	`http://localhost:5173`	Allowed browser origins. Empty in production, where page and API share one origin.
`DATABASE_URL`	`postgresql+psycopg://…`	Primary database. The `+psycopg` suffix selects the driver.
`TEST_DATABASE_URL`	separate database	Required when `APP_ENV=test`; must differ from `DATABASE_URL`.
`POSTGRES_PASSWORD`	URL-safe random	Compose only: initialises the database container and is spliced into the connection URL.
`SESSION_TTL_MINUTES`	`480`	Absolute session lifetime.
`SESSION_IDLE_MINUTES`	`60`	Idle timeout, regardless of absolute lifetime.
`COOKIE_SECURE`	`false` / `true`	HTTPS-only cookies. Must be `true` whenever TLS is in front.
`BIOMETRIC_ENCRYPTION_KEYS`	`k1:base64…`	Comma-separated `key_id:base64(32 bytes)`. First key encrypts; all decrypt.
`MODEL_DIR`	`../ai_service/models`	Where the three `.onnx` files live.
`MAX_UPLOAD_BYTES`	`2097152`	Largest accepted frame (2 MB).
`PUBLIC_URL` / `WEB_PORT`	`http://localhost:8080` / `8080`	Compose: public origin for reset links, and the one published host port.
`SEED_ADMIN_EMAIL` / `SEED_ADMIN_PASSWORD`	development only	Used by `seed-admin`; leave the password empty in production.

C. Command reference

Command	Purpose
`pg_ctl -D .tools/pgdata -l .tools/pg.log -o "-p 5432" -w start`	Start the portable database and wait until it accepts connections.
`pg_ctl -D .tools/pgdata stop -m fast`	Clean shutdown: roll back open transactions, flush, stop.
`alembic upgrade head`	Apply all pending migrations.
`alembic downgrade -1`	Revert the last migration. Development only; it does not restore lost data.
`python -m app.cli seed-admin`	Create the development administrator from `.env`.

Command	Purpose
`python -m app.cli seed-demo`	Create demo faculty, students, courses and sessions.
`python -m app.cli create-admin --email … --name …`	Interactive administrator creation; works in production.
`python -m app.cli purge-biometrics --dry-run`	Report how many templates are past retention. Changes nothing.
`python -m app.cli purge-biometrics --yes`	Delete those templates and withdraw consent. Prints counts only.
`uvicorn app.main:app --reload --port 8000`	Development API with auto-restart.
`uvicorn app.main:app --port 8000 --proxy-headers --forwarded-allow-ips=…`	API behind a reverse proxy.
`npm run dev` / `npm run build` / `npm run preview`	Vite dev server / type-check plus production build / serve the build locally.
`npm test` / `npm run lint`	Vitest run / oxlint.
`python -m pytest`	Backend suite against `smartattend_test`.
`python ai_service/scripts/fetch_models.py --check`	Re-verify model checksums.
`docker compose up -d --build` / `ps` / `logs -f api`	Build and start / check health / follow logs.
`pg_dump -Fc -f backup.dump` / `pg_restore --clean --if-exists`	Logical backup / restore.

D. Error codes

Every failure returns the same envelope — `{"error": {"code", "message", "details"}}`. The code is for programs, the message is for people, the details are for diagnosis. These are the codes a client or an operator will actually meet.

Status	Code	Meaning
400	`invalid_transition`	The session state machine rejected the move — for example finalising a session that is not closed.
400	`session_not_markable`	Attendance cannot be recorded in this session's current state or outside its window.
400	`session_not_editable`	Only `scheduled` sessions may be edited.
400	`session_cancelled`	A cancelled session accepts no marks and no corrections.
400	`session_not_started`	Corrections can only be raised once the session has begun.
400	`not_eligible`	The student is inactive or is not on this session's roster.

Status	Code	Meaning
400	`course_archived`	Archived courses accept no new sessions or enrolments.
400	`course_has_no_faculty`	Assign a faculty member before scheduling.
400	`cannot_deactivate_self`	An administrator cannot deactivate their own account.
400	`invalid_current_password`	Password change rejected: the current password is wrong.
400	`invalid_reset_token`	The reset link is unknown, expired or already used.
401	`unauthenticated`	No valid session. Sign in.
401	`invalid_credentials`	Generic by design: it never reveals whether the account exists.
403	`forbidden`	Authenticated but not permitted. Staff routes may also return 404 where existence itself is sensitive.
403	`csrf_failed`	Missing or mismatched `X-CSRF-Token`. Reload and retry.
403	`self_marking_closed`	Self-marking is off for the session or locked for this student; the message says which.
404	`not_found`	The resource does not exist *or* is outside your scope.
409	`session_not_active`	Recognition is only accepted while the session is active.
409	`no_session`	The session referenced by a recognition event no longer exists.
413	`file_too_large`	The frame exceeds `MAX_UPLOAD_BYTES`.
415	`unsupported_media_type`	Upload a JPEG or PNG.
422	`validation_error`	The request body failed schema validation; `details` lists each field.
422	`weak_password`	Fewer than 10 characters, or missing a letter or digit. `details` lists every failed rule.
422	`consent_required`	Enrolment without a confirmed, current-policy consent.
422	`no_face` / `multiple_faces`	Exactly one clear face is required per sample.
422	`invalid_image` / `empty_file`	The upload could not be decoded, or was empty.
422	`inconsistent_samples`	The enrolment samples do not appear to be the same person.
422	`model_unavailable`	A required model file is missing or failed to load. Explicit, never a simulated result.
422	`invalid_window`	The late cutoff must fall inside the attendance window.
422	`invalid_time_range`	A session must end after it starts.
422	`no_candidate`	No enrolled student was close enough to propose a match.

Status	Code	Meaning
422	`no_change`	The record already has the requested status.
422	`invalid_setting` / `unknown_setting`	A settings update used an unknown key, wrong type or out-of-range value.
429	`rate_limited`	Too many requests; `details.retry_after_s` says how long to wait.
500	`internal_error`	Unexpected failure. The response carries no stack trace; the log entry carries the request id.

E. The twelve documents

Document	Lines	The question it answers
`docs/srs.md`	421	What was the system required to do, for whom, and under what constraints?
`docs/architecture.md`	155	How are the parts arranged, and why are the boundaries where they are?
`docs/database-design.md`	262	What are the 17 tables, their keys and their invariants?
`docs/api-documentation.md`	222	What are the 78 endpoints, their inputs, outputs and error codes?
`docs/ai-models.md`	93	Which models, which licences, which checksums, which known weaknesses?
`docs/evaluation.md`	159	How would accuracy and liveness actually be measured?
`docs/privacy-security.md`	184	What data is held, for how long, protected how, and against which threats?
`docs/deployment.md`	163	How is it configured, started, backed up, upgraded and diagnosed?
`docs/ui-wireframes.md`	148	What do the screens contain before any code existed?
`docs/diagrams.md`	439	The ER diagram, sequence flows, state machine and deployment topology as one pack.
`docs/report-material.md`	401	The material a written dissertation can quote: scope, method, results, limitations.
`docs/viva-prep.md`	350	The questions likely to be asked, with answers grounded in the code.

F. Ten ideas worth carrying away

1. **Separate the opinion from the decision.** A model outputs numbers; a rule set decides; a human resolves what the rule set flags. Mixing those three is how systems become unfalsifiable.
2. **Put invariants in the database.** Consent, window ordering and roster uniqueness are constraints, not conventions. A constraint cannot be forgotten in a code path nobody reviewed.
3. **Authorise on the server, and authorise the *object*.** Knowing a user is faculty says nothing about whether they may see *this* course.

4. **When in doubt, return 404.** A 403 that confirms existence is an information leak.
5. **Fail at start-up, not at 09:00.** Validate configuration when the process boots; a misconfiguration should be loud before the lecture begins.
6. **Bind ciphertext to context.** Associated data makes a copied row undecryptable, so an insider cannot move a template to another student's record.
7. **Never fabricate a result.** Missing models produce unavailability; unmeasured accuracy produces a blank; absent liveness produces `not_enabled`. An honest gap beats a convincing lie.
8. **Every state change names its author.** Source, actor, reason and timestamps are columns, not conventions — and the audit snapshot keeps the previous value.
9. **A default value in a form is a claim about the world.** Most of them are false; ask, do not assume.
10. **Test the empty state and the ordering.** The defects that reach humans are usually "correct data, wrong place on screen".

G. Where to study next

Each line names a concept and the kind of material that teaches it — no specific links, because the best current source changes. Search for the named resource.

If you want to understand…	Study
HTTP, cookies, CORS, the browser security model	MDN Web Docs — the "HTTP", "Cookies" and "Same-origin policy" sections. Read the actual request and response examples, not summaries.
Web vulnerabilities and how they are categorised	The OWASP Top Ten, then the OWASP cheat sheets for session management, CSRF and password storage.
Password hashing and authenticated encryption	The Argon2 specification's rationale and NIST SP 800-38D for GCM mode. The "why" matters more than the API.
Python web APIs	The official FastAPI tutorial, then SQLAlchemy 2.0's ORM and session documentation.
Relational design	PostgreSQL's official documentation on constraints, indexes and transaction isolation, plus any standard database textbook's chapter on normalisation.
React and TypeScript	The official React documentation (especially "Thinking in React", state, and effects), and the TypeScript handbook's narrowing and generics chapters.
Face recognition	The papers behind the models used: YuNet for detection, SFace for the embedding and its cosine-based matching objective. Read the evaluation sections to learn what "accuracy" means operationally.
Presentation-attack detection	Survey papers on face anti-spoofing; pay attention to the datasets and the attack types, which is exactly what this project has not yet measured.
Privacy engineering	The principle vocabulary of data minimisation, purpose limitation and storage limitation — GDPR Article 5 is the clearest short statement — and your jurisdiction's education-data rules.

If you want to understand…	Study
Testing	pytest fixtures and parametrisation; Vitest and Testing Library's queries-by-role guidance, which pushes tests toward what a user can perceive.

How to read a codebase like this one

Start from the shape, not the details: read the route list, then one route end to end (route → service → model), then the tests for that route. After three routes, the pattern is visible and the rest is variation. The most efficient question is never "what does this file do?" but "what would break if this file were deleted?"

END OF THE GUIDE

"This system exists to answer one question sixty times in a lecture hall: who is here?" Everything else in these pages — the encryption, the state machine, the review queue, the audit trail, the reports — exists so that the answer can be trusted, disputed, explained and forgotten on schedule. The code is the easy part to admire; the decisions about what *not* to claim are the part worth copying.

SmartAttend AI — The Complete Guide
github.com/Abhishek-stacked/smartattend-ai · MIT Licence · 2026